

VB2000

An *ab initio* Valence Bond Program based on the Generalized Product Function Method and the Algebrant Algorithm

Version 3.0

Jiabo Li ¹, Brian Duke ², David W. O. de Sousa³, Rodrigo S. Bitzer³,
and Roy McWeeny⁴

¹ ChemXAI Inc, 53 Barry Lane, Syosset, NY 11791, USA

² Monash Institute of Pharmaceutical Sciences, Monash University
381 Royal Pde, Parkville, Victoria, 3052, Australia

³ Chemistry Institute, Federal University of Rio de Janeiro, Brazil.

⁴ University of Pisa, Pisa, ITALY (May 19th, 1924 – April 29th, 2021)

Date code finalized: September 2021

Date of most recent change in manual: September 30th, 2021

Copyright © 2000-2017 by Jiabo Li, Brian Duke and Roy McWeeny
All Rights Reserved.

Copyright (C) 2018- Jiabo Li, Brian Duke, and Roy McWeeny

This program is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program. If not, see <<https://www.gnu.org/licenses/>>.

Citation of VB2000 and the Theory

Publications with results obtained from using VB2000 should cite the software and the theory in the following way.

Reference for the software:

Jiabo Li, Brian Duke, David W.O. de Sousa, Rodrigo S. Bitzer, Roy McWeeny. VB2000 Version 3.0, ChemXAI Inc., Syosset, NY, 2021.

References for the theory:

Jiabo Li, and Roy McWeeny, "VB2000: Pushing Valence Bond Theory to New Limits", *Int. J. Quantum Chem.*, 89 (2002) 208-216.

Jiabo Li, Brian J. Duke, Thomas M. Klapötke, and Roy McWeeny, "Spin Density of Spin-Free Valence Bond Wavefunction and Its Implementation in VB2000", *J. Theor. Comput. Chem.*, 7 (2008) 853-867.

Peter B. Karadakov, David L. Cooper, Brian J. Duke, and Jiabo Li, "Spin-Coupled Theory for 'N Electrons in M Orbitals' Active Spaces", *J. Phys. Chem. A*, 116, 7238 - 7244, 2012.

Contents

1	Introduction	8
1.1	What is VB2000?	8
1.2	What is new in VB2000 version 3.0?	10
1.3	What was new in VB2000 version 2.7?	11
1.4	What was new in VB2000 version 2.6?	11
1.5	What was new in VB2000 version 2.5?	12
1.6	What was new in VB2000 version 2.1?	13
1.7	What was new in VB2000 version 2.0?	13
1.8	What was new in VB2000 version 1.8(R2)?	14
1.9	What was new in VB2000 version 1.8?	14
2	Installation Guide	17
2.1	Download VB2000 packages	17
2.2	Stand alone VB2000 on a UNIX-like platform	18
2.3	GAMESS / VB2000 Version	18
2.3.1	Step by step instructions	18
2.3.2	Run GAMESS / VB2000	19
2.4	GAMESS Windows Version	20
2.5	Gaussian-VB2000 Version	20
2.6	Systems supported	20
2.6.1	Linux	20
2.6.2	Cygwin	21
2.6.3	Silicon Graphics IRIX64	21
2.6.4	Windows platform	21
2.6.5	Mac OS X	21
2.6.6	The future	21
2.7	Running the stand alone tests cases on UNIX-like platforms	21
2.8	Running the GAMESS / VB2000 tests cases	22
3	Capabilities of VB2000	23
3.1	Hartree-Fock molecular orbital theory	23
3.2	Pipec-Mezey localization method	23
3.3	Modern <i>ab initio</i> Valence Bond method	23
3.4	Group Function Theory and its combination with VB method	24
3.5	Energy profile of chemical reactions	24
3.6	Structure weights	25
3.7	Spin density	26
3.8	Visualization	26
3.9	Strict localization and enhanced localization of VB orbitals	27
3.10	Extended functionality with GAMESS / VB2000	27

4 Quick Start Tutorial	28
4.1 Simple VB calculation of CH ₄	28
4.2 How does it work?	29
4.2.1 Hartree-Fock calculation	29
4.2.2 Pipek-Mezey MO localization	29
4.2.3 LMO group assignment	29
4.2.4 Initial VB orbitals	30
4.3 Two-group VB wave function of CH ₃ -CH ₂ OH	30
4.4 GVB calculation of ethane	31
4.5 CASVB(4,4) calculation of H ₂ O	31
4.6 Run VB2000 in GAMESS	32
4.7 Tips and tricks	32
5 Terminology	34
5.1 Electron groups	34
5.2 Group function	34
5.3 System wave function function	34
5.4 Macro iteration	34
5.5 Rigid rotation of a system wave function	34
5.6 VB, VBSCF, SCGVB (SC, SCVB), BOVB, CASVB and CASSCF	35
5.7 Group Function Theory (GFT) vs. ORMAS	36
6 Input Description	37
6.1 General consideration	37
6.2 Basic controls and molecule specification	38
6.2.1 Method	38
HF	38
VB(m)	38
SCGVB(m)	38
SCGVB(m,n)	39
CASVB(m,n)	39
ION(m,n) or SEN(m,n)	39
GPF(n _g)	39
Dot notation	40
6.2.2 Use of libraries from VBOLIB directory	40
6.2.3 Basis set	40
6.2.4 Options	43
PRINTALL	43
RESTART	43
CIONLY	43
NOROT	43
FROZEN	43
UNITS=BOHR	44

SPDEN.....	44
DIIS.....	44
SPHER.....	44
TEST.....	44
6.3 Group Function controls.....	45
6.3.1 \$GENCTL.....	45
6.3.2 \$GRPDIM.....	45
6.3.3 \$VBGA (\$VBGROUPASSIGNMENT).....	45
6.3.4 \$LMOGRPMODIFY.....	47
6.3.5 \$MACROITER.....	47
6.3.6 \$ECONV.....	47
6.3.7 \$ROTATION.....	47
6.3.8 \$NOTROT.....	48
6.3.9 \$FULLHESS.....	48
6.3.10 \$DAMPROT.....	48
6.4 VB orbital optimization controls.....	49
6.4.1 \$VBSCF.....	49
6.4.2 \$##VBSCF.....	49
6.4.3 \$##VBSTR.....	49
6.4.4 \$HESSCONST.....	50
6.4.5 \$BRILMASK.....	51
6.4.6 \$##LENHANCE.....	52
6.4.7 \$DPWEIGHT.....	53
6.4.8 \$CMAXCUT.....	53
6.4.9 \$NOLOCVBO.....	53
6.5 Initial VB orbital generation.....	54
6.5.1 \$##VBORB.....	54
6.5.2 General expression.....	54
6.5.3 \$##PIVBO.....	55
6.5.4 \$LMODISTORTION.....	55
6.5.5 \$WRITEGUESS.....	56
6.5.6 \$READGUESS.....	56
6.5.7 \$RESTARTFILE.....	57
6.5.8 \$RESTARTMAPPING.....	57
6.5.9 \$AOGROUP (original \$PIORB).....	60
6.6 Control for chemical reaction.....	61
6.6.1 \$REACTION.....	61
6.7 Advanced controls.....	61
6.7.1 \$##ROOT.....	61
6.7.2 \$##STRSYMM.....	62
6.7.3 \$MEMORY.....	62
6.7.4 \$VBOLIBGEN.....	63
6.7.5 Controls for canonicalization of LMOs.....	64
6.7.6 \$PRINTHS.....	65
6.7.7 \$CENTROID (for GAMESS version).....	65
6.7.8 \$NOVBPROP (for GAMESS version).....	65

6.7.9 Other directives.....	66
6.8 Visualization of VB orbitals and molecules.....	67
6.8.1 \$MOLPLT.....	67
6.8.2 \$PLTORB.....	67
6.8.3 \$VECONLY, \$VECMOS, \$VECLMOS and \$VECINIT.....	68
6.8.4 \$CUBE.....	69
6.8.5 \$GENGRID or \$GRID.....	70
6.8.6 \$LINE.....	70
6.8.7 \$XYZFILE.....	71
6.8.8 \$MOLEKEL.....	71
6.8.9 \$MOLDEN.....	71
6.8.10 \$DENSECUBE.....	72
6.8.11 Using the visualization directives in TEST runs.....	73
6.8.12 3D pictures of two VB orbitals of a O-H bond	73
6.9 Getting quick help on commands.....	73
7 Construction of Initial Wave Functions	75
7.1 Notations of localized molecular orbitals.....	75
7.1.1 Lone pair.....	75
7.1.2 Two-center bond LMO.....	77
7.1.3 Multi-center bond LMO.....	78
7.2 Notations of VB orbitals.....	79
7.2.1 σ VB orbital.....	79
7.2.2 π VB orbital.....	79
7.2.3 Multiple VBOs.....	80
7.3 Construction of initial VB orbitals.....	81
7.3.1 Initial VBOs by splitting LMOs.....	81
7.3.2 Initial VBOs by VBO libraries.....	81
7.3.3 Linear combination of standard VBOs.....	81
7.3.4 A method for construction of CASSCF space.....	82
8 Test Cases and Output Descriptions	83
8.1 VB(4) calculation of H ₂ O.....	83
8.2 CASVB(4,4) calculation of H ₂ O.....	94
8.3 CASVB(8,6) calculation of H ₂ O.....	106
8.4 VB(6) calculation of C ₆ H ₆	100
8.5 VB(6) calculation of the triple bond of C ₂ H ₂	102
8.6 VB(6) calculation of C ₂ H ₂ with three equivalent banana bonds.....	104
8.7 VB(10) calculation of C ₂ H ₂	105
8.8 VB(12) electron VB calculation of C ₂ H ₄	106
8.9 VB(14) electron VB calculation of C ₂ H ₆	107

8.10 Triplet state of trimethylmethene (TMM) and spin density.....	108
8.11 Singlet state of TMM.....	110
8.12 VB(8)•VB(8) calculation of ethanol.....	112
8.13 All-valence electron VB calculation of C ₆ H ₆	116
8.14 VB(8)•CASVB(6,6)•VB(8) calculation of Si ₄ H ₆	121
8.15 VB calculation of the transition state of [Cl---CH ₃ ---Cl] ⁻	123
8.16 Energy profile of a S _N 2 reaction.....	125
8.17 Visualization of VB orbitals of H ₂ O.....	127
8.18 Calculation with dynamic VB orbitals.....	128
8.19 VB calculation with strictly localized orbitals.....	130
8.20 VB calculation of structure weights with localization enhanced VB orbitals.....	133
8.21 Benzene with ionicity (0,2) / seniority (6,2)	138
8.22 SCGVB(6,5) calculation of the cyclopentadienyl anion	141
8.23 Multiconfigurational SCGVB calculation of the allyl cation	143
8.24 Other included input files	145
9 Acknowledgments	146
10 Appendix I, Limits on parameters	148
10.1 Limitations on molecule size.....	148
10.2 Limitation on number of electrons.....	149
11 References	153

1 Introduction

1.1 What is VB2000?

VB2000 is an *ab initio* electronic structure package for performing modern Valence Bond calculations based on a highly efficient VB algorithm—the so called [Algebrant Algorithm](#)[1-6]. It has a general implementation of the [Group Function \(GF\) Theory](#) [8-10], in which a large molecule is described in terms of its constituent parts of physically identifiable "electron groups". The modern VB method implemented in VB2000 produces CASSCF(N,N) quality wave functions with only a small fraction of CPU cost for a CASSCF calculation for $N > 10$. The combination of efficient VB methods and the general implementation GF theory in VB2000 makes it a very powerful tool with the investigation of the nature of chemical bonds[11-21].

The development of VB2000 has been motivated by two main considerations. First, there is a need to obtain high-precision electronic wave functions, capable of giving quantitative substance to the empirical and intuitive ideas distilled from more than two hundred years of experimental chemistry. There exist within molecules "structural units" such as chemical bonds and functional groups, often with highly individual properties, which are transferable from one environment to another with little change; the behavior of such units conforms, in many cases, to empirical "additivity rules" which have never received a convincing theoretical explanation. Second, although valence bond theory, as developed in the 'thirties and forties', was brilliantly successful in rationalizing such ideas, within the conceptual framework of quantum mechanics, it failed completely to provide rigorous quantitative predictions. The reason was simply that for a system of N electrons the computational cost of *ab initio* VB calculations tended to grow like N factorial.

With use of the Algebrant Algorithm, *ab initio* VB calculations on systems with up to about 14 electrons are now becoming 'routine' — even though $N!$ is then of the order 9×10^{10} . However, the majority of molecules of real chemical interest possess far more than 14 electrons. It therefore becomes mandatory to make some kind of conceptual 'separation' of a larger molecule into smaller and more tractable components. By using the Group Function (GF) approach for this purpose, VB2000 brings molecules with many groups within the scope of rigorous non-empirical investigation. Even though the current release has a limitation to a maximum of 16 electrons for each VB group, the full VB calculation of up to a 16-electron group is technically feasible with even a home PC. With the VB2000 program, innovative VB calculations on relatively large molecules can be performed.

The following special features of VB2000 should be noted:

- The program has been carefully engineered and is highly modular. There is in principle no limit to the number of electron groups that can be recognized; the

larger the molecule, the larger the number of groups to be selected. The actual form of each group function is entirely at the discretion of the user. VB2000 provides a general platform for GF calculations of any kind (e.g. using a VB function for one group and a CASVB (CASSCF equivalent) function for another). Both CASSCF and GVB methods can be considered as special cases of GF Theory.

- The methodology used ensures that calculated energies are true variational upper bounds and that the results obtained are size consistent. The detailed forms of the group functions are *not* prescribed at the beginning of a calculation (as they were in early GF calculations [9], where the global basis was partitioned into subsets, each considered adequate for describing a particular group); they are determined variationally as a result of optimization of the wave function for the entire system, subject only to a strong-orthogonality constraint. This permits the study of chemical reactions, for instance, in which the subsystems representing reactants and products may have forms which change quite dramatically during the reaction process.
- In most cases, even a simple one-structure VB wave function can recover most of the corresponding CASSCF correlation energy with only a small fraction of the CPU cost for the CAS calculation. The use of wave functions of VB type for the groups of greatest chemical interest ensures correct behavior when bond-breaking processes occur within a group, and also ensures that the wave functions describing separate fragments possess a high degree of localization.
- The GF theory approach used in VB2000 leads to wave functions which are highly compact and therefore provide clear links with chemistry and 'chemical intuition'. For example, the electron density for the entire molecule is always the sum of the densities of its constituent parts -- even though the forms of the group densities may change appreciably in some chemical processes. This provides a firm basis for the discussion of the 'additivity rules' which apply to many molecular properties.
- Very often only a small part of a large molecule is responsible for certain chemical and physical properties. The great flexibility of VB2000 allows one to study such a limited region, with a high-precision wave function, using more approximate group functions (such as Hartree-Fock) to calculate the mean field provided by the rest of the molecule.
- One important feature of the VB method is that the optimized orbitals tend to be highly localized. The GF framework combined with localized VB group functions provides the possibility of developing a linear scaling algorithm for truly large systems by harnessing the exponentially increasing computer power.

1.2 What is new in version 3.0?

The VB2000 3.0 release was initially intended to be the 2.8 version, but due to significant postponements in the release and the inclusion of two new developers into the VB2000 team, we have renamed it as the 3.0 version for the public release in 2021.

This version is primarily due to developments that removes restrictions on the kind of GAMESS calculation that is used to construct initial guess orbitals in GAMESS/VB2000. Now all values of SCFTYP can be used, allowing UHF, GVB and MCSCF orbitals, in addition to RHF and ROHF. All kinds of DFT orbitals can be used. MP2, coupled-cluster and configuration interaction can be calculated but the HF orbitals are used. GVB is restricted to closed shell and uses the actual “bond” GVB orbitals after using localized orbitals to calculate the GVB CI pairs. The GVB orbitals can be a particularly good initial guess to use VB2000 for more advanced VB methods than GVB-PP. RUNTYP = OPTIMIZE and RUNTYP = HESSIAN can now be used to optimize the geometry or calculate frequencies with the GAMESS method before doing a VB2000 calculation, as well as optimizing the geometry or calculate frequencies with the VB method. RUNTYP = IRC and RUNTYP = SURFACE can be used with a range of GAMESS methods, calculating VB function at each point.

\$GMSPATH/vb2000/DOC/GMS_VB_test.README explains these methods in more detail and describes a set of test files for them.

Other new features are:

- A *.vec file can be created containing a GAMESS like \$VEC file for initial orbitals. MOs, LMOs and final VB orbitals. This is now supported by MacMolPlt which labels the orbitals correctly
- Molden files can now be read directly by the stand-alone version, to use the basis set, geometry and MOs directly from another program.
- VB calculations can be done after a optimisation or Hessian with another method or along an IRC or set of saddle points calculate with another method.
- VB2000 can now use the structures for a set of ionicities (number of double occupied orbitals) or, equivalently, seniority numbers (number of singly occupied orbitals).
- The Generalized Product Function Energy Partitioning code from the Nascimanto Group in Brazil was added to the GAMESS/VB2000 version.
- See the CHANGELOG and Change_VB2.7-3.0.pdf in the DOC directory for more details and for other developments.

1.3 What was new in version 2.7?

The VB2000 2.7 release is primarily due to the development of a new way of using Gaussian with VB2000, but there have been many other improvements, which were applied into the GAMESS 2014 release. The new Gaussian/VB2000 version is in the directory “g09-vb2000” below the VB2000 directory. The Gaussian/VB2000 version can calculate centroids of orbitals, but not extents.

Other new features are:

- Changes to the VBOLIB system for the GAMESS version. The number of VBOLIBs has been reduced in this release, but additional VBOLIBs are now created on the fly for the addition of polarization functions and diffuse functions. For example there is now only one VBOLIB for the 6-31G basis sets with VBOLIBs for 6-31G*, 6-31G**, 6-31G++G**, 6-31G(2df,p) and many more created on the fly. The new Gaussian/VB2000 version uses a similar approach but also creates VBOLIBs for spherical harmonics from Cartesian VBOLIBs on the fly. A new VBOLIB for 6-311G has been created but for H and 1st row elements only.
- BRILLMASK has been extended to automatically create the mask conditions for the case of localization of VB orbitals to just atoms.
- A new visualization directive \$DENS_CUBE creates CUBE file for the total density, the density of specific groups and, for open shell systems, the spin density.
- Since both GAMESS and Gaussian with conventional integral storage have both been extended to use h and i basis functions, VB2000 has been extended in this way. However SPHER can not be used with such basis sets and a bug in Gaussian (fixed for the next release) prevents the use g, h and i with Cartesian basis sets.
- The vb2blas.src file has been renamed to vb2000-so.src and it now contains routines, previously in vb2000.src, that are only used in the stand-alone version.
- Greater control over maximum values for many parameters has been introduced and is explained in Appendix I.

1.4 What was new in version 2.6?

The VB2000 2.6 release is primarily due to the fact that from this point VB2000 will be released as part of the official GAMESS(US) release. Everything needed to get the GAMESS(US)/VB2000 program working will be found in the directory “vb2000” below

the “gamess” directory.

There are however some new features:

- \$MOLDEN added to the visualization directives to produce a Molden format file for use by the Molden program.
- Extension of the \$PLTORB visualization directive to read data so that a “ready to go” input file is prepared for the pltorb program. Previous the file produced had to be manually edited.
- Make the visualization routines prepare files for the LMOs, rather than the VB orbitals, if the TEST directive is on the command line.
- Improved memory allocation in GAMESS(US)/VB2000. It now uses the maximum of the remaining memory from the GAMESS code or that specified by \$MEMORY

1.5 What was new in version 2.5?

The new functionality and enhancements in VB2000 2.5 release are listed as follows:

- 64 bit Linux system support. This allows access to much large memory.
- Extension for Spin-Coupled method. A new method SC(m,n) is added, where m is the number of electrons, and n is the number of orbitals. In the new definition, SC(m,n) specifies all spin-coupled states of all electron configurations of m electrons in n orbitals. The configurations are defined as those that have the maximum number of occupied orbitals. When m=n, SC(m,n) becomes the original SC method (7)
- Localized non-orthogonal Hartree-Fock calculations. Better localized orbitals with better transferability can be obtained. Overlap penalty is used for the optimization of non-orthogonal Hartree-Fock molecular orbitals. This is necessary to avoid the collapsing of non-orthogonal MOs.
- Projection to use spherical harmonic basis functions. GAMESS and VB2000 stand-alone use Cartesian basis functions internally. Some basis sets are defined in spherical harmonics, such as cc-pVnZ basis sets.
- Computation of VB orbital centroids and extents. This function is only available in GAMESS / VB2000, and latest stand-alone Gaussian / VB2000.
- Extension for the maximum number of elections in each VB group. Up to 16 electrons per VB group can be handled.

1.6 What was new in version 2.1?

The new functionality and enhancements in VB2000 2.1 release are listed as follows:

- New expression of initial VB orbitals using atomic orbitals, LMOs and their linear combinations.
- New VBO libraries for more basis sets.
- Grid format of orbital visualization files which can be displayed with Accelry's Discovery Studio Version 2.5
- Many enhancements, including better memory management for large number of basis functions, increasing the maximum number of basis functions in the whole molecule as well as in an individual VB group and several bug fixes.
- Improvement of convergence for VBSCF, especially for enhanced localization of VB orbitals and BOVB calculations.
- VB2000 2.1 is updated for the comparability with the current version of GAMESS (2009).

1.7 What was new in version 2.0?

The new functionality and enhancements in VB2000 2.0 release are listed as follows:

- Adjustable control for the localization enhancement of VB orbital optimization.
- Strict localization of VB orbital optimization.
- Interface to GAMESSPLUS for VB-solvation calculation in GAMESS (not recently tested).
- New user interface (UI) of VB2000 on Windows platform can now handle both VB2000 and GAMESS inputs. For GAMESS calculation, a user should have WinGAMESS installed. Since WinGAMESS has VB2000 module compiled in, a user can also run VB calculations with WinGAMESS once a VB2000 license has been obtained. N.B. As of 2018, WinGAMESS does not include VB2000.

1.8 What was new in version 1.8(R2)?

The following functionality and enhancements have been added in version 1.8(R2)

- Spin-density of VB wavefunction. A direct summation method for spin-density calculation of VB wavefunctions has been implemented.
- DIIS convergence control. The Direct Inversion of Iterative Subspace method has been implemented for the convergence control of VB orbital optimization.
- Enhancement for multi-structure VB calculations with dynamic VB orbitals (BOVB type calculations). The VB orbital optimizer allow user to use a set of VB orbitals which can be linearly dependent.
- Extended map files. The map files for N=1 and 13 are added to the MAP directory. (N is the number of electrons in a VB group).
- Many bug fixing. The new version allows VB(1) calculation. Two crashing bugs have been fixed: the VB calculation on a single atom and triplet state of two electron VB.

1.9 What was new in version 1.8?

Since the first formal release of VB2000 (Version 1.4) in 2000, we have made numerous improvements and added new functionalities. The most significant developments that have been made since version 1.7 are:

- VB2000 has been fully integrated into two major quantum chemistry programs, GAMESS(US) and Gaussian so that the some of the functionalities of GAMESS(US) and Gaussian can be used for VB wave function. See the **Installation Guide**
- Valence Bond Orbital (VBO) Library Files have been introduced for construction of initial VB orbitals. The simple intuitive expression of VBOs provides extreme flexibility for VBO construction.
- The limitation for the number of basis functions for the integral module has been removed so that larger molecules can be treated in the stand-alone version.
- Graphic utilities have been added for VB orbital visualization.
- There have been other technical enhancements:
- Introduced a 'dot' notation for group function calculations.
- Introduced dynamic memory allocation.

- Added many more built-in basis sets.
- General efficiency has been Improved.
- Added many new controls.
- Enhanced the portability on many more platforms/compilers.
- Enhanced Pipek-Mezey localization routine.

The Pipek-Mezey MO localization algorithm becomes ill conditioned when orbitals are nearly localized on the same atom; this is because the localized molecular orbital (LMO) function value changes very little for the mixing of two orbitals nearly localized on the same atom. For this reason, the final LMO orbital coefficients and the LMO orbital energies becomes very sensitive to the convergence criteria and platforms. Even though in most VB calculations, the small differences of LMO do not affect the final results, it can cause confusion. To reduce the numerical instability of the PM localization, we added some numerical controls; if the change in value of the objective function of an LMO is smaller than a certain threshold, then do not perform orbital rotation. Another condition to improve the stability is to first prevent the orbital mixing of energetically close MOs. Thus the inner atom core orbitals of different energies on the same atoms are not allowed to be mixed at the beginning. In this way, the mixing of orbitals localized on the same atom is reduced significantly.

- We have introduced maximum localization of the VBOs in CASVB calculations.

Since the CASVB is a full CI wave function in a subspace, the orbitals that span the orbital space could be totally arbitrary. One method to reduce the arbitrariness is to use a localization scheme. In CASVB calculations, the orbitals are not necessary orthogonal to each other, therefore this gives some extra flexibility for the orbital localization. To take advantage of this, a method to achieve the maximum localization of an VB orbitals has been introduced and implemented in VB2000. In this method, the CASVB calculation starts from a set of good initial VBOs, and the localization is performed at the last macro iteration to maximize the localization object function for each VBO within the CAS orbital subspace. This procedure leads to a set of highly localized AO-like orbitals, and the CASVB wave functions are usually very compact (i.e. only a few VB structures give significant contributions).

- We have added more Map files.

Table 1: Map files with the following combinations of number of electrons (N) in the VB groups and the number of unpaired spins (S).

<i>S/N</i>	<i>1</i>	<i>2</i>	<i>3</i>	<i>4</i>	<i>5</i>	<i>6</i>	<i>7</i>	<i>8</i>	<i>9</i>	<i>10</i>	<i>11</i>	<i>12</i>	<i>13</i>	<i>14</i>
0		x		x		x		x		x		x		x
1	x		x		x		x		x		x		x	
2		x		x		x		x		x				
3			x		x		x		x					
4				x		x		x						
5					x		x							
6						x								

The first row is the numbers of the electrons (N) in VB groups, and the first column is the numbers of unpaired spins (S). The mark "X" indicates that the map file of the corresponding combination of N and S is included in the package. Map files for larger systems are available on personal request. If you need a map file for a combination of N and S not in the above table, consult the authors and they will be sent to you.

2 Installation Guide

2.1 Downloading VB2000 packages

As of version 3.0 VB2000 has two flavors, a stand-alone program, and a plugin version embedded into the GAMESS(US) release. The former VB2000 / Gaussian variation is not supported anymore.

VB2000 stand-alone version is for all UNIX-like systems. It can be used on Windows, either using Cygwin or using the Windows Subsystem for Linux (WSL) available in Windows 10. The former WinVB2000 variation is not supported anymore.

This section describes the installation on UNIX-like platforms. The following instructions assume that a user has received the compressed file of VB2000 package.

It will be helpful if you let us know that you are using VB2000 by emailing us at jiaboli@yahoo.com, david.sousarj@gmail.com or rodrigobitzer@gmail.com.

Use the following commands to unpack the compressed tar file on UNIX-like platforms (Unix, Linux and Cygwin):

```
tar -zxvf vb2000.tar.gz
```

If the -z flag is not supported on your particular version of UNIX, type:

```
gunzip vb2000.tar.gz  
tar -xvf vb2000.tar
```

Once the tar file is unpacked successfully, one should see a new directory VB2000 created in the current directory. This is the main directory of VB2000. Go to this directory and you should see the following sub-directories:

BASET	Basis set files.
DOC	Contains this manual and other relevant documentation files.
gamess	Files for the Gamess version.
MAP	Auxiliary files for the Algebrant algorithm.
molten	Molten test input files.
SRC	Source code.
TESTINP	Input files of test cases.
TESTOUT	Output files.
TOOLS	Utilities.
VBOLIB	VB orbital library files.

2.2 Stand alone VB2000 on a Unix-like platform

To install VB2000 as a stand alone program, one can use the installation script *install* in the VB2000 root directory and follow the instructions for various platforms and compilers. For GAMESS / VB2000 installation, please read the corresponding specific instructions.

To compile the source code, one can run the installation by issuing the following command:

```
./install
```

When the compilation is completed, an executable file *vb2000.exe* will be found in the current directory, i.e. the main directory *VB2000*. Check the log file for possible error messages. If your system is not one of the ones included in the install script, you can probably amend one of the choices to fit your system. Please pass on any successful examples to us.

2.3 GAMESS / VB2000 Version

From version 2.6, VB2000 is part of the standard GAMESS release. The *vb2000* directory, below the *gamess* directory defined by *\$GMSPATH*, contains the VB2000 files. Note that this is GAMESS(US), not GAMESS(UK). The following instructions assume that you have installed GAMESS released in 2012 or later. As of VB2000 3.0, the VB2000 code is compiled by default when you compile GAMESS in the normal way. If it is not the case, follow the instructions below as you would after making your own changes to VB2000.

2.3.1 Step by step instructions

After running *config*, carry out the following steps:

Step 1. Typing:

```
./compall
```

will compile all GAMESS without VB2000 modules. The four VB2000 modules can be compiled by editing *compvb* in the *vb2000/gamess* directory, and then copying it one up to the *gamess* directory.

You must also define the global variable *\$VB2000PATH* to point to the directory where the stand-alone VB2000 is. See the file *gamess.README* for more instructions.

Step 2. Run:

```
./compvb
```

This will compile and link GAMESS with VB2000. You probably need to modify the script `lkd`. You may want to alter the `VERNO` in `compvb` to give a different executable from one linked earlier.

Step 3. Test GAMESS with VB2000

Move to the `vb2000` directory and type

```
./runallvb
```

This will run all the VB2000 tests from the tests directory. You will need to read `gamess.README` and `GMS_VB_test.README`. The `runallvb` script uses the `rungms` script in the main GAMESS directory. You need to check the `runallvb` script to ensure it is using the correct version and scratch file. When the jobs have finished, a quick check is to type:

```
grep TERMINATED exam-vb*.log
```

This should show that all jobs have terminated normally. Then type:

```
./checkvbtst
```

which will carry out a more extensive check of the outputs. This script works in a similar way to `checktst` for the normal GAMESS tests and exactly like `checktst` used to work.

2.3.2 Run GAMESS / VB2000

The GAMESS / VB2000 executable can be used for both normal GAMESS jobs and VB calculations as well. `gamess.README` gives a more detailed account. To run VB calculations, you need to ensure that two variables are set in your environment. The `rungms` script should already include the line:

```
setenv GMSJOBNAME $JOB
```

The `$VB2000PATH` environment variable is already set in `rungms` as:

```
setenv VB2000PATH $GMSPATH/vb2000
```

below where `$GMSJOBNAME` is defined. Alternatively you can set `VB2000PATH` in your `.cshrc` or `.bashrc` file, depending on which shell you are using. It must point to `$GMSPATH/vb2000`, but with `$GMSPATH` expanded of course to the actual path. You must then delete the line defining it in `rungms` as that will otherwise take precedence.

2.4 GAMESS Windows Version

The Windows GAMESS executables from the GAMESS site did not include VB2000 due to problems with the compiler. We hope to restore this soon.

2.5 Gaussian-VB2000 Version

From version 3.0, the Gaussian integrations with VB2000 are not supported anymore. Please check the VB2000 manual of older versions if you want to use it.

2.6 Systems supported

Considerable efforts have been put into the improvement of the portability of the VB2000 code. VB2000 has been tested in the past on several platforms with different compilers. It is expected that the code can be ported on to many other platforms without major changes. The details of machines, operating systems and compilers that VB2000 has been tested on are as follows:

2.6.1 Linux

Linux is pretty dominant these days for all codes and most of the development of VB2000 has been under Linux or the related Cygwin. We have used several versions of Linux in the past, including Redhat, Fedora Core, Fedora Enterprise, SUSE, Mint and Ubuntu, but all recent work has been in Ubuntu or Mint. Several compilers have been used:

- a) **gfortran**. Many versions of this compiler have been used with success.
- b) **g77**. In the past, we used most versions of this compiler and had no problems. From version 3.0, it will be not supported anymore, due to inclusion of some Fortran 90 elements.
- c) **Intel** compiler. We have used several versions of this compiler, going back to version 6.0. Both the intel C compiler, icc, and gcc can be used for the C routines. This appears to work fine for the stand-alone version. For GAMESS, some versions used to cause diagonalization problems in the initial guess for SCF.
- d) **Portland** compiler. We have used version 3.2-4, and more recently 10.6 on 64 bit Linux. Both the Portland C compiler and gcc can be used for the C routines. Like ifort, with GAMESS, there are some diagonalization problems in the initial guess for SCF.
- e) **g95**. We have some experience of this compiler using similar flags to g77.

With the stand-alone version, the appropriate flags are given in the install script. It is

difficult to select the best choice of flags for a particular piece of code. It is not clear that our choice is the optimal. Our choice is often based on the choice in the GAMESS(US) comp script.

With GAMESS/VB2000, we made no major changes to the standard flags for all the above compilers.

2.6.2 Cygwin

Cygwin is pretty much like Linux. It provides great convenience to work in both Windows and a Unix-like environment. Both g77 and gfortran compilers are used for our developments. Our very recent tests show that the gfortran compiler has excellent performance, especially for GAMESS.

2.6.3 Silicon Graphics IRIX64

The experience with this system is now quite old and we no longer have access, so no further information is worth giving here

2.6.4 Windows platform

The version formerly distributed as WinVB2000 was compiled under cygwin with the “-no-cygwin” flag so it used the MinGW include files and libraries, which gives a stand-alone executable.

2.6.5 Mac OS X

VB2000 can be successfully installed on the Mac OS with the gfortran compiler, but we have limited experience.

2.6.6 The future

We look to users to provide feedback on the use of VB2000 on other machines and under other environments. Information on other machines and compilers will be available on the VB2000 web site as it becomes available from us or others.

2.7 Running the stand alone test cases on UNIX-like platforms

The script `runall` can be used to run the test jobs. To do this, first go to the VB2000 main directory, and type the command:

```
./runall
```

The test jobs run sequentially, and the output files will be put in a temporary directory `TEMPOUT`. The test run output files are also included in the package, in the directory `TESTOUT`. One can compare the output files in `TESTOUT` and `TEMPOUT` to verify the

correctness of the installation. Use the script `vbqa.pl` to do a simple comparison of the results. Just type `./TOOLS/vbqa.pl`. You must have Perl. You may have to check the first line of the script to make sure it points to the correct location of Perl. You may have to make the file executable.

2.8 Running the GAMESS / VB2000 test cases

The script `runallvb` in the `vb2000` directory in the GAMESS directory will run a set of tests. By running `./checkvbtst`, it will check these in a similar, but not quite identical fashion as the GAMESS `checktst` utility works. Each input file has notes on the test, as does `GMS_VB_test.README`. You should also read `gamess.README`.

3 Capabilities of VB2000

3.1 Hartree-Fock molecular orbital theory

A Hartree-Fock molecular orbital calculation is the starting point for any other type of calculation. Although VB2000 is not designed for pure Hartree-Fock (HF) method, we have made a number of improvements over the last few years. One major improvement is that the maximum number of basis functions that can be handled by the program has been significantly increased. The HF orbitals are used for electron group assignment, initial guess of VB orbitals and initial one-electron densities of all groups based on Pipek-Mezey localization as described in the next section.

3.2 Pipek-Mezey localization

The Pipek-Mezey (PM) method for localization of molecular orbitals (MOs) has been implemented. This utility plays a critical role in VB2000. First, the LMOs provide a solid base for dividing electrons into spatially separable groups. The LMOs of each electron groups are the starting points for generating the initial group functions and the corresponding one-electron densities. One of the initial guess methods for VB orbitals is to split each bonding LMO into two overlapping AO-like orbitals. See more details in Chapter 7. The PM method is based on a simple criterion: the localized MOs should maximize the sum of the squares of Mulliken charges of all atoms from each individual MO. The PM algorithm is also very simple, being based on successive rotations of orbital pairs. This method usually leads to results which are remarkably similar to those obtained from more sophisticated methods, and the cost in CPU time is quite negligible.

3.3 Modern *ab initio* VB method

A major feature of VB2000 is the implementation of modern valence bond theory at *ab initio* level using the algebrant algorithm. The VB engine in VB2000 has been implemented in the most general form. It can be used to perform very general VB calculations[9-12] such as:

- Multiple-Structure VB (VB)
- Spin-Coupled Theory, recently renamed Spin-Coupled Generalized Valence Bond (SCGVB)
- Complete Active Space VB (CASVB)
- VB with Localized Electron Pairs (GVB)
- VB Configuration Interaction (VBCI)
- Breathing Orbital VB (BOVB)

VBCI implements a multi-structure VB calculation without orbital optimization. Usually one starts with the optimized VB orbitals obtained from a single structure VB calculation.

This option can be useful when the further optimization of VB orbitals with multi-structures becomes expensive. Our experiences show that further optimization with multi-structures has little gain as compared with the corresponding VBCI calculation with optimized orbitals of a single structure calculation. The output from VB, SCGVB and VBCI calculations normally includes the weights of all resonance structures.

3.4 Group Function Theory and its combination with VB method

As indicated in the Introduction, an essential feature of VB2000 is the implementation of the Group Function Theory, in which a molecular wave function is expressed as an antisymmetrized product of individual group functions [6-8]. The spin-orbitals (one electron 'groups') of a Slater determinant are thus replaced by many-electron group functions - each of which may allow a considerable degree of intra group electron correlation. In VB2000, the electron-correlated group functions are obtained by the VB method. Even though the general mathematical formulation of this approach was completed more than four decades ago and it provides a unifying frame work for many varieties of electron correlation calculations, the general implementation of GF Theory, especially its combination with VB method, was introduced for the first time in VB2000. The general implementation of GF Theory in VB2000 provides tremendous flexibility for this type of computation: a molecular system can be divided into any number of electron groups and each group can be treated at any level of electron correlation of the user's choice. A common scenario is that a higher level of theory (such as CASVB) is used for the most critical regions in a molecule, for instance the bond breaking and forming region, while for the remaining parts some less accurate methods (such as simple VB, even Hartree-Fock) are used. Even though the general idea of using different levels of theory for different part of a molecule is somewhat similar to QM/MM or ONIOM, the approach in VB2000 is fundamentally different from QM/MM and ONIOM. In QM/MM, the system is not described in full quantum mechanics, and the QM/MM boundary is usually a source of trouble due to the lack of consistency for treating a molecule system. ONIOM is basically a highly sophisticated interpolation method based on the computation of different sizes of model systems and different levels of theory. In contrast, the GFT method is a fully quantum mechanical method with a consistent Hamiltonian. Moreover, the approach as a whole is *variational*.

3.5 Energy profile of chemical reactions

VB2000 provides a functionality for the automatic calculation of the energy profile of a chemical reaction along the reaction path guided by a few intermediate structures (such as reactant, transition state, and product etc.). Only for the simplest cases, the Linear Synchronous Transition (LST) method is a good approximation for the reaction path between reactant and product geometries. However, if a few good representative intermediate structures along the reaction path are available, one can use the linear

interpolation between each two consecutive structures for constructing a reasonably good path for the whole reaction. Assume G_i and G_{i+1} are two consecutive structures, and G_x is the linear interpolation point between them, then the Cartesian coordinates of G_x can be obtained from the following equation:

$$G_x = xG_i + (1-x)G_{i+1}$$

To make the geometry change as smooth and possible, the structures $G_i (i = 1, \dots, N_{\text{Geoms}})$ should be aligned for the best overlap between each two consecutive structures. This is done automatically by the program using the [Kabsch](#) algorithm. The geometries of the representative structures along the reaction path can be obtained from [an independent](#) source (for instance, [an](#) IRC calculation at a lower level of theory, or even created manually according to some basic knowledge of a chemical reaction). VB2000 also provides controls for how fine the resolution is for the geometry changes [along](#) the reaction path. Details of the controls in the input are given in the INPUT section of this manual.

3.6 Structure weights

Important information from a multi-structure VB calculation is provided by the weights of the various VB structures. In VB2000, we have implemented four types of structure weight analysis, i.e. The Mulliken Type (also called Chirgwin-Coulson type[22]), Löwdin type[23], Gallup and Norbeck[24] and Hiberty type[25]. The structure weights of the types are calculated according to the following formulas. Let's assume that the multi-structure VB wave function is expressed as follows:

$$\Psi = \sum_i^M C_i \Phi_i$$

where Φ_i is a normalized VB wave function of structure i . The Chirgwin-Coulson structure weights can be calculated as following:

$$W_i = \sum_j^M C_i C_j S_{ij}$$

where S_{ij} is the overlap of structure i and j .

A drawback of above formalism is that negative values of weights can be obtained, which is not physical. A remedy for this the negative value problem is to use [a](#) Löwdin formalism. This involves the Löwdin orthogonalization of the structure wave functions, i.e.

$$\Psi = \sum_i^M C'_i \Phi'_i$$

where Φ'_i ($i=1, \dots, M$) are the orthogonalized structure wave functions obtained by using the Löwdin procedure), and C'_i is the new coefficient of the corresponding structure i . The Löwdin weight can be calculated as the square of the corresponding coefficient:

$$W_i = C_i'^2$$

Another remedy for the negative value problem was first used by Hiberty, in which the squares of the original coefficients are used as the structure weights, i.e.

$$W_i = \frac{C_i^2}{\sum_j C_j^2}$$

In ideal cases where all the structure wave functions are orthogonal, all the above three formulas give the same results. All problems comes from the case when structure wave functions are not orthogonal, and in this case, none of them are perfect. When the structure functions are not orthogonal discrepancies among the results are sometimes found; but these are usually small. If large discrepancies occur, the numbers obtained should be used with caution.

3.7 Spin density

The spin density for open shell VB wave functions can be calculated. Please note that by default, the open-shell VB group must be the last one.

3.8 Visualization

In both the stand-alone and GAMESS / VB2000 versions, functionality has been added for plotting 2D VB orbitals, displaying 3D orbitals and molecular geometries. The 3D orbital files in Gaussian Cube format can be generated. The files can be visualized by a variety of visualization programs, such as VMD, MacMolPlt, Molekel, Ghemical, Pltorb, Molplt, gOpenMol, Pymol etc. To extend the graphic visualization with the industrial leading tool for molecular modeling and simulation, Accelry's Discovery Studio (DS) and its free version DS Visualizer, the 3D orbitals in grid format can be generated and displayed in DS Visualizer.

These are described in more detail in section 6.8

3.9 Strict localization and enhanced localization of VB orbitals

In most cases, VB orbitals are highly localized. The localized AO-like VB orbitals provide a clear picture that each VB structure corresponds to a Lewis structure unambiguously. However, it is not uncommon that the VB orbitals have considerable delocalization over to the neighbor atoms. Such a situation happens more often in special bonding patterns. Some times, it is not clear whether the delocalization has a strong physical driving force or just a outcome of variational optimization. To study this, one can force the VB orbitals to be localized and to see whether this has significant impact on the energy of VB wavefunction or not. If the energy increases significantly due to the localization constraints, then one may add more ionic VB structures in the wave function, and thus we know the system has considerable contribution of ionic structures.

There are different ways to force VB orbital localization. One straight approach is to divide basis functions into groups, and only allow each VB orbital to be optimized within a particular predefined group of basis functions. This is the strict localization constraints as used in BOVB approach. A control for strict localization (\$BRILLMASK) has been implemented in VB2000 version 2.0. Check out sections 6.4.5 and 8.19 for usage.

The strict localization of VB orbitals is a too harsh constraints and may lead to too high energy. It is not only cumbersome to use but also is limited to some very simple systems. Therefore, we have introduced a more flexible control for localization enhancement of VB orbitals. In this control, an adjustable parameter, which can be considered as a parameter for the strength of localization, is used. If this parameter is set to zero, then the VB orbitals are optimized under no constraints. If this parameter is set to infinity, then the VB orbitals are strictly localized. A particular useful application of such a control is for the cases that the structure weight analysis is needed while the VB orbitals are significantly delocalized so that the structure weights becomes somewhat ambiguous. Another application is for the case that the localization patterns of the final optimized VB orbitals are different from the initial guess.

3.10 Extended functionalities in GAMESS / VB2000

A very significant technical improvement in VB2000 is its ability to be fully integrated with third-party programs: GAMESS and, formerly, Gaussian. In addition to all functionalities of the stand-alone version, some additional features have been added for VB calculations.

- ◆ Dipole moments and higher moments calculations, and other properties
- ◆ Geometry optimization
- ◆ Vibrational frequency calculations
- ◆ Effective core potential (pseudo-potential calculation)
- ◆ More built-in basis sets

4 Quick Start Tutorial

The simplest way to start a calculation on a UNIX-like platform is to submit a job in the installation directory. You can also run a job in a different directory by setting environment variable:

```
setenv VB2000PATH <Your VB2000 Directory>
```

or

```
export VB2000PATH=<Your VB2000 Directory>
```

where **<Your VB2000 Directory>** is the full path of your VB2000 installation directory. You can add VB2000PATH setting in your .cshrc or .bashrc file, depending on which shell you are using.

Before doing a VB calculation, the very first decision that a user should make is how many and which electrons (or which chemical bonds and lone pairs) should be treated by VB method. In this chapter, a few examples are provided.

4.1 Simple VB calculation of CH₄

The first example is a VB calculation of 8 electrons on the 4 C-H bonds of methane. To do the calculation, just copy or write the following lines into a new input file (it is already available at TESTINP/extra/extra01_ch4vb8.inp).

```
#! VB(8)/D95

TEST CH4 5 GROUPS (1 CORE + 4 BONDS)

0 1
6 .000000 .000000 .000000
1 -.880321 .417906 -.470561
1 .259946 .591217 .868290
1 -.202850 -1.018388 .304590
1 .823225 .009265 -.702319
```

Please note that you should always keep at least one blank line at the end of the input file. Save the file as example01.inp (for example) and run the job:

```
./vb2000.exe example01 >example01.out
```

This calculation produces one core orbital corresponding to the 1s core of carbon atom and 8 VB orbitals corresponding to the 4 C-H bonds.

4.2 How does it work?

Here are the basic steps for a typical VB calculation in VB2000.

4.2.1 Hartree-Fock calculation

The first step of a VB calculation is usually a Hartree-Fock molecular orbital calculation. The RHF method is used for a closed shell system and a quasi-ROHF method is used for an open shell system. In the quasi-ROHF method, the Fock matrix is constructed from the total density in the same as in the RHF method.

4.2.2 Pipek-Mezey MO localization

The canonical molecular orbitals from a Hartree-Fock calculation are transformed into a set of localized molecular orbitals (LMOs) by Pipek-Mezey method. The generated LMOs are used for the following two purposes:

1. Group assignment for electrons on LMOs
2. Initial guess of VB orbitals by splitting bonding LMOs

The LMOs are grouped in the following types and order: inner core orbitals, lone pairs and bonding LMOs. For each type of LMOs, the orbitals are ordered in an ascendant order of their orbital energies. Each LMO has a label to indicate its type. The Mulliken population of each LMO is calculated and used to determine its LMO type. If a LMO has approximately equal electron distribution on two directly connected atoms, then this LMO is a bonding LMO between the two. If the population is localized on one atom, then it is either a inner core or a lone pair. If a LMO is localized on more than two atoms, and it is marked as a multi-center (MC) bonding orbital..

4.2.3 LMO group assignment

The program does the LMO group assignment automatically according to the input. For simple VB calculations the default assignment is usually what a user needs. For the above example, the input specifies VB(8), an eight-electron VB calculation. Since the molecule has 10 electrons in total, thus there is a Hartree-Fock core group with two electrons and a VB group with eight electrons. By default, VB2000 starts with the Hartree-Fock group and then the VB group(s) specified in the input. In the CH₄ case, the first LMO is assigned to Hartree-Fock group, and the remaining 4 LMOs, which are bonding orbitals corresponding to the 4 C-H bonds, are assigned to the VB group.

In some cases the default LMO assignment may not be what you need. One should explicitly assign LMOs to the appropriate groups by using \$LMOGRPMODIFY input. A more advanced control for the same purpose is \$VBGA. See Chapter 6 for more details. One can always do a TEST run (keyword: TEST) run and check the order of LMOs.

4.2.4 Initial VB orbitals

By default, the initial VB orbitals for a VB group are generated by splitting the bonding LMOs assigned to this group, and the lone pair orbitals of the VB group are used directly as part of the VB orbitals. Each LMO is split into two VB orbitals. For example, if a VB group has L lone pairs and M bonding LMOs, then $L + 2M$ VB orbitals will be generated for the $2(L+M)$ electrons. Of course, one can always change the number of VB orbitals and the way for generating the initial VB orbitals by using addition controls in the input. See Chapter 6 for **##VBORB** and **\$AOGROUP** controls.

If all above methods fail, there is a last resort: **\$READGUESS**. This option read the initial guess from a text file. The initial guess can be generated from any method. In some cases, you only need to modify a few VB orbitals. In the case, you can first generate such a file by using **\$WRITEGUESS**, and then modify it using any text file editor.

4.3 Two-group VB wave function of CH₃-CH₂OH

Input file (TESTINP/example12_etOH.inp):

```
#! VB(8).VB(8)/D95 PRINTALL
TEST C2H5OH (TWO VB GROUPS)

0 1
6 -0.4413 1.9095 -0.1486
1 0.1529 2.7834 -0.4194
1 -1.2624 1.8366 -0.8622
1 -0.8754 2.0884 0.8352
1 -0.2076 -0.2379 0.0887
6 0.4029 0.6280 -0.1666
1 0.8338 0.4543 -1.1531
1 2.0387 1.4420 0.5056
8 1.4477 0.7230 0.7814

$VBGA
1-2 => 2
1-3 => 2
1-4 => 2
1-6 => 2
6-7 => 3
5-6 => 3
6-9 => 3
8-9 => 3
```

In this case, the molecule is divided into two groups: CH₃- and -CH₂OH, each contains 4 covalent bonds and is described by a VB(8) group function. The system wave function is thus the generalized product of two VB(8) functions and the Hartree-Fock function of the remaining electrons. The \$VBGA flag specifies the VB group assignments. The first group is the Hartree-Fock, the second group is the first VB group, and the third the second VB group. The lines following \$VBGA flag can be read as: the covalent bond between atom 1 and atom 2 is assigned to group 2, etc. See Chapter 7 for more details.

4.4 GVB calculation of ethane

Input file (TESTINP/extra/extra02_c2h6gvb.inp):

```
#! VB(2).VB(2).VB(2).VB(2).VB(2).VB(2).VB(2)/D95
TEST C2H6 (7 SIGMA BONDS)

0 1
6 .000000 .000000 .000000
6 .000000 .000000 1.540011
1 1.027669 .000000 -.363311
1 -.513834 -.889987 -.363311
1 -.513834 .889987 -.363311
1 -1.027669 .000000 1.903322
1 .513834 .889987 1.903322
1 .513834 -.889987 1.903322
```

As a special case of GF theory approach, the 7 GVB pairs can be described by 7 VB group functions each with two electrons in two orbitals. There is a shortcut for GVB specification in the keyword. By getting familiar with these terms, it will be easier to understand the inputs and outputs of VB2000. For the above case, the key word can be simplified as

```
#! GPF(8)/D95
```

By default, if there are no additional flags, the *GPF(n)* key word will be interpreted as n-1 GVB pairs.

4.5 CASVB(4,4) calculation of H₂O

Input file (TESTINP/example02_h2ocas.inp):

```
#! CASVB(4,4)/D95 PRINTALL
TEST H2O

0 1
8 .000000 .000000 .000000
1 .801842 .000000 .555582
1 -.801842 .000000 .555582
```

In this case, the four electrons of the O-H bonds are included in the CASVB calculation.

4.6 Run VB2000 in GAMESS

To run a VB calculation in GAMESS, you need

1. Add VBTYP=VB2000 in \$CONTRL block.
2. Include a normal VB2000 input in a \$VB2000 block.

Here is an example of VB run in GAMESS:

```
$CONTRL SCFTYP=RHF COORD=UNIQUE VBTYP=VB2000 $END
$SYSTEM TIMLIM=20 MEMORY=2000000 $END
$BASIS GBASIS=STO NGAUSS=3 $END
$GUESS GUESS=HUCKEL $END
$DATA
Water STO-3G
C1
OXYGEN      8.0      0.0000000000      0.0000000000      0.0000000000
HYDROGEN    1.0      0.0000000000     -0.7572153434      0.5865355237
HYDROGEN    1.0      0.0000000000      0.7572153434      0.5865355237
$END
$VB2000
#! VB(4)/STO-3G PRINTALL

Water

0 1
8  0.0000000000      0.0000000000      0.0000000000
1  0.0000000000     -0.7572153434      0.5865355237
1  0.0000000000      0.7572153434      0.5865355237

$END
```

One major advantage of running VB2000 in GAMESS is that the properties of VB wave function can be computed. In the log file of GAMESS / VB2000 run, the VB properties, including the dipole moment, are printed after the following line:

```
-----
PROPERTIES FOR THE VB WAVEFUNCTION
-----
```

The dipole moment is printed under the ELECTROSTATIC MOMENTS.

4.7 Tips and tricks

The following advices may help you to run VB2000 more effectively for your systems.

1. If you want to run a multi-group VB calculation, specify the VB groups explicitly using the “dot” notation, such as VB(8).CASVB(6,6).VB(8). You can specify the same kind of calculation by using GPF(ng) and other controls, but the “dot” notation is much easier to use and intuitive.
2. You can do a VB calculation of a molecule at one geometry and do another VB

- calculation on the same molecule at a different geometry by reading the VB orbitals of a previous calculation. This is not only a good way to speed up convergence of a new calculation, but also a necessary step for generating initial VB orbitals for a bond breaking calculation in some cases.
3. If a VB group has many resonance structures and you have difficulty to determine which ones make major contributions, you can try a CASVB calculation first and then check the biggest contributors.
 4. Try to avoid running CASVB or SCGVB for groups with large number of electrons ($N > 8$ for CASVB and $N > 10$ for SCGVB). If you want to run SCGVB(12), you can run a VB(12) for orbital optimization with one or a few structures, and then do a SCGVB(12) as a restart calculation without further optimization of VB orbitals (using key word CIONLY). SCGVB(11) and SCGVB(12) have 132 structures, which is just about feasible but not advised. SCGVB(13) and SCGVB(14) have 429 structures which is not possible.
 5. The spin-density calculation can be very time consuming for group with more than 12 electrons.
 6. If VBSCF for VB orbital optimization becomes unstable, you can add a constant to Hessian diagonal elements by using \$HESSCONST, starting from 0.1. A larger value can make the iteration more stable, but it can slow down the convergence. Use **PRINTALL** option to watch closely the iterations and adjust the parameter so that you can get the converged results without slowing down the iteration too much.
 7. Using the **DIIS** option can reduce the chance of symmetry broken solutions if you wish to keep certain symmetry. See Chapter 6 for more details about key word **DIIS**. However, the **DIIS** option sometimes can lead to false convergence.
 8. If the macro iteration is unstable (total energy becomes oscillating, you can set a damping factor for rigid rotation. See Chapter 6 for control **\$DAMPROT**.

We will post more tips for VB calculations on the VB2000 website. Comments and suggestions from users are highly appreciated.

5 Terminology

The following definitions and related concepts are frequently used in describing the technical details of VB2000. By getting familiar with these terms, it will be easier to understand the inputs and outputs of VB2000.

5.1 Electron group

A group of electrons moving in a particular region of a molecule. For instance, the electrons of the benzene molecule fall into three groups: the pi-electrons, the sigma-bond electrons and the core electrons. Even though electrons are indistinguishable, it is convenient to regard a number of electrons, described using a particular subspaces of basis functions, as an "electron group".

5.2 Group function

An antisymmetrized wave function describing a group of electrons in the mean field of all other groups.

5.3 System wave function

A generalized production function ([GPF](#)) describing the entire molecular system. Such a wave function can be constructed as an antisymmetrized product of group functions of all electron groups in the molecule.

5.4 Macro iteration

A round of group-by-group optimizations of group functions, followed by a 'rigid rotation' of the system wave function. Usually such a procedure is repeated until the system wave function converges. The group wave functions are optimized within their own subspace, and the rigid rotation is performed for the global optimization of the system wave function. In VB2000, the global optimization algorithm is very general and [applies no matter](#) how each group function is constructed.

5.5 Rigid rotation of a system wave function

Let us assume that the system wave function of a molecule is expressed in terms of M orthogonal orbitals (e.g. a set of Löwdin-orthogonalized orbitals). A 'rigid rotation' of the system wave function is one which all the parameters in the wave function, expressed in the orthogonal basis, are held constant (thus, the orbital coefficients, the VB structure coefficients, the one- and two-electron density matrices defined in the orthogonal basis are left unchanged) while the orthogonal basis are rotated in the M -dimensional space.

5.6 VB, VBSCF, SCGVB (SC or SCVB), BOVB, CASVB and CASSCF

There is much confusion in the literature concerning VB terminology. In this section we try to clarify the one used in our present work. Most of the differences among VB computations are purely technical. The term 'VB calculation' generally means the wave function is constructed in VB form by defining VB orbitals and their spin-pairing patterns. In modern VB calculations the VB orbitals are usually optimized in some self-consistent way and this gives rise to the term 'VBSCF method'.

In simple cases (e.g. for a saturated molecule) a single VB structure can often provide a fairly good description of the electronic structure; but cases where the chemical bonds cannot be plausibly assigned in any unique way (as in reactions, where bonds may be continually formed or broken) a single structure is inadequate. No single pairing scheme serves to give a good description of the whole structure or process and a multi-structure VB function becomes necessary. One such a choice is the so-called Spin-Coupled or, more recently renamed, Spin-Coupled Generalised Valence Bond wavefunction in VB form (SCGVB), which uses one set of VB orbitals but all linearly independent spin-pairing patterns—without any change of orbital occupation numbers (i.e. without 'excitations'). SCGVB has been proposed as a compromise between the use of “full-GVB” mainly in the USA and SC or SCVB elsewhere. We support this change.

Another approach used in describing bond-breaking/forming processes also employs a multi-structure VB function, but allows different sets of orbitals for different structures: this is the Breathing Orbital VB (or BOVB) method[26], in which the orbitals are allowed to expand or contract (i.e. 'breathe') on changing from one structure to another. The BOVB method can efficiently incorporate dynamic correlation in a VB wave function while keeping the compact form of the function.

A CASVB calculation is mathematically equivalent to CASSCF: it involves a set of VB orbitals which define the Complete Active Space and includes all linearly independent spin-coupled functions that can be constructed from them. CASVB is not designed as an efficient alternative to CASSCF, but it has some advantages that CASSCF does not provide. In particular, the orbitals have no orthogonality constraints and so can be highly localized like atomic orbitals. Since the CAS wave function is invariant under orbital transformations in the active space, we can require maximum localization of the VBOs, without constraints, in any function of CASVB type – even with a limited number of structures. The CASVB wave function is then also highly compact, a few structures providing almost all significant contributions. One should note the CASVB method implemented in VB2000 is different from Cooper's CASVB approach, in which the CASSCF wave function is transformed into VB form of a subset of CASSCF space.

VB2000 provides a very general implementation of VB theory and is capable of performing all the above types of calculation. The SCGVB and CASVB keywords in VB2000 allow one to perform the specified computation without knowing any of the details involved in constructing all the VB structures.

5.7 Group Function Theory (GFT) vs. ORMAS

To the best of our knowledge, no other electronic structure program has a general implementation of group function theory which can perform full optimization of a wave function which is expressed as a generalized product of group function. GVB and MCSCF can be considered as two special cases of GFT. The Occupation Restricted Multiple Active Space Configuration Interaction (ORMAS) method has been introduced in GAMESS(US)[27,28]. The general philosophy of ORMAS has a certain similarity to GFT in terms of multiple groups. But the two methods are actually totally different both technically and theoretically. It would be interesting to compare the two methods. The ORMAS method involves two steps: first the active space (a set of active orbitals) is partitioned into an unrestricted number of orbitals, and secondly occupation restrictions are specified in the form of the minimal and maximal number of electrons allowed for each group. ORMAS can be considered the most general implementation of restricted active space SCF method, thus all other variations such CASSCF and the 3-group RASSCF can be recovered by appropriate specifications of the restriction parameters. ORMAS efficiently eliminates ineffective configurations in CI computation while keeping the flexibility to account the electron correlations among groups. One should be aware that the flexibility for group-group electron correlation comes at the price the number of exponentially increasing of configurations - even though such an increase is considerably reduced by the restrictions.

The group function method approaches the electron correlation of large systems in a different way. It also involves two steps. The first step is similar to that of ORMAS, i.e. a molecule system can be partitioned into an unrestricted number of electron groups. The second step is quite different from ORMAS: 1) the number of electrons in each group is fixed, i.e. electrons are not hopping between different groups; 2) the wave function is expressed as the antisymmetrized product of group functions; 3) each group function is optimized in the mean field of the other groups thus the small inter-group correlation is neglected. A major advantage of this approach is that the computational cost increases only linearly with the number of groups instead of exponentially as in ORMAS. In principle, any correlation method can be used for the group function of a GF group. For instance, a GF group can be further divided into multiple subgroups and the ORMAS method can be applied to these subgroups.

6 Input Descriptions

VB2000 is designed to balance the flexibility of controls and the convenience of usability. Most of the controls and options needed are automated as defaults, which in most cases work well. This automation reduces the input to a minimum. On the other hand, many controls can be customized and will overwrite the defaults with explicit input, thus providing the maximum flexibility. One should also understand that VB2000 is not designed as a black box. Input backed by chemical knowledge is essential to obtain insights into interesting chemical systems. VB2000 is designed to provide convenient controls to convert sophisticated theory into practice so as to test, verify and refine theoretical ideas about the nature of chemical bonds. This chapter illustrates how to specify different kinds of calculations in the input files.

6.1 General considerations

The controls in the program are divided into three Levels, as follows.

Level 1: Defaults.

When controls are not set explicitly, the defaults take effects automatically, being preset according to the type of calculation.

Level 2: Global controls.

Most of the default settings can be overwritten by providing explicit inputs. There are two kinds of the global controls: (1) controls that concern the optimization of the total wave functions (including the number of subgroups, number of macro iterations, etc.); (2) controls that relate to a specific method. All groups treated by the same method can be dealt with using the same method-specific global controls. A global control can be either a key word or a dollar '\$' flag string.

Level 3: Controls for individual groups.

The optimization of the group wave function of any specific group can be controlled directly by a level 3 control. The general expression of a level 3 control is as follows:

```
###CONTROL - NAME  
CONTROL - CONTENT
```

The control consists of two parts, i.e. the control flag and the control-content. The control flag always starts with a dollar ('\$') sign, followed by a two-digit number and the name of the control (no space in between). We use upper-case letters for all control names. The two digit number (here represented by ##) is the number of the group that the control is applied to. The number starts from 01 up to 99. In most common cases, the first group is

a Hartree-Fock group, and the second one a VB group. The control content can be either empty or a multiple line input.

6.2 Basic controls and the input of a molecule

The input for a calculation starts with ‘#!’ (historically it was so to distinguish it from ‘#’ used in the Gaussian package) and followed by basic controls and molecule specification as shown below:

```
#! METHOD/BASIS_SET [OPTIONS]
[Blank]
Title or comments (can be multiple lines)
[Blank]
Charge, Multiplicity
Atomic#1, X1, Y1, Z1
Atomic#2, X2, Y2, Z2
...
[Blank]
[Other controls]
```

The first input can be continued and terminated by a blank line. Multiple lines of comments can be inserted before providing the numbers of charge and multiplicity. The comments are also terminated with a blank line. For each atom, the atomic number and its Cartesian coordinates should be provided. All these numbers can be given in free format. The atom input is also terminated by a blank line. After the atomic input, additional controls can be provided in the input. All these controls are described in the following sections.

6.2.1 Method

In the above, *METHOD* can be any of the key words explained below.

HF

If the *METHOD* is set to HF, then Hartree-Fock method will be used for the whole molecule. This is essentially a regular molecular orbital calculation.

VB(*m*)

VB method will be used for a group of *m* electrons. By default, only one VB structure will be included in the calculation. One can explicitly specify multiple VB structures by using an additional control flag \$##VBSTR (see section 6.4.3)

SCGVB(*m*)

Spin-Coupled method (recently renamed SCGVB to remove the confusion of different names – SC, SCVB and full-GVB) is used for a set of *m* electrons in *m* orbitals. All linear independent spin-configuration of *m* electrons in *m* orbitals are included in an automatic way. Therefore, there is no essential difference between SCGVB(*m*) and VB(*m*) with multiple VB structures. SCGVB(*m*) provides a convenient shortcut for a special multiple-VB structure calculation that a user don't have to write all linear independent spin

configurations explicitly. The keywords **SC(m)** and **SCVB(m)** can also be used. SCGVB is limited to 12 electrons. It seems that the changing of array bounds to have more structures is a long way short of what is needed. SCGVB(13) and SCGVB(14) have the same number of structures but the time and resources to complete that calculation makes it totally unattainable at present.

SCGVB(m, n)

We can ask ourselves: what would be the most natural way to extend Spin-Coupled method to systems where the number of electrons is not equal to the number of orbitals? In VB2000, we define it as the all spin-coupled states from all possible electron-orbital configurations which have the maximum number of occupied orbitals. For instance, in the case of SC(m,m), there only one such a configuration, that is all orbitals are occupied with one electron on each, and all spin-coupled states are from the spin-coupling schemes from this configuration. Thus $SCGVB(m, m) = SCGVB(m)$. An essential advantage of SCGVB method is that the wave function and optimized orbitals are independent of the way to choose the Lewis structures of the SCGVB space, therefore, the results is not biased to any personal preference. As with SCGVB(m), the largest value of m is 12 but SCGVB(12,11) takes longer to run than SCGVB(12,12),

One can make special use of SCGVB(m,n) when $m = 2n$, i.e. all orbitals are double-occupied. In this case, SCGVB option turns into non-orthogonal Hartree-Fock method. In VB2000, we use this trick to do non-orthogonal localized Hartree-Fock molecular orbital calculations. Even though for regular VB calculation the maximum number of electrons in each VB group is currently limited to 16, the non-orthogonal Hartree-Fock MO calculations does not have such a limitation. The limitation for the number of non-orthogonal molecular orbitals comes from the limit of total number of basis functions which can be handled by the program and the computational time. It should be noted that the non orthogonal Hartree-Fock MO computation is much more expensive than the regular Hartree-Fock method.

CASVB(m, n)

Complete Active Space VB (CASVB) is used for a set of m electrons in n orbitals. It is mathematically equal to a CASSCF(m,n) method, i.e. the wave functions produced by the two methods have the same energy and properties. The CASVB method has some unique advantages that the orbitals in CASVB wave functions are nonorthogonal and can be maximally localized.

ION(m, n) or SEN(m, n)

This provides a level of calculation between SC and CASVB based on the seniority number (SEN) or the equivalent ionicity (ION). SC has structures of maximum possible seniority number and the lowest possible ionicity. CASVB has structures of all possible seniority number or ionicity. It requires \$xxIONICITY or \$xxSENIORITY to define the range of either included, See GAMESS / VB2000 test examvb-40 or stand alone test example21.

This is an old way to specify a multi-group VB calculation, where n_g is the number of electron groups including the Hartree-Fock core. Additional controls are required for this method. This notation will become obsolete in the future release. The most convenient way to specify a multiple-group VB calculation is by the dot notation which is described in the next section.

Dot (•) notation

The dot notation has been introduced since version 1.8 for specifying a multi-group VB calculation. This notation is a more convenient and intuitive than the GPF notation. For example, `VB(8) • CASVB(6, 6) • VB(8)` specify a 3 VB group calculation. One should note that if the molecule is an open-shell system, then the last VB group must include the open-shell electrons. Only one VB group can have unpaired spins.

6.2.2 Use of libraries from VBOLIB directory.

A set of libraries, known as VBOLIBs are included in the VBOLIB directory. These are used to select the initial guess. See section 6.5. These VBOLIBs are basis set specific.

6.2.3 Basis set

a) The stand-alone VB2000.

`BASIS_SET` is the name of the basis set, which can be any one of the following:

`ST0-3G`

`ST0-4G`

`ST0-6G`

`D95` (Dunning/Huzinaga full double-zeta)

`MIDIX` (Minnesota MIDI! basis set)

`3-21G`

`3-21G*`

`6-31G`

`6-31G*`

`6-31+G*`

`6-31G**`

`6-31++G**`

`cc-pVDZ`

`AUG-cc-pVDZ`

`TZVP`

`GEN` (general basis set provided by user with a basis set file)

`GENNG` (general minimal basis set provided by user with a basis set file)

The first 15 are built-in basis sets and VBOLIBs are available for all of them. `GEN` is a flag for a user defined basis set: when specified, the program will expect additional input (after the atomic input) as follows:

```
&basis_set_file
```

where `basis_set_file` is the file name of the user-defined basis set, which should be put in the directory **BASET**. You can also provide the basis set file with the full path if the file is not in the **BASET** directory, for instance,

```
&/usr/people/jli/VB20000/GEN/basis_set_file
```

This file should be prepared in the appropriate format (see the format used for the standard basis set files in **BASET**).

GENNG is identical to **GEN** in all respects except that it is restricted to minimum basis sets, and it uses the STO-nG VBOLIB, while **GEN** does not use any VBOLIB. A program, `TOOLS/get_gen_sto6g.f`, is available to create STO-6G basis sets with general STO exponents. Details of the input is given in comments at the top of the file.

b) GAMESS / VB2000 version.

This version can use any basis set defined in GAMESS, using the \$BASIS group, the \$DATA group or an external file. However, only basis set defined in the \$BASIS group can use a VBOLIB, with one exception:

VBOLIBD95

which can be defined in the \$DATA group but D95 must be correctly given in the `!#` command line. The number of VBOLIBs is less than the number in the stand-alone version, but they are more useful as extra VBOLIBs can be created from them on the fly to add polarisation and diffuse functions, as indicated below.

VBOLIBSTO-nG

is used for any STO-nG basis set defined in GAMESS plus the STO-nG* basis sets that add a single d polarisation function to atoms heavier than Ne. This VBOLIB can also handle the MINI basis set in GAMESS up to Ne. d and f polarisation functions using NDFUNC and NFFUNC are allowed.

VBOLIB3-21G

is similar but can also be used for the rarely used 6-21G basis set. 3-21G* and 6-21G* which has a single d polarisation function on atoms heavier than Ne.

VBOLIB6-31G

can be used for 4-31G, 5-31G and 6-31G and can accommodate up to three d and one f polarisation functions and a single diffuse sp set of diffuse functions on heavy elements and a single s diffuse functions on H using the “++” nomenclature. Thus it can handle at least to 6-31++G(3d2fg,3f2dp).

VBOLIB6-311G

is similar to 6-31G but is only used for atoms up to Ne, as GAMESS uses the McLean-Chandler basis sets beyond that.

```
VBOLIBCCPVDZ
VBOLIBCCPVTZ
VBOLIBCCPVQZ
VBOLIBCCPV5Z
VBOLIBAUG-CC-PVDZ
VBOLIBAUG-CC-PVTZ
VBOLIBAUG-CC-PVQZ
VBOLIBAUG-CC-PV5Z
```

are for the correlation consistent basis sets and their related augmented friends. They are extended in only one way and that is always when they are used for 2nd row atoms. Here an extra set of diffuse d orbitals is added in GAMESS to give the cc-pV(n+d)Z and aug-cc-pV(n+d)Z basis sets. Note that this is different from the behavior of Gaussian09.

VBOLIBMIDIX

can be used for the MIDI basis set.

Note that GAMESS always uses a basis set of cartesian functions, even though the result for spherical harmonics can be obtained in GAMESS using ISPHER=1 and in VB2000 with the SPHER directive. All VBOLIBs use a cartesian function basis.

c) General comments.

In the GAMESS version the VBOLIBs may not work if used in situations that have not been tested. Various constraints are imposed in the code where we are aware that beyond them the VBOLIBs can not be used. However, we probably have not yet caught all of them. We welcome input from users to help us extend the use of VBOLIBs to other basis sets defined in the GAMESS versions and to situations not already covered.

```
$LSTVBOLIB
```

directive lists the selected VBOLIB.

```
$NEWLIB
XXXXX
```

selects a VBOLIB different from the basis set in the command line and can be used when the basis set is input directly, An example would be XXXXX replaced by 6-311G, where the input basis set is 6-311G+ or 6-311G++ or even with more polarization or diffuse functions. The new VBOLIB must be a subset of the correct VBOLIB for the

basis set selected (i.e the n lines of the new VBOLIB must be the first n lines of the correct VBOLIB. Thus VBOLIB6-31G can be used for all basis sets that are 6-31G plus any number of polarization and diffuse functions.

6.2.4 Options

[**OPTIONS**] are key words that control the overall calculation.

PRINTALL

With this option, a lot more messages will be printed during the computation; for example, the initial VB orbitals, Mulliken population of LMOs and VBSCF iterations

RESTART

This option is originally designed for restarting a computation which is terminated premature. However, we find a more useful usage of RESTART is to read VB orbitals from a different calculation as the initial VB orbitals for a new calculation. A typical scenario of such a usage is generating the initial VB orbitals for a bond breaking process. In the dissociation limit, the Hartree-Fock calculation breaks down, and therefore the LMOs based on Hartree-Fock molecular orbitals becomes less meaningful. As a consequence, the method for generating initial VB orbitals from LMOs becomes problematic. The RESTART option can be used to solve the problem: do a VB calculation of the molecule at the normal bonding geometry, and then do same type VB calculation at the bond breaking geometry using the RESTART option. Also see the \$RESTARTFILE control. A VB restart file of a molecule can be even used for another VB calculation of the same molecule but with different number of electrons and geometry. Also see \$RESTARTMAPPING for more details.

CIONLY

This option disables VB orbital optimization during the VBSCF iteration. With this option, a multi-structure VB calculation becomes essentially a VBCI. However, this option does not disable VB orbital optimization completely. The overall wave function optimization involved the in-group optimization (VBSCF) and rigid rotation for group-group optimization. The CIONLY option has no effects on the rigid rotation. To disable rigid rotation, you need use the keyword: NOROT.

NOROT

There are situations that one needs to disable the rigid rotation between groups. The keyword NOROT is designed for this purpose. However, both CIONLY and NOROT does not disable the optimization of Hartree-Fock orbitals. To disable the Hartree-Fock orbital optimization, the following keyword is required.

FROZEN

Freeze all orbitals in the Hartree-Fock group

UNITS=BOHR

This keyword is required if the unit of the the 3D coordinates is Bohr. The default unit is Å

SPDEN

This keyword is required for the computation of spin density of open shell system. For close shell systems, this keyword has no effects. In the current implementation, the spin density calculation can be significantly slow down for VB groups with more than 12 electrons.

DIIS

This option can be used to stabilize the VBSCF iteration based on Direct Inversion of the Iterative Subspace (DIIS) method. The Newton-Raphson (NR) method implemented in VB2000 usually has very good performance for orbital optimization. However, there are cases that the VBSCF iteration becomes unstable. The DIIS option can stabilize the pure NR iteration. It is quite useful for BOVB type of calculations where multiple structures are included and different orbitals sets are used for different structures. We have also found that the DIIS option helps to maintain symmetry.

SPHER

This option is used to enforce the use of spherical harmonics basis sets. GAMESS and the stand-alone VB2000 use cartesian functions for basis sets internally. Some basis sets, such as cc-pVnZ, are officially defined as spherical harmonics. To avoid confusion as well as to keep consistency from one program to another, it is required to stick to the same definition for these basis sets. GAMESS enforces the use of spherical harmonics basis set by the option ISPHER=1, but the VB2000 plugin will still use cartesian functions, unless SPHER is used in the command line of the \$VB2000 block. In VB2000, the SPHER keyword is used for the same purpose. The SPHER keyword can not be used in the GAMESS version if the basis set contains h or I basis functions. It is unlikely that this will be implemented as the use of cartesian functions with h or I basis functions is not recommended as it leads to linear dependencies and a great lose of efficiency. SPHER can not be combined with the use of BRILLMASK.

TEST

This option is usually used to check the LMOs before doing a complete VB calculation. With this option, the program does the Hartree-Fock molecular orbital calculation, orbital localization and then stops. By checking the order and the characteristics of the LMOs one can decide whether it is necessary to use more controls or not to adjust the LMO assignment. TEST runs can use the various visualization routines, such as \$CUBE. The data produced allows you to visualise the LMOs and thus help in checking them.

6.3 Group function control flags

6.3.1 \$GENCTL

This is the main control for group function specification. The input format is as follows:

```
$GENCTL  
(MELE(i), i=1, NG)  
(METHOD(i), i=1, NG)
```

Where *NG* is the number of electrons groups (including the Hartree-Fock group). *MELE*(*i*) is the number of electrons of group *i*, and *METHOD*(*i*) is the method for obtaining wave function of group *i*. The following values of method are possible:

- 1 for HF
- 2 for VB (may contain multiple VB structures)
- 3 for SC
- 4 for CASVB
- 5 for IONICITY or SENIORITY

\$GENCTL is usually combined with the key work **GPF(n)** in the input. If **GPF(n)** is specified and the control flag **\$GENCTL** is missing, then the molecule is treated with (n-1) GVB pairs. Please note that by default, the first electron group is a Hartree-Fock group. Therefore, if a system is divided into n groups, only n-1 groups are expected to be treated with VB methods. If no specification is provided for the n-1 groups, then they are assumed to be GVB pairs.

6.3.2 \$GRPDIM

This flag specifies the dimension of the subspace for each electron group. The format of this control is as follows:

```
$GRPDIM  
(NDIM(i), i=1, NG)
```

where *NDIM*(*i*) is the dimension of the basis for group *i*. In most cases, the default values for all groups will be generated automatically by the program. For instance, with a (close shell) Hartree-Fock group, the dimension should be equal to half the total number of the electrons in the Hartree-Fock group. If the explicit input number is different from that, an assertion error will be detected and the computation will be aborted.

6.3.3 \$VBGA (or \$VBGROUPASSIGNMENT)

Two \$ flags are introduced for this option: **\$VBGROUPASSIGNMENT** or a short notation as **\$VBGA**. This option provides an intuitive electron group assignment for valence electrons. The following example shows how the control is applied.

Example 6.1. VB calculation of H₂O with two O-H bonds (file TESTINP/extra/extra03_h2oVBGA.inp).

```

#! VB(4)/D95 PRINTALL

TEST H2O

0 1
8 .000000 .000000 .000000
1 .801842 .000000 .555582
1 -.801842 .000000 .555582

$VBGA
1-2 => 2
1-3 => 2

$02VBORB
1-2
1-3

```

In this example, the \$VBGA input has two entries. The first one, 1-2 => 2, means that the electrons of the σ bond LMO between atom 1 and 2 are assigned to (=>) electron group 2 (here group 2 is the VB group). The second entry means that the σ bond between atom 1 and 3 is assigned to group 2. One should note that for such a simple molecule, the \$VBGA and \$02VBORB inputs can be omitted, and the LMO groups assignment and VBOs are generated automatically. The next example shows how to explicitly specify the group assignment and how to generate VBOs. For multiple bonds, for instance the triple bond in C₂H₂, one can assign all the three bonds to the same electron group with one entry as follows:

Example 6.2. VB calculation of the triple bond in C₂H₂ (file TESTINP/example05_c2h2sp.inp).

```

#! VB(6)/D95

TEST C2H2 TRIPLE BOND (SIGMA+PI)

0 1
6 .000000 .000000 .610009
6 .000000 .000000 -.610009
1 .000000 .000000 1.690006
1 .000000 .000000 -1.690006

$VBGA
1#2 => 2

```

the \$VBGA input is equivalent to the following:

```

$VBGA
1-2(1) => 2
1-2(2) => 2
1-2(3) => 2

```

The first entry means the first (σ) bond is assigned to group 2. The second entry means

that the second (i.e. the first π bond) is assigned to group 2. The last entry assigns the third bond (the second π bond) to group 2.

6.3.4 \$LMOGRPMODIFY

The **\$VBGA** (or **\$VBGROUPASSIGNMENT**) control is based on the automatic LMO classification, which assigns each LMO as a bonding orbital between two atoms or a lone pair (including the inner core) orbital on an atom. However, there are cases where such an automatic LMO assignment does not work well. For instance in the multi-center bonds and some conjugated systems, the LMOs are not necessarily localized on one or two atoms. Therefore the **\$VBGA** logic does not apply for such LMOs. To deal with cases of this kind, a less intuitive but more flexible control has been implemented in VB2000. This control explicitly assigns each LMO to a specific electron group. The general control format is as follows:

```
$LMOGRPMODIFY
NLMO
N1, G1
N2, G2
. . . . .
```

Where *NLMO* is the number of LMOs, followed by *NLMO* pairs of integers (*N_i*, *G_i*). *N_i* is the LMO index, and *G_i* is the electron group that the *N_i*-th LMO belongs to.

6.3.5 \$MACROITER

The default maximum number of macro iterations is 12. If this default number needs to be changed, then **\$MACROITER** flag can be used as follows:

```
$MACROITER
MITER
```

where *MITER* is the new number of macro iterations.

6.3.6 \$ECONV

The default energy threshold for the total energy of a molecule is 10^{-6} a.u. This can also be overwritten by this control flag as follows:

```
$ECONV
n
```

and the new energy threshold will be 10^{-n} a.u.

6.3.7 \$ROTATION

The optimization of a molecule wave function expressed as a generalized product function involves the optimization of each group function within its corresponding subspace and the Jacobi rotations between the basis functions of different groups. By default, only one rotation is performed for each pair during each macro iteration. However, this can be changed by using this control flag:

```
$ROTATION
nRot
```

where *nRot* is the number of Jacobi rotations for each macro iteration. The default value is 1. In some cases, by increasing this number (say 2-5), the macro iteration can be made more stable. However, a large value of *nRot* also means a much more expensive computation.

6.3.8 \$NOTROT

There are some cases where we need to disable the Jacobi rotation for basis functions of two groups, such as the σ and π groups in a conjugated system. This can be done in the following way:

```
$NOTROT
NR
NA1, NB1
NA2, NB2
. . . .
```

Where *NR* is the number of group pairs to be disabled, and followed by the *NR* pairs. The first pair is group *NA₁* and group *NB₁*, and so on. This option may make the program more efficient and converge faster. To give an example, benzene can be divided into two electron groups, the σ -electron group and the π -electron group. A set of virtual orbitals can be used to define a third group, containing no electrons. The optimization of the system wave function involves the inter-group rotations of the subspaces of all groups. Clearly, however, the rotation between σ and π orbitals should be avoided for benzene; and this can be achieved by turning off the inter-group rotation between the σ and π groups. This option is used whenever a molecule has such kind of symmetry, thus avoiding inter-group rotations that would break the symmetry. For more details, please check the example inputs.

6.3.9 \$FULLHESS

The Jacobi rotation involves the evaluation of a Hessian matrix, which is usually an expensive procedure. To improve the efficiency, our experience shows where only evaluating the block diagonal Hessian matrix elements can be as good as the full Hessian matrix in most cases. Therefore, by default, only block diagonal matrix elements are evaluated, and any matrix elements involving rotations of more than two groups are ignored. However, there are cases that the full evaluation of Hessian matrix is essential, especially for systems with unusual bonding, as in chemical reactions. To enable the full Hessian evaluation, just put the above control flag in the input.

6.3.10 \$DAMPROT

Occasionally, one may see the macro iteration becomes unstable. This may happen for BOVB type calculations. If the total energy of the wave function oscillates, then a damping factor for rigid rotation will most likely solve the problem. The damping factor *d* should be in the range ($0 < d < 1.0$). A small factor means a strong damping effect, and

can also significantly slow down convergence. A good starting point is 0.5. For instance, you can add the following control parameter in the input:

```
$DAMPROT
0.5
```

6.4 VB Orbital optimization controls

6.4.1 \$VBSCF

This is a global SCF control for VB groups. The general formation of the input is as follows:

```
$VBSCF
EPS, MaxIter
```

Where *EPS* is the threshold for the maximum change of orbital coefficients of VB orbitals, and *MaxIter* is the maximum number of VBSCF iterations. This control applies to all VB groups, unless the specific values are specified for a particular VB group. The default values of *EPS* and *MaxIter* are 0.0001 and 15, respectively. Also see the next control for a specific VB group.

6.4.2 \$##VBSCF

This control is very similar to the above control, but it applies only to a specific group as denoted by the number. The general format of this control is:

```
$##VBSCF
EPS, MaxIter
```

Where *##* is a two digit number of a group index. The default values are specified by \$VBSCF.

6.4.3 \$##VBSTR

This control is used for VB structure specification of a group. The general format of this control is:

```
$##VBSTR
NSTR
N1 1 N1 2 . . . . . N1 N
N2 1 N2 2 . . . . . N2 N
. . . . .
Ni 1 Ni 2 . . . . . Ni N
. . . . .
NNSTR 1 NNSTR 2 . . . . . NNSTR N
```

NSTR is the number of VB structures for a VB group, *N* is the number of electrons in this VB group. Each line specifies a VB structure, Each structure is specified by a set of orbital indexes. In VB2000, the following convention is used for the pairing pattern of each VB structure: If the total spin of the electrons of the current group is *S*, and the

number of electrons is N , then there are $(N-2S)/2$ pairs, and $2S$ unpaired electrons. The pairing pattern of the first structure can be interpreted in the following way:

```

N1 1 - N1 2
N1 3 - N1 4
.....
N1 N-2S-1 - N1 N-2S
N1 N-2S+1
N1 N-2S+2
.....
N1 N

```

Note: The indexes of orbitals in the symbolic VB structure specification refer to the *index of the orbitals of the group*. For instance, if an index is 1, it refers to the first orbital in this group. Here the double sharp (##) stands for a two digit number. For instance, **\$02VBSTR** stands for the VB structure of group 2. Note: the program expects a two-digit number even it is less than 10.

Let's consider an example. The two Kekule structures and three Dewar structures of benzene can be represented as:

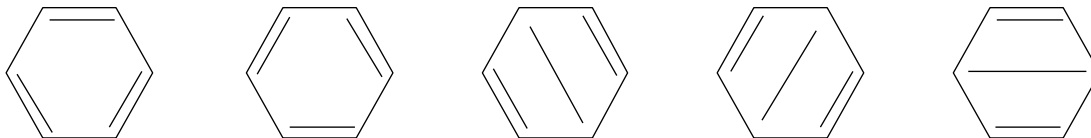


Figure 6.1. VB structures of the pi system of benzene.

The 5 VB structures can be given as follows:

```

$02VBSTR
5
1 2 3 4 5 6
2 3 4 5 6 1
1 4 2 3 5 6
2 5 3 4 1 6
3 6 1 2 4 5

```

The first two are Kekule structures, and the last 3 are Dewar structures.

6.4.4 \$HESSCONST

This control is used to increase the stability of VB orbital optimization by adding a constant to Hessian diagonal elements. The formation of this input is:

```

$HESSCONST
D

```

where D is a constant, suggested value is 0.1. A larger value can make the iteration more stable, but also slow down the convergence. For difficult cases, one can try to divide the computation into multiple restart runs, and watch carefully the convergence behavior by using **PRINTALL** option. Once the convergence becomes stable but slow, one can reduce the constant D , or even set it to zero to speed up the convergence.

6.4.5 \$BRILLMASK

This control is used for VB orbital optimization under strict localization constraints. This control specifies a set of orbital pairs that are *not* allowed to be mixed during orbital optimization. Generally, only orbitals on the same atom are allowed to be mixed during orbital optimization. The resulting VB orbitals are strictly localized. To make this work, the initial VB orbitals must be strictly localized. One can start from a set of AOs as initial VB orbitals, and then each VB orbital is optimized within the basis functions of the atom that the initial VBO belongs to. The general format of this control is as follows:

```
$BRILLMASK
M
NA1, NB1
NA2, NB2
. . . . .
NAM, NBM
```

Where M is the number of orbital pairs that the orbital mixing between the two is to be avoided. For instance, orbital NA_i does not have any contribution from orbital NB_i during orbital optimization. The orbitals NA_i have to be all the orbitals specified in the definition of the VB structures using \$02VBSTR for example.

An extension to **\$BRILLMASK** makes this process automatic for the case where all the VB orbitals are to be localized on a single atom. If the directive is followed by a blank line, or $M=0$, the initial orbitals must be defined using the \$AOGROUP directive and the orbitals used in defining the structures must be all at the beginning of the list of AOs. Each of these will be mixed only with AOs centred on the same atom. No further data is required as the atoms the orbitals are centered on can be obtained from the \$AOGROUP data.

This automatic case can also be used without using the \$AOGROUP group, but the atoms the orbitals are centered on must be obtained elsewhere. Alternatives to the \$AOGROUP group are to get the orbitals from a RESTART or from the \$##VBORB group using AOs. In the latter case, care must be taken to define all orbitals, but it does give the opportunity to define a better initial guess than the single basis functions of \$AOGROUP allow. In these cases, data has to be provided that indicates the atom that each VBO is to be centered on. The data is:

```
$BRILLMASK
-NP
NA1, NA2 ... (NP values)
```

The negative number in the line after the directive distinguishes this case from the original case. NP is the total number of VBOs defined from the RESTART or in the \$##VBORB group. This is followed by NP integers being the atom number that each VBO is localized on. Care must be taken in using the \$##VBORB group, to ensure that orbitals used there are compatible with this data.

The BRILLMASK directive can not be combined with SPHER.

6.4.6 ###LENHANCE

This is a localization enhancement control. For some applications, the strict localization is a too strong constraint imposed on the orbital optimization and can lead to much higher energy of wave functions than that from free optimization. It is also technically cumbersome to set up the calculation and can only be applied in very limited systems. Therefore a more flexible and general way to obtain localization enhanced VB orbitals is needed. In VB2000 2.0, a new algorithm is implemented. In this new algorithm, a penalty function of delocalization of a set of VB orbitals is added to the total energy of the VB wavefunction. The delocalization of a VB orbital is defined as the Mulliken population of the VB orbital on atoms which are not in the localization target of the orbital. The VB wavefunction is optimized by minimize the following objective function:

$$P = E(VB) + W * (\sum_{i=1}^N P_i)$$

where P_i is the delocalization penalty of VB orbital ϕ_i defined as follows:

$$P_i = \langle \phi_i | \overline{S}_{X_i} | \phi_i \rangle / \langle \phi_i | S | \phi_i \rangle$$

S is the overlap matrix of basis functions, ϕ_i is the VB orbital localized on atom X_i (or a set of atoms X_i), and $\overline{S}_{X_i}$ is the delocalization overlap matrix defined as follows:

$$[\overline{S}_{X_i}]_{kl} = S_{kl} \text{ (if } k \vee l \notin X_i, \text{ otherwise } 0)$$

i.e. the delocalization overlap matrix $\overline{S}_{X_i}$ is from the original overlap matrix S by setting the rows and columns of basis functions of X_i to zero.

Such a penalty can be scaled by a weighting parameter. The general format of this control is as follows:

```
###LENHANCE
M
VBO1, N1, A1, A2, ... AN1
VBO2, N2, B1, B2, ... BN2
...
```

where M is the number of VBOs to be localized. By default, the delocalization penalty weight W is set to 0.1, and can be modified by **SDPWEIGHT** input (see the next section). The following lines specify how each VBO will be localized. For instance, for VBO_1 , it is localized on N_1 atoms for the atom list $A_1, A_2, \dots A_{N1}$. There are M lines to specify the localization of M orbitals.

This control for obtaining more localized VB orbitals has the following advantages.

- It is very general. It does not require any special orientation of a molecule and pre-classification of atomic basis functions.

- It is very flexible. The penalty weight control how much localization is needed. By setting a very large weight, the optimized VB orbitals are close to the strictly localization. We recommend **0.1** as a starting point. It is also recommended that the same weight is used for a comparison study of a number of molecules. One can also use the total energy change and the delocalization coefficients to make good judgment for a reasonable weight.
- The input is intuitive and relatively easy to set up. One only needs to specify which atoms a VB orbital should be localized on. Such a control does not need the details of the basis set used in the calculation.
- By increasing the delocalization penalty weight to an extremely large value, say 10^6 , one can get strictly localized VB orbitals. See the example input: TESTINP/h2bovb.inp. However, one should note that it is possible to get strictly localized VB orbitals for only one VB group, unless the multiple VB groups are separated by molecule symmetry (such as σ - π systems). Due to strong orthogonality constraints, it is generally impossible to keep orthogonality between groups and strict localization of orbitals at the same time.

6.4.7 \$DPWEIGHT

This controls the delocalization penalty weight for localization enhanced VB orbital optimization. The format is as follows:

```
$DPWEIGHT
W
```

where W is the delocalization penalty weight defined in the previous section. Without this input, the default value is 0.1 for localization enhancement option.

6.4.8 \$CMAXCUT

This controls the Hessian matrix evaluation of VBSCF iterations. If the maximum coefficient change is less than the threshold specified by \$CMAXCUT, then the Hessian matrix of previous iteration is used instead of recomputed. This is especially important for VB groups with more than 10 electrons. However, if the region described by the VB wave function is non-classical (i.e. the Lewis structures are non-classical), one should use a more accurate Hessian by using a smaller threshold. To force full Hessian matrix evaluation for each iteration, one can set the threshold as 0.0. The default value is 0.025.

6.4.9 \$NOLOCVBO

By default, the maximum VBO localization will be performed at the last macro iteration of a CASVB calculation. This is designed to eliminate the intrinsic invariance of VBOs of CASVB space. In most cases, the CASVB calculation leads to well localized AO-like VBOs even without maximum localization. However, one can turn off the maximum localization of CASVB by adding the above control flag in the input. No extra data is needed for this control.

6.5 Initial VB orbital generation

6.5.1 `$$$VBORB`

This control flag provides a very flexible and intuitive way to specify a set of initial VB orbitals for a specific group. The `##` is a two digit number of the group index. The general format of this control is as follows:

```
$$$VBORB
VB0-Specification-1
VB0-Specification-2
...
```

Each VBO specification line specifies one or more VB orbitals, several lines can be used and the control is ended with a blank line. Each VBO specification can take one of the following forms:

- a) `NA->NB`
- b) `NA-NB`
- c) `NA:(n)`
- d) `NA^(n)`
- e) `NA=NB`
- f) `NA#NB`
- g) Expression

Where *NA* and *NB* are atom indexes. *NA->NB* stands for a sigma orbital on atom *NA* pointing to atom *NB*. *NA-NB* stands for two orbitals forming a single bond between atom *NA* and *NB*. *NA:(n)* stands for the *n*-th lone pair on atom *NA*. *NA^(n)* stands for the *n*-th π orbital on atom *NA*. *NA=NB* and *NA#NB* stands for 4 and 6 orbitals for double and triple bonds, respectively. For more details, please go to Chapter 7. The last case stands for general expression for initial VB orbitals. See more details in the following section.

6.5.2 General expression

The most general way to specify an initial VB orbital is to express it in a linear combination of atomic orbitals. VB2000 version 2.1 provides a simple and intuitive way for this purpose. The general expression of an orbital is shown as follows:

```
$$$VBORB
c1 AO n1 + c2 AO n2 + ... ci AO ni + ...
.....
```

where c_i is a coefficient for atomic orbital n_i , denoted as *AO n_i*. Generally speaking, there is no limitation for the number of terms in the expression, however, in practice, all expression must fit into one line for each VB orbital. A good example can be found in `TESTINP/example23_mcsc.inp`:

```
$04VBORB
0.32 AO 6 +0.22 AO 9 +0.49 AO 26 +0.31 AO 29
0.32 AO 6 +0.22 AO 9 +0.49 AO 41 +0.31 AO 44
0.71 AO 6 +0.30 AO 9
0.16 AO 9 +0.37 AO 26 +0.30 AO 29 +0.37 AO 41 +0.30 AO 44
```

The order of the terms is not important. A possible scenario of using such expressions is that one already has a good initial guess of VB orbitals using whatever methods, and want to use this guess as an input. Since the expression is limited one orbital per line, you have to truncate the expression to only a few terms with the largest coefficients.

6.5.3 \$##PIVBO

Even though the control flag **\$##VBORB** can be used to specify a set of π orbitals, a more convenient way to specify a set of π orbitals is the PIVBO flag. The general format is as follows:

```
##PIVBO
nOrbitals
n1, n2, n3 ....
```

where ## stands for a two digit number of the group index, *nOrbitals* is the number of π orbitals, and *n1,n2..* are the atom indexes on which the π orbitals are located. It is assumed that the atoms specified in this control form a conjugated plane (or approximately plane), and the orbital orientation of each π orbital is perpendicular to the plane of the atom and its connected conjugate atoms.

6.5.4 \$LMODISTORTION (or \$DISCON)

This control specifies the distortion constant for splitting bonding LMOs. The general input format is as follows:

```
$LMODISTORTION
SplitPara
```

Here *SplitPara* a splitting parameter (range 1.0 – 0.0, and the default value 1.0 gives the maximum split). Assume a bonding LMO localized between atom A and B, can be expressed as following

$$LMO(A-B) = \begin{pmatrix} C_{A1} \\ C_{A2} \\ \dots \\ C_{B1} \\ C_{B2} \\ \dots \end{pmatrix}$$

Where $C_{A1}, C_{A2} \dots$ are the orbital coefficients on atom A and $C_{B1}, C_{B2} \dots$ on atom B. With the default value, the LMO can be split into two VBOs, one is localized on atom A and the other on atom B as follows:

$$VBO(A \rightarrow B) = \begin{pmatrix} C_{A1} \\ C_{A2} \\ \dots \\ 0 \\ 0 \\ \dots \end{pmatrix}$$

and

$$VBO(B \rightarrow A) = \begin{pmatrix} 0 \\ 0 \\ \dots \\ C_{B1} \\ C_{B2} \\ \dots \\ \dots \end{pmatrix}$$

Experiences show that the default value works fine in almost all cases. A small value of the splitting parameter means that the two generated VB orbitals are very close. A zero value means two VB orbitals are identical, and it may cause the VBSCF unstable.

6.5.5 \$WRITEGUESS

This control specifies a file name for writing an initial guess of the orbitals into a file. The input format is:

```
$WRITEGUESS
FileName
```

where *FileName* is the file name for the generated initial guess of VB orbitals. The file has the following format:

```
Nbasis, Norbitals
((orbital(i,j),i=1,Nbasis), j=1,Norbitals)
```

where the first two integers are the number of basis functions and the number of orbitals, respectively, *orbital(i,j)* are AO coefficients of orbitals. The file is in free text format, and can be manually edited. The file can be read for the initial guess by VB2000. This can be useful in that one can modify some of the VBOs manually, and restart the computation by reading the file for the initial orbitals by using another control flag \$READGUESS. Note: all occupied orbitals (including both HF orbitals and VB orbitals) are exported.

6.5.6 \$READGUESS

This control can be considered as a complementary of \$WRITEGUESS. It controls the initial guess to be created from a disk file generated by a previous computation with

control \$WRITEGUESS. Input format:

```
$READGUESS  
FileName
```

Here *FileName* is the file name of the initial guess of the orbitals. The file is a free format text file. Even though the automatically generated initial orbitals are normally quite robust, there are some cases where user-defined orbitals may be required. This option provides a convenient way of loading the initial guess, which may have been generated in an independent program, or even created manually.

A similar directive \$READVEC reads the initial guess from a GAMESS \$VEC

For VB2000 stand-alone version, the syntax is:

```
$READVEC  
FileName  
NORB
```

It reads from file specified in FILEGUESS, with NORB orbitals. For GAMESS version it reads \$VEC group in the input file. The syntax is:

```
$READVEC  
NORB
```

The directive \$READMOLDEN in the stand alone version reads the geometries, basis set and molecular orbitals from a *.molden file with same jobname. The directory molden/ contains some example files of this feature. See the file molden/molden.README.

6.5.7 \$RESTARTFILE

Each VB2000 calculation will generate a binary restart file, which stores all necessary information for a restart calculation. The file contains the 1D density matrix for each electron group and the VB orbitals. This can be useful in case of a not fully converged computation to start a new computation for the same molecule based on a previous computation. If the key word "RESTART" is specified in the input, then the program is expecting a restart file, and by default this restart file is the input file name with extension .V84. To restart the computation from a different file, one needs the control flag to specify the restart file in the following way:

```
$RESTARTFILE  
RestartFileName
```

Please note that the restart file can be used as a way to provide initial VB orbitals for a different calculation of the same molecule but with a different geometry and even different electrons and methods.

6.5.8 \$RESTARTMAPPING

The restart file not only allows a user to restart a computation with the same electron group partitioning, Version 1.8 also allows a restart of a computation with different

electron group partitioning. To make this to work, another control, which tells how to correlate the electron group partitioning of a previous computation and the current one, is needed. The control is \$RESTARTMAPPING. Two or more VB groups in a previous calculation can be combined into one VB group in the second calculation, and the LMOs in a Hartree-Fock group can be re-assigned to a VB group in the second calculation. The general format is as following:

```
$RESTARTMAPPING
NHFG(i), (i=1.... NHFX)
NGP(j), (j=1 .... NGPX)
```

where *NHFX* is the number of HF orbitals of previous calculation, and *NGPX* is the number of groups of previous calculation. *NHFG(i)* is the group assignment of the *i*-th HF orbital of previous calculation, and *NGP(J)* is the group assignment of the *j*-th group of the previous calculation. Please note that if *NGP(1)* is Hartree-Fock group, then all of these orbital assignments are controlled by *NHFG(I)*.

This can be explained by a two-step calculation on ClO₂. The first step is a GPF(3) calculation, with one Hartree-Fock group and two VB groups:

Example 6.3. Group function description of σ - π system of ClO₂ (file TESTINP/extra/extra04_Cl02.inp):

```
#! GPF(3)/D95 PRINTALL

TEST Cl02

0 2
17 .000000 .000000 .37
8 .0 1.260000 -.39
8 .0 -1.260000 -.39

$GENCTL
24,4,5
1, 2, 4

$GRPDIM
12,4,3

$LMOGRPMODIFY
17
1,1
2,1
3,1
4,1
5,1
6,1
7,1
8,1
9,1
10,1
11,3
12,1
13,1
14,3
```

```

15, 3
16, 2
17, 2

$03VBORB
1: (2)
2: (3)
3: (3)

$02VBORB
1->2
2->1
1->3
3->1

```

Group 2 has 4 electrons in 4 VB orbitals, corresponding to the two Cl-O bonds. Group 3 has 5 electrons in 3 VBOs. Each O has three LPs. The first LP on O is opposite to the Cl-O bond, the second LP is perpendicular to the Cl-O bond and in the O-Cl-O plane, and the third LP is perpendicular to the O-Cl-O plane (also see the rules in Chapter 7). For group 3, the CASVB method (Method No. 4) is used in order to keep the localization of VBOs as tight as possible. It is usually more efficient and more stable to divide a system into a number of subsystems.

In the next step, we want to perform a VB calculation by combining groups 2 and 3 of the previous calculation into one VB group with two more LPs from the two oxygen atoms. To do this, one needs to specify explicitly how to map the VB groups and LMOs of the previous calculation into the groups of a new calculation. This can be controlled by \$RESTARTMAPPING:

Example 6.4. Restart calculation for σ - π system of ClO_2 (file TESTINP/extra/extra05_ClO2rst.inp):

```

#! VB(13)/D95 RESTART CIONLY PRINTALL

TEST ClO2

0 2
17 .000000 .000000 .37
8 .0 1.260000 -.39
8 .0 -1.260000 -.39

$GENCTL
20,13
1, 2

$GRPDIM
10,9

$02VBSTR
9
1 1 2 2 3 4 5 6 8 8 9 9 7
1 1 2 2 3 4 5 6 7 7 8 8 9
1 1 2 2 3 4 5 6 7 7 9 9 8
1 1 2 6 3 4 5 5 8 8 9 9 7
2 2 1 4 5 6 3 3 8 8 9 9 7

```

```

1 1 2 6 3 4 5 5 7 7 8 8 9
2 2 1 4 5 6 3 3 7 7 9 9 8
1 4 2 6 3 3 5 5 8 8 9 9 7
1 2 4 6 3 3 5 5 8 8 9 9 7

$RESTARTMAPPING
1,1,1,1,1,1,1,1,1,1,2,2
1,2,2

$MACROITERATION
1

```

The first line specifies the new group assignment of the 12 LMOs of the Hartree-Fock group of the previous calculation. The first 10 still belong to group 1 (i.e. the H-F group), and the last two are assigned to group 2 (i.e. the VB group). The second line is the group-wise assignment. Thus group 1 of the previous calculation is still assigned to the new group 1 (modified by the first line specification), and groups 2 and 3 are combined into the new group 2. A question is: how can we know that the orbital 11 and 12 of the first line are the ones we want to included in the VB group (the second group)? The orbitals 11 and 12 in the final printout of the previous calculation are identified to be the second LPs of the two oxygens by comparing the similarity of the starting LPs and the resulting LPs.

6.5.9 \$AOGROUP

Even though the optimal system wave function can be obtained from a reasonably good initial guess, there are cases where a classification of atomic orbitals can be helpful. A typical situation is the σ - π separation in a conjugated system. In such a case, the system can be divided into a σ group and a π group, and one may wish to avoid σ - π mixing during the optimization. To achieve this goal, a control is used to assign atomic orbitals into different groups, and group functions are then optimized within their corresponding subspaces. The input format is as follows:

```

$AOGROUP
NAO
NA1, NA2, . . . . . NANAO
GA1, GA2, . . . . . GANAO

```

where NAO is the number of atomic orbitals to be classified. The next line is the atomic orbital indexes, and following the atomic indexes are the group indexes GA of their corresponding atomic orbitals. With this control, the pure atomic orbitals are used as the initial orbitals for the group wave function.

6.6 Control for a chemical reaction

6.6.1 \$REACTION

This control is used to specify the energy profile calculation along a chemical reaction path. It is assumed that the reaction path has been obtained, or a reasonable guess has been made. The general input format is shown as follows:

```
$REACTION
IPOINT, NGEOM, NATOMS
GEOM1
atomic#1      x11      y11      z11
atomic#2      x12      y12      z12
...
atomic#n      x1n      y1n      z1n
GEOM2
atomic#1      x21      y21      z21
atomic#2      x22      y22      z22
...
atomic#n      x2n      y2n      z2n
...
```

where *NGEOM* is the number of geometries of the reacting system along the reaction path, and *NATOMS* is the number of atoms in the reacting system. The atomic numbers are the nuclear numbers of atoms, *x*, *y* and *z* are the coordinates of atoms. *IPOINT* is the number of geometric interpolation points between each two consecutive structures along the reaction path. In a typical calculation, three geometries are used, include the geometries of reactants, the transition state, and the products.

The directive \$XYZREACTION creates a single file jobname-full.xyz of the coordinates at each point. The coordinates are those selected for the run or in GAMESS specifically in the VB2000 command line with UNITS-BOHR to specify the coordinates for the defined points. The energy of each point is also written in the file, so MacMolPlt (from version 7.7.1) can make an energy plot. See \$XYZFILE to give separate files for each point.

6.7 Advanced controls

6.7.1 \$##ROOT

The number of the CI root for an excited state. In most cases, the ground state is calculated, thus this input can be omitted. The general format of this control is:

```
$##ROOT
NROOT
```

Where ## stands for the two digit number of an electron group, *NROOT* is the number of the CI root for an excited state, in ascending order. For instance, if *NROOT* is 2, the first excited state of group ## will be optimized. This is only meaningful when the electron

group is described with multiple structure VB wave function (for instance, a CASVB wave function).

This method only works well for the case where the excited state and the group state have different symmetry (i.e. symmetry vs. anti-symmetry) so that the excited state and the group state are always orthogonal, otherwise, the excited state may collapse into the ground state during the optimization.

6.7.2 \$\$\$STRSYMM

This control specifies the symmetry constraints on VB structure coefficients. The general format of the control is:

```
$$$SYMSYMM
NP
IA1, F1, IB1
IA2, F2, IB2
.....
```

where *NP* is the number of constraints, followed by the *NP* entries of the constraint specification. Each line specifies a symmetry related equality constraint as following:

<i>IA_i</i>	Index of structure coefficient
<i>F_i</i>	The symmetry factor, which can be either 1 or -1.
<i>IB_i</i>	Index of structure coefficient symmetrically related to <i>IA</i>

Assume the structure coefficients are denoted as *CSTR(i)*, where *i* is the index of the structure, then the constraint is

$$CSTR(IA_i) = F_i \times CSTR(IB_i)$$

6.7.3 \$MEMORY

This control is used to specify the size of dynamically allocated memory for the calculation. Following this \$ flag is the memory size in double-precision. For instance,

```
$MEMORY
10000000
```

The control will allow the program to allocate 10M words memory during the computation. This is very necessary for the integral transformation of large basis sets where a large memory will speed up the transformation, otherwise the program will switch to the disk file mode for the integral transformation, and this will significantly increase the elapsed time of the computation. One can tell from the output file of a computation if the integral transformation is in the disk mode or not. If the disk mode is used, the output file also tells how much more memory is needed for the in-core integral transformation. The default size of the scratch memory is 6.5 *Mwords*

In the GAMESS / VB2000 version, the vb2000 code checks the unused memory from that set in GAMESS. It then uses either that or the value specified by the \$MEMORY flag, which ever is largest. Thus if more memory is required, this can be done by increasing the value set in the GAMESS \$SYSTEM block, or that specified by \$MEMORY.

6.7.4 \$VBOLIBGEN

This control is used for generating a VBO library for a new basis set; therefore, it may require a number of trial and error attempts. The general strategy of building a VBO library is to choose a set of simple molecules containing different types of bonds and atoms, and perform simple VB calculations which do not rely on a VBO library, and then store the VB orbitals from the calculation. The general format of the control is:

```
$VBOLIBGEN
VBO-NAME
NA NB
NVBO
Vx Vy Vz
```

where *VBO-NAME* is a string for the VBO name, and N_A and N_B are the two atoms defining the VBO orientation. The VBO is located on atom N_A and pointing to atom N_B . N_{VBO} is the orbital index of the VBO in the output from the VB calculation. For lone pairs or π VBOs, N_B is zero, and the orientation of the VBO is defined by a vector with components (V_x , V_y and V_z). Usually case, one does not know the VBO index before the calculation. Therefore, for creating a new VBO record, one has to do two calculations. Once the first calculation is done, one needs to inspect the VBOs in the final output, and find out the VBO index, then add the \$VBOLIBGEN control with the correct VBO index. If N_B is not zero, the VBO orientation is defined by the vector from atom N_A atom N_B , and the vector line is not required. Multiple VBO records can be created under this control. One can repeat the above inputs for each VBO record without any lines in between and the input ends with a blank line.

The VBO-label consists of two parts: a prefix and an extension. The search for a VBO in a library file is by the prefix, and the same VBOs for different basis sets are stored in different VBO library files. Each VBO library file is a collection of VBOs of the same basis set. Currently, the following VBO-Label prefixes are supported:

σ Orbitals

```
$SIGMA-H-
$SIGMA-LI
$SIGMA-BE
$SIGMA-B-
$SIGMA-N-
$SIGMA-O-
$SIGMA-F-
$SIGMA-NA
$SIGMA-MG
$SIGMA-AL
$SIGMA-SI
```

```
$SIGMA-P-  
$SIGMA-S-  
$SIGMA-CL  
$SIGMA-BR  
$SIGMA-I-
```

π Orbitals

```
$PI-H-IN-HH  
$PI-LI-IN-LIH  
$PI-BE-IN-H2BE  
$PI-B-IN-BH3  
$PI-C-IN-C2H4  
$PI-N-IN-CH2NH  
$PI-O-IN-CH2O  
$PI-F-IN-HF  
$PI-NA-IN-NAH  
$PI-MG-IN-MGH2  
$PI-AL-IN-ALH3  
$PI-SI-IN-SIH3  
$PI-P-IN-PH3  
$PI-S-IN-CH2S  
$PI-CL-IN-HCL  
$PI-BR-IN-HBR  
$PI-I-IN-HI  
$PI-N-IN-PYRROLE  
$PI-O-IN-FURAN  
$PI-S-IN-THIOPHENE
```

Lone Pairs

```
$LP1-N-  
$LP1-O-  
$LP2-O-  
$LP1-F-  
$LP2-F-  
$LP1-P-  
$LP1-S-  
$LP2-S-  
$LP1-CL  
$LP2-CL  
$LP1-BR  
$LP2-BR  
$LP1-I-  
$LP2-I-
```

To initialize a VBO by using a VBO library, the program searches for the exact match first. If no exact match can be found, then it looks for the most the similar one. For each VBO library, the minimal VBO entries are those for H, C, O and S for 1-3 period elements. The molecules and the utility scripts for creating the minimal VBO library files are released in this package in the subdirectory VBOLIBMOL.

6.7.5 Controls for canonicalization of LMOs

The VBO library is used in two ways: the canonicalization of LMOs and the generation of initial VB orbitals. If the library file exists, then one can use canonicalization option for triple bonds and atoms with multiple lone pairs so that the triple bond LMOs and lone

pairs have the standard orientation. To enable canonicalization, you can use the following control flags.

```
$CANONLP
```

or

```
$CANONPI
```

The first one is used for the canonicalization of lone pairs, and the second one is for the two π LMOs of a triple bond. For a triple bond such as that in C_2H_2 , the MO localization procedure does not guarantee the resulting two π LMOs are in the standard orientations.

6.7.6 \$PRINTHS

If you need VB matrix elements, you can add control flag \$PRINTHS in the input. With this control appearing in the input, both overlap and Hamiltonian matrix elements of VB structures will be printed. The normalization constants for VB structures are also printed. However, this option only prints matrix elements with four decimal digits. You need modify the code if you wish to get higher precision numbers. It also prints the eigenvalues of the overlap matrix.

6.7.7 \$CENTROID (only for GAMESS version)

The default is to print centroids for all groups with method > 1 (i.e VB). If the run is a reactor, optimization or frequency run, centroids are printed for the first step only. The \$CENTROID option can be followed by a line that selects an option:

```
none      No centroids printed.
hf        Add Method 1 (HF) to default VB orbitals.
group n   Print centroids only for group n. This can only appear once.
orbitals n Print centroids only for n (< 100) orbitals in a list on the following line, e.g.:
orbitals 3
4 5 6
```

These can be optionally followed by "distances n" and then a line giving n atoms. Distances of the centroids from these atoms will then be printed. The default is to use the first few atoms up to a maximum of 5.

6.7.8 \$NOVBPROP (only for GAMESS version)

Introduced only in version 2.8, this directive allows GAMESS to do many calculations before doing a VB calculation. For example with this directive RUNTYP=OPTIMIZE will optimize the geometry in GAMESS before running VB2000 at the optimized geometry. This is different from the numerical optimization of geometry with the VB function.

6.7.9 Other directives

See the code, or elsewhere as directed, for details of the following minor, outdated or otherwise not yet documented directives:

Alternatives to #! command line options

\$HYBRID – Roy McWeeny’s hybrid method – no longer supported
\$FROZEN
\$PRINTALL
\$NOBIAS

vb2000.src for all versions

\$WSTART
\$PUNCHS
\$SPLITRATIO
\$LOCVBO
\$NOATMLWD
\$SPDEN
\$DIISSTART
\$SCVBSCF – similar to \$VBSCF
\$HFCOREDAMP
\$LDIAGVB
\$ERCTL
\$SFACTOR
\$ALLAOS
\$CHAPRT
\$LMOTRAN
\$LPTVIR
\$OLDLOWDIN
\$WOX
\$VBOSYM
\$LPIMO
\$DIAGHESS
\$PIORB – older form of \$AOGROUP
\$LMOLIBGEN – similar to \$VBOLIBGEN
\$xxPIDIR (xx is group)
\$xxSYMORB (xx is group)

In vb2gms.src only for GAMESS version

\$SKIPGMSHF – skips GAMESS HF if read from Molden file
\$READMOS - reads HF orbitals from \$VEC group in data
\$PRINTNOS - prints the natural orbitals of the VB function with
AIMPAC= . TRUE.

\$GPFEP,
\$GPFDP - those are used in Generalized Product Function Energy
Partitioning code. See GPFEP_DOCUMENTATION_v2.pdf in
DOC folder.

See also gamess.README in DOC folder.

6.8 Visualization of VB orbitals and molecules

Since version 1.8, we have introduced a number of routines to prepare data for a number of molecular visualization programs. In some cases there are different procedures in the different programs - the stand-alone VB2000 and the GAMESS(US) version. This is particularly true where the basis set has to be obtained and output. The options for visualization are as follows.

6.8.1 \$MOLPLT

This option is for the GAMESS(US) graphics. The data created by this option can be used to display the molecule in an X window or to create an image in a postscript file for printing. In all three programs, inclusion of the keyword:

\$MOLPLT

in the VB input data will create a file with the '.mol' extension to the jobname. In GAMESS(US), the MOLPLT data can be appended to the PUNCH file (jobname.dat) by adding 'MOLPLT=.TRUE.' to the \$CONTRL group. In GAMESS(US) all the visualization files are moved by runrms from the scratch file (\$SCR) to the user scratch file (\$USER_SCR). However, the file produced by GAMESS is more detailed than the file produced by VB2000, so the best advice is to use this directive in the stand-alone VB2000 program and use the 'MOLPLT=.TRUE.' command in \$CONTRL in the GAMESS(US)/VB2000 program. To use the molplt program, you will need the manual, which is 'molplt.man' in the gamess/graphics directory (NB, this is not a man page; it is a straight text file).

However, the program MOLPLT is very out of date and much better options are available including MacMolPlt, which can read *.mol MOLPLT files directly..

6.8.2 \$PLTORB

This option is for the GAMESS(US) graphics program PLTORB. The data created by this option is used to display individual orbitals as contour plots in a specified plane. It requires two input files - jobname.orb and jobname.vec. In the GAMESS version, these can be created very easily. The VB2000 directive **\$PLTORB** in the VB data creates two files, jobname.orb and jobname.vec, and works in an identical way in the stand-alone,

and GAMESS(US) versions. The jobname.orb file is more detailed than the file obtained by adding 'PLTORB=.TRUE.' to the \$CONTRL group, and appended to the PUNCH file. The jobname.vec file contains the coefficients of the VB orbitals in the format that will be familiar to anyone who has used the PUNCH file and the MOREAD keyword in the \$GUESS group. Unlike the pltorb output in the PUNCH file, which requires editing to add the plane for printing the orbitals and so on, the one generated by VB2000 is ready to run, but may be improved by hand editing. To do that, you will need the manual, which is 'pltorb.man' in the gamess/graphics directory (NB, this is not a man page; it is a straight text file). All distances are assumed to be Angstroms.

The data is:

```
$PLTORB
NB  NA1B1 NA2B1 NA1B2...
NVBO IVB01 IVB02 ...
Method
MethodOption
Offset
xmin xmax ymin ymax
Title1
Title2
...
```

where

NB	No. of bonds
NA1Bi, NA2Bi...	No. of atom index pairs of each bond, e.g. 2 2 1 3 1
NVBO	No. of VBOS to be plotted
IVB0i...	List of VBO numbers, e.g. 2 6 7
METHOD	SYMBOLIC or 3ATOMS - only these two are supported.
MethodOption	If SYMBOLIC, one of the strings XY, YX, XZ, ZX, YZ, ZY to choose the orientation of the plotting axes, or if 3ATOMS a list of three atom indices to define the plotting plane. The first will be set as the origin. E.g. YZ if SYMBOLIC or 1 3 1 if 3ATOMS.
Offset	Offset in Angstroms for the plotting plane relative to the defined plane, e. g. 0.0
imin imax	Limits (i = x, y) for the plotting areas, e. g. -2.0 2.0 -2.0 2.0
Title	A title up to 40 characters for each VBO plot. E.g. Water VBO 4

However, the program PLTORB is very out of date and much better options are available including MacMolPlt.

6.8.3 \$VECONLY, \$VECMOS, \$VECLMOS and \$VECINIT

The *.vec file produced by \$PLTORB can be used by MacMolPlt to plot the valence bond and Hartree-Fock group orbitals. For a GAMESS / VB2000 run, open MacMolPlt, and then open the output *.log file to display the molecule. Then import the *.vec file followed by "Surfaces" in the "Subwindow" menu, selecting "3D Orbitals". This opens a

“Surfaces” window. In the “Select Orbital Set” box you have several options. “Molecular Eigenvectors” gives you the GAMESS Hartree-Fock orbitals. For the *.vec file orbitals select “VB2000 Valence Bond Orbitals”. You can then select an orbital under “Select Orb.” and click “Update” to display it. This directive, \$VECONLY, on its own just gives the required *.vec file. In the stand-alone VB2000, the .vec file can also be used to view the orbitals in MacMolPlt, but only if you also create a Molden format (/molden) file (see 6.8.9 below) and then open that in place of the *.log file to get access to the basis set.

There are three other related commands, \$VECMOS, \$VECLMOS and \$VECINIT, which give a \$VEC group respectively for the molecular orbitals, the localized orbitals and the initial guess. All these \$VEC groups are output in a single file called *.vec. In import in MacMolPlt all \$VEC groups will be shown and you just repeat import until you have selected all those required. The “Select Orbital Set” in MacMolPlt displays “VB2000 Molecular Orbitals”, “VB2000 Localized Molecular Orbitals” and “VB2000 Initial Orbitals”. These can be helpful in getting a good initial guess in difficult cases.

6.8.4 \$CUBE

Orbitals can be displayed from a standard Gaussian cube file, a format that can be read by several visualization programs, including VMD and Molekel.

The data block:

```
$CUBE
M
(N(i), i=1,M)
Title
NP
```

will produce M output files, called jobname- $\{N(i)\}$.cube. M is the number of VB orbital CUBE files to be created and $N(i)$ is an array of the orbital numbers. E.g.

```
$CUBE
3
15 16 17
Molecule X
64
```

will generate 3 files, called jobname-015.cube, jobname-016.cube and jobname-017.cube containing the cube data for the VB orbitals, 15, 16 and 17. NP is the number of grid points in each Cartesian coordinates. NP has a maximum of 100 and defaults to 80 if given as 0.

Note that cube files generated by Gaussian itself contain many orbitals, but not all visualization codes can read them. Some can only read one orbital, so VB orbitals are created in separate cube files. Note also that the current code is not very efficient and can be slow. Do not create cube files unless you really need them. The \$CUBE directive can be used in all three versions of VB2000.

The sum of the value of the density at each CUBE point is used to estimate the

normalisation. It should be equal to unity. It gives an indication whether the number of CUBE points is adequate.

6.8.5 \$GENGRID or \$GRID

The grid files can be generated in very similar way as above. The input is almost identical except the flag \$CUBE is replaced by \$GENGRID or \$GRID. Note that \$GRID can not be used in GAMESS(US)/VB2000 as it conflicts with a GAMESS data directive. The grid files generated as .grd as the file extension. The grid files can be visualized by DS Visualizer, which is free from Accelrys. One can generate both cube and grid files by using a shortcut as follows:

```
$CUBE$GENGRID
M
(N(i), i=1,M)
Title
NP
```

\$GENGRID\$CUBE can be used as an alternative. The prohibition of \$GRID in GAMESS / VB2000 still holds here. `$CUBE$GRID` or `$GRID$CUBE` can be used likewise.

The sum of the value of the density at each GRID point is used to estimate the normalisation as with CUBE.

6.8.6 \$LINE

The purpose of this directive is to produce the value of a VBO along a line which can be divided into sections of the same length. The data is output in a form (*.csv) where it can readily be input into a spreadsheet such as Excel, OpenOffice or LibreOffice.

The data block:

```
$LINE
M
(N(i), i=1,M)
NC NP
NPT, X(I), I=1,3
```

will produce M output files, called jobname- $\{N(i)\}$.csv. M is the number of VB orbital LINE files to be created and $N(i)$ is an array of the orbital numbers. The line is drawn through a number of critical points. (NC) Just 2 critical point gives a single straight line. Each line between critical points has NP points where the VB orbitals are evaluated. NPT is 1 if the critical point is an atom and 0 otherwise. $X(I)$ are the coordinates of the critical point. The last line is repeated for each critical point.

It is also relatively easy, once you have understood the structure of CUBE files, to write code in various languages such as Fortran, C or python to construct LINE files from

CUBE files.

6.8.7 \$XYZFILE

To create an XYZ file for the molecule one can use the above directive in the input. A file with file name jobname.xyz will be created. In cases where several geometries are used, such as REACTION runs, a combined file of coordinates for all geometries can be produced with \$XYZREACTION - see \$REACTION (\$6.6.1). For the GAMESS version, the units will be AU or Bohr, whichever is selected for the run. The xyz file can be read by many visualization packages, such as VMD:

VMD (Visual Molecular Dynamics) <http://www.ks.uiuc.edu/Research/vmd/>.

VMD is a freely available and very powerful toolkit for visualization and analysis of molecular dynamics (MD) simulation data. Although it was initially developed with focus on MD, the newer versions contain many features that make it a very attractive choice for visualizing results of VB2000. One can find the tutorial of VMD at <http://www.theochem.ruhr-uni-bochum.de/~axel.kohlmeyer/cpmd-vmd/index.html>.

Here are a few tips for how to use VMD for displaying a VB orbital:

1. Load the molecule (.xyz file) into the UI.
2. Add the cube file of the molecule orbital to the molecule.
3. Use the Graphics->Representations menu from the VMD main Windows and choose Volume as the coloring method and isosurface as the drawing method. You'll just have to choose the appropriate isovalue for the orbital you want to see. You can also make orbital transparent.

One can also use another free software **Molekel** <http://www.cscs.ch/molekel/> to display a Gaussian cube file. Molekel accepts the so-called .mkl file for molecule structure. You may need to use the directive:

6.8.8 \$MOLEKEL

This generates a file called jobname.mkl (in the GAMESS case in the scratch area). This is the molecule input file for **Molekel**. The format is discussed at: <http://mtz01-a.stanford.edu/programs/documentation/molekel/interface.html>. Molekel files can also be read by MacMolPlt.

Tested with version 5.4 - Ugo Varetto, MOLEKEL 5.4; Swiss National Supercomputing Centre: Manno (Switzerland).

6.8.9 \$MOLDEN

The directive in the VB data block generates a file called jobname.molden (in the GAMESS case in the scratch area). This is the molecule input file for **Molden**. The full

description of the Molden format is at:
https://www3.cmbi.umcn.nl/molden/molden_format.html.

The stand-alone VB2000 can use \$READMOLDEN to get the geometry data from a Molden file.

Molden files can also be used in other programs, particularly MacMolPlt.

6.8.10 \$DENSCUBE

This control is used to generate CUBE files for densities, unlike the CUBE directive which gives CUBE files for orbitals, The data is:

```
$DENSCUBE
DENS SPIN GRPS GMSD HPRT
NGRPS
G1 G2 ...
Title
NGRIDPTS
```

The second line contains up to five 5-letter labels with the following meaning:

DENS	- A CUBE file for the total density will be generated.
SPIN	- A CUBE file for the spin density density will be generated.
GRPS (or VBOS)	- CUBE files will be generated for a number of groups. These can be VB groups or the HF core group.
GMSD	- A CUBE file for the HF density is created. This is only for the GAMESS version. This can be used with the total VB density from DENS to obtain a CUBE file for the difference in density. This can be obtained using the <code>get_cube_diff.f</code> program in the TOOLS directory.
HPRT	- This does not give a CUBE file and it is also only available in the GAMESS version. It gives a higher level of output, printing the various density matrices that are used for the CUBE files. It is recommended that you also add <code>NPRINT=9</code> in the GAMESS \$CONTRL group if GMSD is selected as this gives you the HF density in the GAMESS output.

The 3rd and 4th lines are present only if GRPS (or VBOS) is present and indicates the selected groups.

An example is:

```
$DENSCUBE
DENS SPIN VBOS
2
1 2
cyclopentadienyl anion cc-pVDZ
80
```

This will generate CUBE files for the total density, the spin density if the molecule is not a singlet, and for two groups, 1 (the HF core) and 2 (the VB group). The molecule is cyclopentadienyl anion using the cc-pVDZ basis set. The CUBE files will have 80 x 80 x 80 points. The maximum number of points is 100 x 100 x 100.

The sum of the value of the density at each CUBE point is used to estimate the normalisation. It should be equal to the number of electrons involved in the density. It gives an indication whether the number of CUBE points is adequate.

Some of the density matrices in the GAMESS version will be printed if the directive \$PRINTDENS, without \$DENS_CUBE, is present in the VB2000 data.

6.8.11 Using the visualization directives in TEST runs.

The TEST directive on the command line terminates the program after generation of the initial orbitals. The program now calls the visualisation routines at this point prior to termination of the run, but uses the localised MOs rather than the VB orbitals when the visualization routine requires orbitals. All the above visualization directives can be used. Note that the numbering of the LMOs will be different from the numbering of the VBOs that you will finally want to use to obtain CUBE files.

6.8.12 3D pictures of two VB orbitals of a O-H bond

Here are two orbitals forming an O-H bond in the water molecule. The pictures are created with VMD [29].

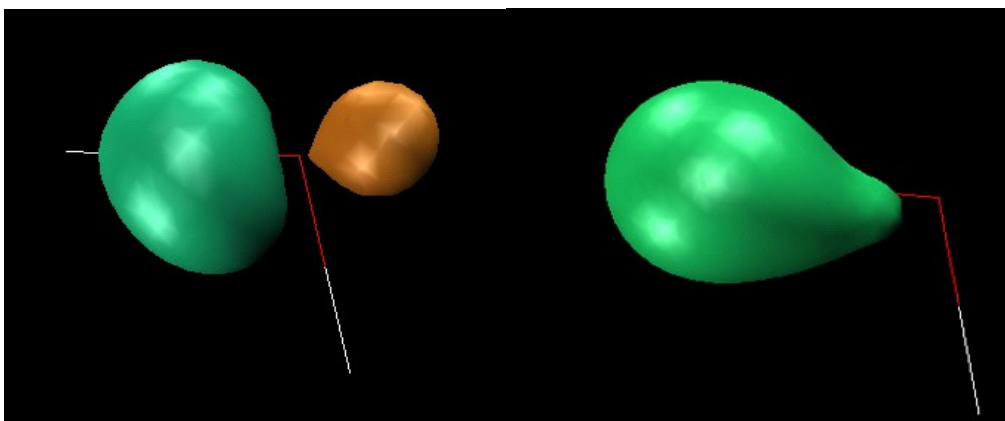


Figure 6.2 H₂O VB orbital localized on oxygen atom (left) and on hydrogen atom (right).

6.9 Getting quick help on commands.

The script `vbhelp` in the TOOLS directory gives access to a text copy of this section of the manual. Edit the top few lines to point to `VB-help-file` in the DOC subdirectory and move `vbhelp` to be on your path. Typing `vbhelp -h` will then give you more information about how to use it. It can give a direct access to the manual section about a

specific command, e.g. `vbhelp GRPDIM` will tell you about that command. Note that this is a simple “save as text” from a word processor file that has only been lightly edited. It is useful as a reminder. The equations have disappeared in the process.

7 Construction of Initial Wave Functions

The initial guess of the system wave function of a molecule in the framework of GF theory is essential for the GFT calculation. It involves the partitioning of electron groups and the construction of initial orbitals for each group function. A good initial guess of VB orbitals is essential for VB group function optimization. Construction of high quality initial VB orbitals according to a user's intention is especially challenging. To solve the problem, two technologies have been introduced in VB2000 version 1.8. One involves the standard notation for LMOs and VBOs and the other depends on the standard VBO libraries. This chapter describes the standard notations for different types of LMOs and VBOs, and how to use those notations to specify the electron group partitioning and the construction of initial VB orbitals.

7.1 Notations for localized molecular orbitals

Owing to the locality and transferability of localized molecular orbitals and valence bond orbitals, the orbitals preserve certain characteristics in different molecular environments. Therefore it should be possible to create a set of standard notations for different types of orbitals. From a Hartree-Fock wave function, the molecular orbital localization method generates the following types of LMOs: the inner core orbitals, lone pairs and the bonding orbitals. The following notations are used to label different LMOs.

7.1.1 Lone pair

A lone pair orbital can be denoted by

$A:(m)$

where A is the atom index in the input, m is the index of the lone pair, which roughly follows the order of orbital energies. For instance, the lowest lone pair of the valence shell is labeled $A:(1)$, and the higher energy lone pairs (if any) are labeled $A:(2)$ and $A:(3)$. Follow a similar convention, all inner core orbitals on atom A are thus labeled as

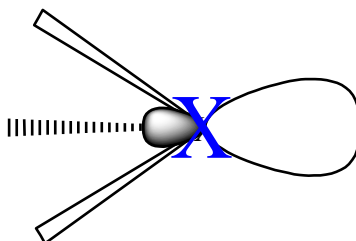
$A:(0)$

Let's use H₂O as an example. Assume the atoms are in the order as O, H and H in the input. The first lone pair (LP) on oxygen (which should be the sigma lone-pair and is in the plane of the molecule) can be denoted by

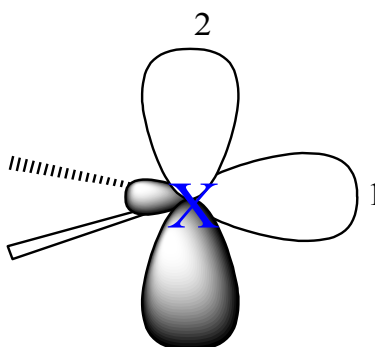
$1:(1)$

The second LP, which is actually the lone-pair perpendicular to the molecule plane, can be denoted as $1:(2)$. For multiple LPs on an atom, rules are used to label different LMOs.

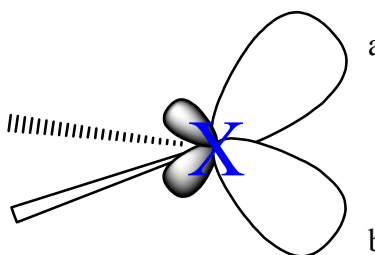
Case 1. XR_3 Type, for example, NH_3 . Only one LP exists. The LP is shown as follows:



Case 2. XR_2 Type. Example, H_2O . There are two LPs. The first one is defined as lying in the molecular plane, but pointing symmetrically away from the two bonds; the second LP direction is perpendicular to the R-X-R plane, as shown below



As shown in the above scheme, LP1 has σ symmetry, and LP2 has π symmetry. Often one can use another way of describing the two LPs, taking them as 'equivalent orbitals', shown as follows:

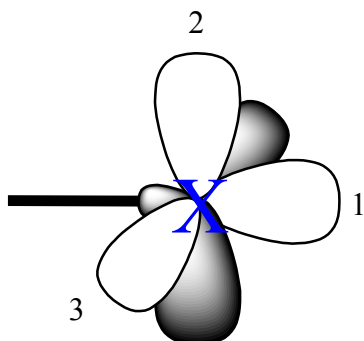


In VB2000, only the primitive σ - π LPs are defined automatically. However, there is a simple way to construct the two equivalent LPs, using the following linear combination of the primitive LPs:

$$\begin{aligned}\text{LP(a)} &= \text{LP(1)} + \text{LP(2)} \\ \text{LP(b)} &= \text{LP(1)} - \text{LP(2)}\end{aligned}$$

The normalization coefficients are ignored. It is assumed that the LP(1) and LP(2) are normalized. The current version of VB2000 provides a simple way to construct initial orbitals for the above combinations.

Case 3. XR Type. For example, Ph-F.



The first LP is defined as the one opposite to the X-R bond. The remaining two LP directions are perpendicular to the X-R bond direction. If R is a conjugated atom and has a well defined conjugate plane, then the second LP is parallel to the conjugate plane, and the third one is perpendicular to the conjugate plane. For instance in the case of Ph-F, the first LP is the one opposite to the Ph-F bond, the second one is parallel to the phenyl ring, and the third one is perpendicular to the phenyl ring.

7.1.2 Two-center bond LMO

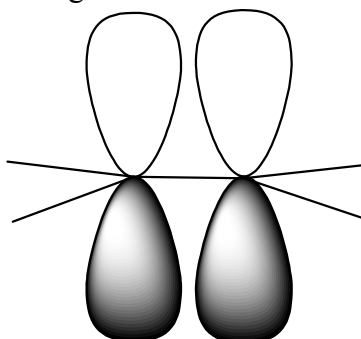
Each local bond(between two atoms) can be denoted by

$$A-B(k)$$

where A and B are two bonded atom, and k is the index of the bond. The first bond $A-B(1)$, or simplified as $A-B$, is the sigma bond as shown in the following scheme:

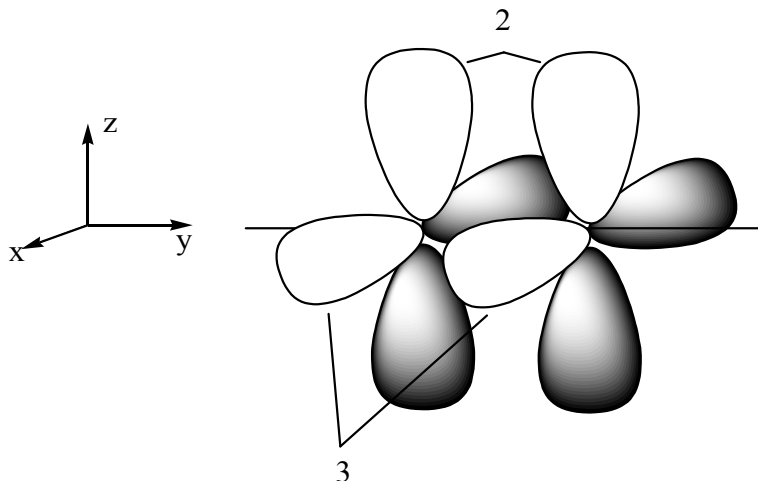


The second bond (if any) is the first π bond. For a double bond, there is no ambiguity for the π bond as shown in the following scheme:



For a triple bond, the third bonding LMO is the second π MO. There are two cases: if the

triple bond atoms are co-linear with a conjugate neighbor, then the second LMO is a π orbital and is defined with direction perpendicular to the neighbor conjugate plane. If there is no such plane, then the π orbital orientation is not well defined. If we put the co-linear atoms on the Y -axis (as shown below), and take π orbital on the Z -axis as the second LMO, then the π orbital on the X -axis is the third LMO. If the triple bond atoms are not co-linear with a third neighbor atom, the π orbitals perpendicular to the plane (defined by the three non-co-linear atoms) is the second LMO, and the other one is the third:



For instance, if we have the molecule C_2H_2 on the Y -axis as shown above, then the first bond is the σ bond (not shown in the above scheme), and the second bond (i.e. the first π bond) is in P_z direction, and the third bond (i.e. the second π bond) is the P_x direction. If we align the two triple bonded atoms on the Z -axis, the first and the second π orbitals are in the X - and Y -axis, respectively. The third case is that the two atoms are on the X -axis, and the first and the second π orbitals are on the Y - and Z -axis, respectively. For an arbitrary orientation of a triple bond, we need define three new axes in the molecule frameworks, and label the three axes according the maximum overlap (absolute values) of the orientation of each axis with the direction of the original axes. Once the axes in the molecule framework are so labeled, define the first and second π orbital orientations in the same way as above. However, if the triple bond is connected to a conjugate system, for instance, Ph-CN, the first π orbital direction is chosen to be the that perpendicular to the conjugate plane. In Ph-CN, the first π is perpendicular to the phenyl ring.

There are special cases that the triple bond atoms are not co-linear, for instance, R-Si#Si-R triple bond. The four atoms R-Si-Si-R are not co-linear. In such a case, the P_z is defined as the direction perpendicular to the R-Si-Si plane.

7.1.3 Multi-center bond LMO

A multi-center (MC) bond LMO can be denoted by

$$MC(k)$$

where k is the index of the MC bond. Thus the first MC bond LMO (i.e the LMO with

the lowest energy) is marked as $MC(1)$, the second as $MC(2)$, etc.

7.2 Notations for VB Orbitals

7.2.1 σ VB orbital

A σ VB orbital can be denoted as

$A \rightarrow B$

where A is the atom that the VBO belongs to, and B is the atom that this VBO bonds to. In other words, this is a VBO on atom A pointing to atom B , and can be schematically represented as following:



Since bonding VB orbitals always appear in pairs, we can use $A-B$ to represent two bonding VBOs forming a single bond between atom A and B :



7.2.2 π VB orbital

Since a π VBO is not necessary pointing to a particular atom, the following notation is introduced:

$n^{\wedge}(k)$

where n is the VBO atom, the hat ' $\wedge$ ' indicates a π orbital, and k is the index of the π VBO. For instance, in C_2H_2 , there are two π VBOs on each carbon. Special rules are used to define the first- and second- π bond directions. See previous sections for more details.

In most simple calculations, the initial VB orbitals are generated automatically by splitting the corresponding bonding LMOs. However, there are cases where you need to explicitly specify a set of VB orbitals for your calculation. The following example explains how the above notations and rules can be used to specify the VB orbitals for a CASVB(6,5) calculation, which involved 5 VB orbitals for 6 electrons.

Example 7.1. Explicit specification of VB orbitals in H_2O (file TESTINP/extra/extra06_h2o65.inp).

```
#! CASVB(6,5)/D95 PRINTALL
TEST H2O
0 1
```

```

8 .000000 .000000 .000000
1 .801842 .000000 .555582
1 -.801842 .000000 .555582

$02VBORB
1->2
2->1
1->3
3->1
1:(2)

```

In this example, 5 VB orbitals are explicitly specified in input group \$02VBORB. Each line specifies one VBO. The input is terminated by a blank line. The first 4 orbitals are used for forming two O-H σ -bonds, while the last one is the second LP (perpendicular to the HOH plane).

Not only the above primitive (σ and π) orbitals can be specified, any orbitals which can be expressed as a linear combination of those primitives can also be specified in a quite intuitive format. See section 7.3.3 for more details.

7.2.3 Multiple VBOs

Since each bond involves two VBOs, it is intuitive to use a bond notation to specify two VBOs. Thus

n - m (single bond)

is equal to

n ->m

m ->n

and

n=m (double bond)

is equal to

n ->m

m ->n

n^

m^

and

n#m (triple bond)

is equal to

n ->m

m ->n

n^(1)

m^(1)

n^(2)

m^(2)

7.3 Construction of initial VB orbitals

7.3.1 Initial VBOs by splitting LMOs

This is the default method for generating initial VB orbitals if no VBO directives are provided in the input. The technique is based on a very simple and intuitive idea: chemical structures in the sense of qualitative valence bond theory are associated with alternative allocations of electron-pair bonds, which are easily translated into the language of modern VB theory. Each 'structure' becomes a spin-coupling scheme, involving pairs of electrons occupying rather localized, strongly overlapping, orbitals. The overlapping orbitals of any bond are expected to bear some similarity to the AOs, or hybrids, used in constructing a localized MO. A simple and natural way to generate such an orbital pair is to generate a set of LMOs (e.g. by transformation of a simple MO-type wave function) and then to cut each LMO into two pieces (also see section 6.5.2), as described very briefly in reference[1]. In this way, one can construct initial VB orbitals for all groups; and by orthogonalization, using Löwdin's method, it is easy to set up active subspaces satisfying the strong orthogonality constraints of the GF method. This first approximation is used for the sole purpose of starting the calculation and the group functions are allowed to be optimized in the full basis function space within the strong orthogonality constraints. Our experience shows that this method works very well for simple covalent bonds and the generated initial VB wave function has quite good convergence behavior. Moreover, the method is very general, it can be used for any kind of basis set and any simple covalent bonds which can be represented by a bonding LMO.

7.3.2 Initial VBOs from VBO libraries

For most simple calculations, the above default method works very well. However, there are cases in which the above method does not work well, especially for special bonding situations. Examples are molecules with conjugated systems or multi-center bonds; or special bond types (e.g. 'banana bonds'). In VB2000 version 1.8, the standard notation is used to specify the initial VBOs. Example 7.1 in section 7.2.2 explains how the standard VBO notations are used in the input file. The VBO directives in the input file are used by the program to load the standard VBOs from a library file provided in the package. In VB2000 version 1.8, 14 VBO library files are included, each for one or more basis sets. A user can generate a VBO library file for a user-specified basis set by using a utility provided in the package. See Chapter 6 for more details.

7.3.3 Linear combination of standard VBOs

From version 1.8, a user can use a linear combination of library VBOs to form a new VBO with specific shape and orientation. The formation of this linear combination is very simple and intuitive as shown in the following example.

Example 7.2. Specification of an arbitrary combination of the primitive σ and π orbitals

(file TESTINP/example06_c2h2bn.inp):

```
#! VB(6)/D95 PRINTALL
TEST C2H2 TRIPLE BOND (3 BANANA BONDS)

0 1
6 .528275 0.305 0.0
6 -.528275 -0.305 0.0
1 1.463583 0.845 0.0
1 -1.463583 -0.845 0.0

$02VBORB
(1->2) + 1^
(2->1) + 2^
(1->2) - 0.5 (1^ ) + 0.866025 (1^(2))
(2->1) - 0.5 (2^ ) + 0.866025 (2^(2))
(1->2) - 0.5 (1^ ) - 0.866025 (1^(2))
(2->1) - 0.5 (2^ ) - 0.866025 (2^(2))
```

In this example, we want to perform a three banana bond calculation for the CC triple bond. Instead of using one σ and two π VBOs on each carbon, one uses three symmetrically equivalent SP^3 hybrid orbitals. Since the program uses the primitive σ and π VBOs, we need to recombine σ and π VBOs to obtain 3 symmetry-equal VBOs. Each of the first VBOs has two components, and the last 4 VBOs have 3 components for each. The coefficients are chosen in such a way that all the three VBOs on each carbon are symmetrically equivalent.

7.3.4 A method for construction of the CASSCF space

Although both VBO library and the LMO-splitting methods are used to generate initial VB orbitals, the methods can be used for more general purposes such as the construction for the active space for a CASSCF wave function. It is often a big challenge to construct a proper active space for a CASSCF calculation. The tools in VB2000 will make the CAS-type calculation more flexible and easy to control.

8 Test Cases and Output Descriptions

In this chapter, the input and output files of the examples 01-23 in the TESTINP directory are extensively discussed. For the sake of clarity, only the most relevant parts of the output files are shown; but the full output from each test calculation can be found in the TESTOUT directory.

8.1 Simple VB calculation on H₂O

Input (example01_h2ovb.inp):

```
#!/ VB(4)/cc-pVDZ PRINTALL
TEST H2O
0 1
8 .000000 .000000 .000000
1 .801842 .000000 .555582
1 -.801842 .000000 .555582
```

For illustration purpose, the full text of the output file of Example 1 is given below:

```
*****
*
*                               V B 2 0 0 0
*
* An ab initio Valence-Bond Program Based on the Generalized
*   Product Function Method and the Algebrant Algorithm
*
*                               Version 3.0 September 2021
*                               Stand-alone Version
*                               Compiler: GCC version 9.3.0
*
*   Memory = 2500000 for MAP file up to 14 electrons
*
*   Jiabo Li(1), Brian Duke(2), David W.O. de Sousa(3)
*   Rodrigo S. Bitzer(3), and Roy McWeeny(4)
*
*   (1) ChemXAI Inc, 53 Barry Lane, Syosset, NY 11791, USA
*
*   (2) Monash University Institute of Pharmaceutical Sciences,
*       Monash University, 381 Royal Pde, Parkville,
*       Victoria, 3052, Australia
*
*   (3) Instituto de Quimica, Universidade Federal do
*       Rio de Janeiro, Brazil
*
*   (4) University of Pisa, Italy
*       (May 19th, 1924 - April 29th, 2021)
*
*   Reference for the software:
```

```

*   Jiabo Li, Brian Duke, Roy McWeeny, David W.O. de Sousa, *
*   and Rodrigo S. Bitzer, VB2000 Version 3.0, *
*   ChemXAI Inc., Syosset, NY, 2021. *
*
* References for the theory: *
* General: *
*   Jiabo Li, and Roy McWeeny, "VB2000: Pushing Valence *
*   Bond Theory to New Limits", Int. J. Quantum Chem., *
*   89(2002)208-216. *
*
* Spin densities:- *
*   Jiabo Li, Brian J. Duke, Thomas M. Klapoetke, and Roy *
*   McWeeny, Spin-Density of Spin-Free Valence Bond Wave *
*   Functions and Its Implementation in VB2000. J. Theor. *
*   Comp. Chem. 7(2008)853-867. *
*
* Spin coupled - n electrons in m orbitals:- *
*   Peter B. Karadakov, David L. Cooper, Brian J. Duke, *
*   and Jiabo Li, Spin-Coupled Theory for N Electrons in *
*   M Orbitals Active Spaces, J. Phys. Chem. A, 116, *
*   7238 - 7244, 2012 *
*
*   Copyright (C) 2018 GNU General Public License *
*****

JOB NAME IS example01_h2ovb

VB2000 DIRECTORY IS: .
MAXIMUM DYNAMIC MEMORY IS: 6500000

Input of molecule
=====
#! VB(4)/CC-PVDZ PRINTALL

Title: TEST H2O

Charge = 0 Multiplicity = 1

BASIS SET FILE IS: ./BASSET/cc-pVDZ

Cartesian coordinates of atoms in Angstrom
-----
      X           Y           Z
O      8.0      0.000000      0.000000      0.000000
H      1.0      0.801842      0.000000      0.555582
H      1.0     -0.801842      0.000000      0.555582
-----

Total number of two-electron integrals = 25620
Time for integral evaluation 0.030

SKIPPED INTEGRALS = 23683
Number of iter = 20, DMAX = 0.000008547425
Final Hartree-Fock Energy = -76.024351732446

Molecular orbitals by HF method
=====
MO# 1 2 3 4 5 6

```

E(a.u.)				-20.551	-1.323	-0.700	-0.551	-0.490	0.181
1	1	0	s	1.00098	-0.00802	0.00000	0.00238	-0.00000	-0.05679
2	1	0	s	0.00206	0.44344	0.00000	-0.14454	-0.00000	0.08478
3	1	0	s	-0.00114	0.38746	0.00000	-0.34076	-0.00000	1.06437
4	1	0	x	-0.00000	-0.00000	0.48446	-0.00000	0.00000	0.00000
5	1	0	y	-0.00000	-0.00000	-0.00000	-0.00000	0.63106	0.00000
6	1	0	z	0.00154	0.07079	0.00000	0.55171	0.00000	0.18904
7	1	0	x	0.00000	0.00000	0.21715	0.00000	0.00000	0.00000
8	1	0	y	0.00000	-0.00000	-0.00000	-0.00000	0.49762	0.00000
9	1	0	z	-0.00044	-0.00877	0.00000	0.38237	0.00000	0.33395
10	1	0	xx	-0.00108	0.00578	0.00000	0.00300	-0.00000	-0.02660
11	1	0	yy	-0.00119	-0.00026	-0.00000	-0.00596	0.00000	-0.04026
12	1	0	zz	-0.00116	0.00354	0.00000	0.02384	0.00000	-0.02445
13	1	0	xy	-0.00000	-0.00000	-0.00000	-0.00000	0.00000	-0.00000
14	1	0	xz	0.00000	0.00000	0.02503	-0.00000	0.00000	0.00000
15	1	0	yz	0.00000	0.00000	-0.00000	-0.00000	0.01730	-0.00000
16	2	H	s	-0.00009	0.18307	0.32775	0.19665	0.00000	-0.07225
17	2	H	s	0.00022	0.01186	0.08897	0.04666	0.00000	-0.83420
18	2	H	x	0.00038	-0.03671	-0.02612	-0.02896	-0.00000	0.01232
19	2	H	y	-0.00000	-0.00000	-0.00000	-0.00000	0.02997	0.00000
20	2	H	z	0.00029	-0.01731	-0.03100	0.00989	-0.00000	0.01179
21	3	H	s	-0.00009	0.18307	-0.32775	0.19665	-0.00000	-0.07225
22	3	H	s	0.00022	0.01186	-0.08897	0.04666	0.00000	-0.83420
23	3	H	x	-0.00038	0.03671	-0.02612	0.02896	-0.00000	-0.01232
24	3	H	y	-0.00000	0.00000	-0.00000	-0.00000	0.02997	-0.00000
25	3	H	z	0.00029	-0.01731	0.03100	0.00989	0.00000	0.01179
MO#				7	8	9	10		
E(a.u.)				0.253	0.798	0.810	1.159		
1	1	0	s	-0.00000	-0.00000	0.05691	-0.01233		
2	1	0	s	-0.00000	-0.00000	0.23839	-0.09812		
3	1	0	s	0.00000	0.00000	-0.23123	-0.57172		
4	1	0	x	-0.28554	0.29362	0.00000	-0.00000		
5	1	0	y	0.00000	-0.00000	-0.00000	0.00000		
6	1	0	z	0.00000	-0.00000	0.27959	0.78898		
7	1	0	x	-0.67781	0.41880	0.00000	0.00000		
8	1	0	y	0.00000	0.00000	0.00000	-0.00000		
9	1	0	z	0.00000	0.00000	0.06517	-1.21139		
10	1	0	xx	0.00000	0.00000	-0.09947	-0.04330		
11	1	0	yy	-0.00000	-0.00000	0.08383	-0.03242		
12	1	0	zz	0.00000	0.00000	0.01043	-0.05273		
13	1	0	xy	0.00000	-0.00000	-0.00000	-0.00000		
14	1	0	xz	-0.02022	-0.11036	-0.00000	-0.00000		
15	1	0	yz	0.00000	-0.00000	0.00000	-0.00000		
16	2	H	s	0.02568	-0.93900	-0.83714	0.44132		
17	2	H	s	1.38415	0.69185	0.55338	0.11867		
18	2	H	x	-0.02058	-0.07197	-0.26542	-0.12181		
19	2	H	y	-0.00000	-0.00000	0.00000	-0.00000		
20	2	H	z	-0.01550	-0.16306	-0.02719	-0.24133		
21	3	H	s	-0.02568	0.93900	-0.83714	0.44132		
22	3	H	s	-1.38415	-0.69185	0.55338	0.11867		
23	3	H	x	-0.02058	-0.07197	0.26542	0.12181		
24	3	H	y	0.00000	-0.00000	-0.00000	0.00000		
25	3	H	z	0.01550	0.16306	-0.02719	-0.24133		

PARAMETER LIMIT VALUES

```

Max. no. of atoms          MAXCEN =   200
Max. no. of basis functions MAXBFN =  1350
Max. no. of shells        MAXSHL =   500
Max. no. of total primitives MAXEXP =  4400
Max. no. of primitives per orbital MAXPRM =   15
Max. no. of atoms * no. of orbitals MAXNAB = 135000
Increase max basis set size by NEXTRA =   20
Max. size for Rumer data    MAXST1 = 120000
Max. no. of orbitals in group MAXNOB =   200
Max. no. of CUBE orbitals   MAXCUB =   100
Max. no. of groups         MAXGRP =    49
BRILLMASK array sizes      MAXBLL =  12500
  
```

VBO LIB IS: ./VBOLIB/VBOLIBCCPVDZ

Localized molecular orbitals and the bond labels

LMO#		1	2	3	4	5
E(a.u.)		-19.408	-1.909	-0.490	-0.905	-0.905
Bond Label		1:(0)	1:(1)	1:(2)	2-1(1)	3-1(1)
1	1 0 s	0.97313	-0.22947	-0.00001	0.03465	-0.03465
2	1 0 s	-0.10175	-0.36557	0.00000	0.19176	-0.19176
3	1 0 s	-0.12585	-0.48820	0.00001	0.07767	-0.07768
4	1 0 x	-0.00000	-0.00000	0.00000	0.34257	0.34257
5	1 0 y	0.00001	0.00001	0.63106	-0.00000	0.00000
6	1 0 z	0.07631	0.39555	-0.00000	0.27121	-0.27121
7	1 0 x	-0.00000	-0.00000	0.00000	0.15355	0.15355
8	1 0 y	0.00001	0.00000	0.49762	-0.00000	0.00000
9	1 0 z	0.06196	0.30721	-0.00000	0.15503	-0.15503
10	1 0 xx	-0.00162	-0.00064	0.00000	0.00450	-0.00450
11	1 0 yy	-0.00205	-0.00428	0.00000	-0.00268	0.00268
12	1 0 zz	0.00202	0.01711	-0.00000	0.01195	-0.01195
13	1 0 xy	-0.00000	-0.00000	0.00000	-0.00000	0.00000
14	1 0 xz	-0.00000	-0.00000	0.00000	0.01770	0.01770
15	1 0 yz	0.00000	0.00000	0.01730	-0.00000	0.00000
16	2 H s	-0.00218	0.05184	-0.00000	0.41816	0.04535
17	2 H s	0.00547	0.03011	-0.00000	0.08919	0.03663
18	2 H x	0.00245	-0.00220	-0.00000	-0.05145	0.01451
19	2 H y	0.00000	0.00000	0.02997	-0.00000	0.00000
20	2 H z	0.00501	0.01754	-0.00000	-0.02761	-0.01623
21	3 H s	-0.00218	0.05184	-0.00000	-0.04535	-0.41816
22	3 H s	0.00547	0.03011	-0.00000	-0.03663	-0.08919
23	3 H x	-0.00245	0.00220	0.00000	0.01451	-0.05145
24	3 H y	0.00000	0.00000	0.02997	-0.00000	0.00000
25	3 H z	0.00501	0.01754	-0.00000	0.01623	0.02761

Mulliken population of localized molecular orbitals

LMO#		1	2	3	4	5
E(a.u.)		-19.408	-1.909	-0.490	-0.905	-0.905
Bond Label		1:(0)	1:(1)	1:(2)	2-1(1)	3-1(1)

1	O	2.00065	2.02871	1.95912	1.17571	1.17571
2	H	-0.00032	-0.01435	0.02044	0.83151	-0.00722
3	H	-0.00032	-0.01435	0.02044	-0.00722	0.83151
PARTITIONING OF LMOs INTO GROUPS (\$LMOGRPMODIFY)						
=====						
LMO#	1	2	3	4	5	
Group#	1	1	1	2	2	
Split	0	0	0	1	1	
Initial guess of orbitals						
=====						
ORBITAL#	1		2		3	
Bond Label	1:(0)		1:(1)		1:(2)	
=====						
1	1	O	s	0.97313	-0.22947	-0.00001
2	1	O	s	-0.10175	-0.36557	0.00000
3	1	O	s	-0.12585	-0.48820	0.00001
4	1	O	x	-0.00000	-0.00000	0.00000
5	1	O	y	0.00001	0.00001	0.63106
6	1	O	z	0.07631	0.39555	-0.00000
7	1	O	x	-0.00000	-0.00000	0.00000
8	1	O	y	0.00001	0.00000	0.49762
9	1	O	z	0.06196	0.30721	-0.00000
10	1	O	xx	-0.00162	-0.00064	0.00000
11	1	O	yy	-0.00205	-0.00428	0.00000
12	1	O	zz	0.00202	0.01711	-0.00000
13	1	O	xy	-0.00000	-0.00000	0.00000
14	1	O	xz	-0.00000	-0.00000	0.00000
15	1	O	yz	0.00000	0.00000	0.01730
16	2	H	s	-0.00218	0.05184	-0.00000
17	2	H	s	0.00547	0.03011	-0.00000
18	2	H	x	0.00245	-0.00220	-0.00000
19	2	H	y	0.00000	0.00000	0.02997
20	2	H	z	0.00501	0.01754	-0.00000
21	3	H	s	-0.00218	0.05184	-0.00000
22	3	H	s	0.00547	0.03011	-0.00000
23	3	H	x	-0.00245	0.00220	0.00000
24	3	H	y	0.00000	0.00000	0.02997
25	3	H	z	0.00501	0.01754	-0.00000
=====						
ORBITAL#	7					
Bond Label	3-1(1)					
=====						
1	1	O	s	0.03157		
2	1	O	s	0.05422		
3	1	O	s	0.07238		
4	1	O	x	0.00000		
5	1	O	y	0.00000		
6	1	O	z	-0.05858		

```

7 1 0 x 0.00000
8 1 0 y 0.00000
9 1 0 z -0.04550
10 1 0 xx 0.00010
11 1 0 yy 0.00064
12 1 0 zz -0.00253
13 1 0 xy 0.00000
14 1 0 xz 0.00000
15 1 0 yz 0.00000
16 2 H s -0.00765
17 2 H s -0.00446
18 2 H x 0.00032
19 2 H y 0.00000
20 2 H z -0.00260
21 3 H s -0.87535
22 3 H s -0.18953
23 3 H x -0.10708
24 3 H y 0.00000
25 3 H z 0.05470

```

GENERAL CONTROLS (\$GENCTL)

```

=====
Number of electron groups      = 2
Maximum macro-iterations      = 12
Energy threshold               = 10**(- 6)
Restart calculation(0/1/2)    = 0
Group#                         1 2
Num. of electrons              6 4
Num. of spins                  0 0
Num. of orbitals               3 4
Method#                        1 2

```

SYMBOLIC VB STRUCTURE(S) OF GROUP 2

```

STR. NO.      RUMER PATTERN
1             1 2 3 4

```

OVERLAP MATRIX OF VB ORBITALS FOR GROUP 2

```

      1      2      3      4
=====
1 1.0000
2 0.6271 1.0000
3 -0.0278 -0.0990 1.0000
4 -0.0990 -0.1918 0.6271 1.0000

```

EIGENVALUES OF OVERLAP MATRIX OF VB ORBITALS

```

=====
0.3557D+00 0.3790D+00 0.1425D+01 0.1841D+01

```

MAP FILE IS: ./MAP/MAP0400t

VB-ORBITAL OPTIMIZATION OF GROUP 2 ...

```

GROUP = 2 ITER = 1 CMAX = 0.150000000000000
GROUP = 2 ITER = 2 CMAX = 0.10059583951236
GROUP = 2 ITER = 3 CMAX = 0.05497309829693
GROUP = 2 ITER = 4 CMAX = 0.02175149050372
GROUP = 2 ITER = 5 CMAX = 0.00498670883326
GROUP = 2 ITER = 6 CMAX = 0.00215779929998

```

```

GROUP = 2 ITER = 7 CMAX = 0.00091351647232

ENERGY AND DIFF OF MACROITER 1 = -76.0568859177 -76.0568859177

VB-ORBITAL OPTIMIZATION OF GROUP 2 ...
GROUP = 2 ITER = 1 CMAX = 0.01557009163916
GROUP = 2 ITER = 2 CMAX = 0.01304677506871
GROUP = 2 ITER = 3 CMAX = 0.01163598968117
GROUP = 2 ITER = 4 CMAX = 0.00943787687671
GROUP = 2 ITER = 5 CMAX = 0.00400439223838
GROUP = 2 ITER = 6 CMAX = 0.00158904213938
GROUP = 2 ITER = 7 CMAX = 0.00056391888929

ENERGY AND DIFF OF MACROITER 2 = -76.0663814373 -0.0094955196

VB-ORBITAL OPTIMIZATION OF GROUP 2 ...
GROUP = 2 ITER = 1 CMAX = 0.00446022182445
GROUP = 2 ITER = 2 CMAX = 0.01638150506272
GROUP = 2 ITER = 3 CMAX = 0.00227897839028
GROUP = 2 ITER = 4 CMAX = 0.00003025692260

ENERGY AND DIFF OF MACROITER 3 = -76.0668509683 -0.0004695310

VB-ORBITAL OPTIMIZATION OF GROUP 2 ...
GROUP = 2 ITER = 1 CMAX = 0.00276070879299
GROUP = 2 ITER = 2 CMAX = 0.00682182605769
GROUP = 2 ITER = 3 CMAX = 0.00033159724526

ENERGY AND DIFF OF MACROITER 4 = -76.0670404461 -0.0001894778

VB-ORBITAL OPTIMIZATION OF GROUP 2 ...
GROUP = 2 ITER = 1 CMAX = 0.00170100415638
GROUP = 2 ITER = 2 CMAX = 0.00416603282070
GROUP = 2 ITER = 3 CMAX = 0.00013966566763

ENERGY AND DIFF OF MACROITER 5 = -76.0671224366 -0.0000819905

VB-ORBITAL OPTIMIZATION OF GROUP 2 ...
GROUP = 2 ITER = 1 CMAX = 0.00090558855210

ENERGY AND DIFF OF MACROITER 6 = -76.0671447586 -0.0000223221

VB-ORBITAL OPTIMIZATION OF GROUP 2 ...
GROUP = 2 ITER = 1 CMAX = 0.00067716159292

ENERGY AND DIFF OF MACROITER 7 = -76.0671540779 -0.0000093193

VB-ORBITAL OPTIMIZATION OF GROUP 2 ...
GROUP = 2 ITER = 1 CMAX = 0.00066419990451

ENERGY AND DIFF OF MACROITER 8 = -76.0671594780 -0.0000054001

VB-ORBITAL OPTIMIZATION OF GROUP 2 ...
GROUP = 2 ITER = 1 CMAX = 0.00055740375073

ENERGY AND DIFF OF MACROITER 9 = -76.0671622595 -0.0000027815

VB-ORBITAL OPTIMIZATION OF GROUP 2 ...
GROUP = 2 ITER = 1 CMAX = 0.00048457273861

```

ENERGY AND DIFF OF MACROITER 10 = -76.0671637235 -0.0000014640

VB-ORBITAL OPTIMIZATION OF GROUP 2 ...
GROUP = 2 ITER = 1 CMAX = 0.00039217572045

ENERGY AND DIFF OF MACROITER 11 = -76.0671644657 -0.0000007422

VB-ORBITAL OPTIMIZATION OF GROUP 2 ...
GROUP = 2 ITER = 1 CMAX = 0.00032777881084

OVERLAP MATRIX OF VB ORBITALS FOR GROUP 2

```

      1      2      3      4
=====
1 1.0000
2 0.8081 1.0000
3 -0.1867 -0.0682 1.0000
4 -0.0682 0.0280 0.8081 1.0000

```

EIGENVALUES OF OVERLAP MATRIX OF VB ORBITALS

```

=====
0.1730D+00 0.1964D+00 0.1668D+01 0.1962D+01

```

ENERGY AND DIFF OF MACROITER 12 = -76.0671648529 -0.0000003872

MULLIKEN CHARGES ON ATOMS

```

-----
ATOM          CHARGE
1             -0.30204
2              0.15102
3              0.15102
-----

```

ORBITALS OF EACH ELECTRON GROUP IN AO BASIS: ORBITAL(GROUP)

```

=====
Orbital#      1      2      3      4      5      6
Group#        1      1      1      2      2      2
-----
AO# LABELS
1 1 0 s 0.99231 0.13186 -0.00001 0.03108 -0.02658 -0.03108
2 1 0 s -0.05969 0.43404 -0.00000 0.23467 -0.00453 -0.23467
3 1 0 s -0.07345 0.51396 -0.00001 0.04768 -0.10155 -0.04768
4 1 0 x -0.00000 0.00000 -0.00000 0.47296 0.15487 0.47296
5 1 0 y 0.00001 0.00001 0.63124 0.00000 0.00000 -0.00000
6 1 0 z 0.03948 -0.28284 0.00000 0.50242 0.16480 -0.50242
7 1 0 x -0.00000 0.00000 -0.00000 0.15861 0.10720 0.15861
8 1 0 y 0.00000 0.00001 0.49787 0.00000 0.00000 -0.00000
9 1 0 z 0.03214 -0.23924 0.00000 0.21801 0.18116 -0.21801
10 1 0 xx -0.00149 0.00240 -0.00000 0.00327 0.00457 -0.00327
11 1 0 yy -0.00173 0.00364 -0.00000 -0.01537 0.00739 0.01537
12 1 0 zz 0.00037 -0.01184 0.00000 0.01858 0.00967 -0.01858
13 1 0 xy 0.00000 -0.00000 -0.00000 0.00000 -0.00000 0.00000
14 1 0 xz -0.00000 0.00000 -0.00000 0.03321 -0.00022 0.03321
15 1 0 yz 0.00000 0.00000 0.01715 0.00000 0.00000 -0.00000
16 2 H s -0.00377 0.01830 -0.00000 0.11941 0.70133 0.00395
17 2 H s 0.00298 -0.02088 0.00000 0.03922 0.13997 0.01288
18 2 H x 0.00211 -0.01101 0.00000 -0.03287 -0.05703 0.01834

```

```

19 2 H y 0.00000 0.00000 0.02943 0.00000 0.00000 -0.00000
20 2 H z 0.00300 -0.01916 0.00000 -0.01871 -0.02002 -0.01481
21 3 H s -0.00377 0.01830 -0.00000 -0.00395 -0.03993 -0.11941
22 3 H s 0.00298 -0.02088 0.00000 -0.01288 -0.03714 -0.03922
23 3 H x -0.00211 0.01101 -0.00000 0.01834 0.01062 -0.03287
24 3 H y 0.00000 0.00000 0.02943 0.00000 0.00000 0.00000
25 3 H z 0.00300 -0.01916 0.00000 0.01481 0.01942 0.01871

=====
Orbital# 7
Group# 2
-----
AO# LABELS
1 1 0 s 0.02658
2 1 0 s 0.00453
3 1 0 s 0.10155
4 1 0 x 0.15487
5 1 0 y -0.00000
6 1 0 z -0.16480
7 1 0 x 0.10720
8 1 0 y -0.00000
9 1 0 z -0.18116
10 1 0 xx -0.00457
11 1 0 yy -0.00739
12 1 0 zz -0.00967
13 1 0 xy -0.00000
14 1 0 xz -0.00022
15 1 0 yz 0.00000
16 2 H s 0.03993
17 2 H s 0.03714
18 2 H x 0.01062
19 2 H y -0.00000
20 2 H z -0.01942
21 3 H s -0.70133
22 3 H s -0.13997
23 3 H x -0.05703
24 3 H y -0.00000
25 3 H z 0.02002

WORDS OF DYNAMIC MEMORY USED IN VB2000 CODE ONLY: 71358

STATISTICS OF CPU TIMES (SECONDS)
CPU TIME FOR INITIALIZATION 0.019
CPU TIME FOR MACROITERATION 0.548
TOTAL CPU TIME 0.567

```

VB2000 output consists of many parts, and some parts may or may not be printed according to the calculation type and the print control. Example 01 is a typical case of VB calculation. It consists of the following parts:

1. The banner

The output starts with the banner of the program, which includes the version number, the version type (stand alone or GAMESS), the used compiler, the maximum MAP size allowed by the build, the suggested citations for this program and the reference paper and the copyright information.

2. Key words and coordinates

The keywords of the controls and the atom coordinates are echoed in the output.

3. Hartree-Fock molecular orbital coefficients

The Hartree-Fock molecular orbitals and the orbitals energies E(a.u.) will be printed if the keyword **PRINTALL** is included in the control. The molecular orbital coefficients are printed by columns. The first column of each block is the atomic orbital indexes, the second column is the atom indexes, and the third column is the atomic symbol, and the fourth column is the atomic orbital symbol. Starting from the fifth column are the MO coefficients. Each block a maximum of 6 orbitals are printed. If the molecule has more than 6 orbitals, multiple blocks of MO coefficients will be printed.

4. Localized molecular orbitals and the corresponding labels

The orbital coefficients of LMOs are printed in a very similar format as the Hartree-Fock MOs. Each LMO has a bonding label. The LMOs are sorted first by the bonding type and then by orbital energies. The inner core orbitals and lone pairs come first, then the bonding LMOs. For the same bonding type of LMOs, the lower energy LMOs come first. In H₂O, there three non bonding LMOs, one 1s core of the oxygen atom, labeled as 1:(0), and the two lone-pairs, labeled as 1:(1) and 1:(2), respectively. The two bonding LMOs, which corresponding to two O-H bonds, are labeled as 1-2(1) and 1-3(1), respectively.

5. Mulliken charges of LMOs

The atomic charges from Mulliken population analysis of each individual LMO are also printed. It is much easier to check the localization of each LMO from its Mulliken charge distribution. For the inner core and the two lone-pair orbitals, the Mulliken electron charge is almost localized on one atom. For the two bonding LMOs, the Mulliken charge is roughly equal distributed on two bonding atom.

6. General controls

This gives a summary of the GFT controls.

7. Partitioning of LMOs

This table gives the group assignments of all LMOs. The splitting flag of each LMO is also printed. For H₂O, the orbital 1,2 and 3, which are the core, and two lone pairs of O atom, are assigned to group 1, and orbital 4 and 5, which are the two bonding orbitals, are assigned to group 2 (i.e. the VB group). The orbital 1,2 and 3 are not split, and orbital 4 and 5 are split into 4 VB orbitals.

8. Initial guess of orbitals

The initial guess of orbitals of all groups are printed in a very similar format as

LMOs. The orbitals and their corresponding orbital labels are listed in the consequence of the electron groups that they belong to. The first three orbitals belong to the first group, i.e. the Hartree-Fock group, and the last four orbitals belong to the VB group. Orbital 4 and 5 are derived from the bonding LMO of atom 3 and 1, and orbital 6 and 7 are derived from the bonding LMO of atom 2 and 1.

9. Macro iterations

The energy of each macro iteration and the energy change from previous macro iteration are printed. The maximum orbital coefficient change of VB orbital optimization are also printed.

10. Overlap of VB orbitals

The overlap matrix of VB orbitals are printed. Since the matrix is symmetric, only the lower half of the matrix is printed.

11. Final orbitals

The final optimized orbitals of all groups are printed in the same format as above. The group indexes of the orbitals are also printed.

12. Time information

The time elapsed in the total calculation is printed.

8.2 A CASVB(4,4) calculation of H₂O

Input file (example02_h2ocas.inp):

```
#! CASVB(4,4)/D95 PRINTALL  
  
TEST H2O CASVB(4,4)  
  
0 1  
8 .000000 .000000 .000000  
1 .801842 .000000 .555582  
1 -.801842 .000000 .555582  
  
$MACROITER  
13
```

In this case the \$MACROITER directive had to be inserted because the calculation did not converge in 12 iterations (the default value).

Output:

CASVB calculations have more output as compared to the simple VB calculation of Example 01. The automatically generated VB structures are printed:

```
SYMBOLIC VB STRUCTURE(S) OF GROUP 2  
STR. NO.      RUMER PATTERN  
  1          1  2  3  4  
  2          1  3  2  4  
  3          1  1  2  3  
  4          1  1  3  4  
  5          1  1  2  4  
  6          2  2  3  4  
  7          2  2  4  1  
  8          2  2  3  1  
  9          3  3  4  1  
 10          3  3  1  2  
 11          3  3  4  2  
 12          4  4  1  2  
 13          4  4  2  3  
 14          4  4  1  3  
 15          1  1  2  2  
 16          2  2  3  3  
 17          3  3  4  4  
 18          1  1  3  3  
 19          1  1  4  4  
 20          2  2  4  4
```

The final energy and the structure weights computed from Mulliken, Löwdin and Hiberty methods are shown as follows:

```
====Normalized structure coefficients====  
-0.4184 -0.0270 0.0246 -0.2841 0.0208 -0.0658 -0.0120 -0.0128 0.0246 -  
0.2841  
0.0208 -0.0659 -0.0120 -0.0128 0.0061 -0.0912 0.0061 -0.1417 -0.0912  
0.0068  
  
====Mulliken Weight (Chirgwin-Coulson)====  
0.3593 -0.0142 0.0029 0.2080 0.0022 0.0316 0.0009 0.0012 0.0029
```

```

0.2079
  0.0021  0.0317  0.0009  0.0012 -0.0000  0.0400 -0.0000  0.0831  0.0401 -
0.0017

====Inverse Overlap Weight (Gallup-Norbeck)====
  0.1052  0.0005  0.0017  0.2254  0.0012  0.0123  0.0004  0.0005  0.0017
0.2254
  0.0012  0.0123  0.0004  0.0005  0.0004  0.0937  0.0004  0.2223  0.0938
0.0005

====Lowdin Weight====
  0.1050  0.0088  0.0019  0.1794  0.0013  0.0397  0.0006  0.0008  0.0019
0.1794
  0.0013  0.0397  0.0006  0.0008  0.0000  0.1039  0.0000  0.2228  0.1039
0.0083

====Hiberty Weight====
  0.4542  0.0019  0.0016  0.2094  0.0011  0.0112  0.0004  0.0004  0.0016
0.2094
  0.0011  0.0113  0.0004  0.0004  0.0001  0.0216  0.0001  0.0521  0.0216
0.0001

ENERGY AND DIFF OF MACROITER   13 =      -76.0628774423      -0.0000002913

ORBITALS OF EACH ELECTRON GROUP IN AO BASIS: ORBITAL(GROUP)
=====
  Orbital#           1           2           3           4           5           6
  Group#           1           1           1           2           2           2
-----
AO# LABELS
  1  1  0      s  -0.59147   0.07205   0.00000   0.05087  -0.01690   0.05087
  2  1  0      s  -0.47760   0.13527   0.00000   0.07233  -0.02652   0.07233
  3  1  0      s   0.05370  -0.52098   0.00001  -0.26865   0.18995  -0.26867
  4  1  0      s   0.05196  -0.52405   0.00001  -0.06200   0.02694  -0.06200
  5  1  0      x   0.00000  -0.00001  -0.00000   0.64258  -0.08049  -0.64265
  6  1  0      y  -0.00001  -0.00001  -0.72799   0.00000   0.00000   0.00000
  7  1  0      z  -0.02962   0.27982  -0.00000  -0.62116   0.08894  -0.62117
  8  1  0      x   0.00000  -0.00000  -0.00000   0.05370   0.14211  -0.05369
  9  1  0      y  -0.00000  -0.00000  -0.40992   0.00000  -0.00000   0.00000
 10  1  0      z  -0.01672   0.17059  -0.00000  -0.10582  -0.27956  -0.10579
 11  2  H      s   0.00268  -0.02444   0.00000  -0.00770   0.03987  -0.00320
 12  2  H      s  -0.00341   0.03565  -0.00000   0.00885   0.06452   0.00390
 13  3  H      s   0.00268  -0.02443   0.00000  -0.00327  -0.65427  -0.00770
 14  3  H      s  -0.00341   0.03566  -0.00000   0.00387  -0.30932   0.00885
=====

  Orbital#           7
  Group#           2
-----
AO# LABELS
  1  1  0      s  -0.01690
  2  1  0      s  -0.02652
  3  1  0      s   0.18997
  4  1  0      s   0.02696
  5  1  0      x   0.08049
  6  1  0      y   0.00000
  7  1  0      z   0.08895
  8  1  0      x  -0.14211
  9  1  0      y  -0.00000

```

```

10 1 0      z  -0.27957
11 2 H      s  -0.65427
12 2 H      s  -0.30932
13 3 H      s   0.03985
14 3 H      s   0.06451

```

```

STATISTICS OF CPU TIMES (SECONDS)
CPU TIME FOR INITIALIZATION      0.002
CPU TIME FOR MACROITERATION      0.112
TOTAL CPU TIME                   0.114

```

8.3 A CASVB(8,6) calculation of H₂O

Input file (example03_h2o86.inp):

```

#! CASVB(8,6)/D95

TEST H2O CASVB(8,6)

0 1
8 .000000 .000000 .000000
1 .801842 .000000 .555582
1 -.801842 .000000 .555582

$VBGA
1-2 => 2
1-3 => 2
1:(1) => 2
1:(2) => 2

$02VBORB
1:(1)
1:(2)
1-2
1-3

```

Output:

```

====Normalized structure coefficients====
-0.4200 -0.0215 -0.0030 -0.0018 -0.0000 0.0000 0.0000 0.0000 -0.0000
0.0000
0.0000 0.0000 -0.0000 -0.0000 -0.0000 0.0000 0.0000 0.0000 -0.0013
0.0010
0.0076 0.0050 0.0061 0.0024 -0.0000 -0.0000 -0.0000 -0.0000 0.0000
0.0000
0.1294 -0.2982 0.1037 0.0076 -0.0022 -0.0017 0.0001 -0.0000 -0.0000 -
0.0000
0.0000 -0.0000 0.0079 0.0000 -0.0000 0.0000 -0.0000 -0.0011 0.0000
0.0072
-0.0000 0.0023 -0.0013 0.0054 0.0001 0.0096 -0.0001 -0.0000 -0.0000
0.0001
-0.0661 -0.0359 -0.0698 0.0216 -0.2834 0.0118 -0.0717 -0.0102 -0.0124
0.0027
0.0027 -0.0109 0.0007 0.0014 -0.0038 -0.0000 0.0000 0.0001 -0.0000 -
0.0000
0.0010 0.0000 -0.0041 -0.0000 -0.0000 0.0014 -0.0000 -0.0007 -0.0002 -
0.0008
-0.0180 0.0087 -0.0004 0.0055 0.0009 0.0008 -0.0918 0.0061 0.0084

```

```

0.0055
-0.1423 -0.1027 0.0006 0.0016 0.0067

====Mulliken Weight (Chirgwin-Coulson)====
0.3496 -0.0096 -0.0006 -0.0003 0.0000 0.0000 0.0000 -0.0000 0.0000
0.0000
-0.0000 -0.0000 0.0000 -0.0000 -0.0000 -0.0000 0.0000 0.0000 -0.0003 -
0.0001
-0.0003 -0.0002 -0.0006 0.0001 0.0000 -0.0000 -0.0000 0.0000 -0.0000
0.0000
0.0162 0.2165 0.0118 -0.0004 0.0001 0.0001 -0.0000 0.0000 -0.0000
0.0000
0.0000 0.0000 0.0001 0.0000 -0.0000 0.0000 0.0000 0.0000 -0.0000
0.0001
0.0000 -0.0000 -0.0003 0.0001 0.0000 0.0002 -0.0000 0.0000 0.0000
0.0000
0.0300 0.0030 0.0064 -0.0052 0.1956 -0.0022 0.0343 -0.0016 -0.0023 -
0.0002
-0.0007 0.0003 -0.0001 0.0000 -0.0003 0.0000 -0.0000 0.0000 -0.0000 -
0.0000
0.0000 0.0000 0.0000 0.0000 0.0000 0.0000 0.0000 -0.0001 0.0000 -
0.0000
0.0001 -0.0004 -0.0000 0.0001 0.0000 0.0000 0.0356 -0.0002 0.0001
0.0001
0.0835 0.0442 -0.0000 0.0000 -0.0017

====Inverse Overlap Weight (Gallup-Norbeck)====
0.0920 0.0003 0.0000 0.0000 0.0000 0.0000 0.0000 0.0000 0.0000
0.0000
0.0000 0.0000 0.0000 0.0000 0.0000 0.0000 0.0000 0.0000 0.0000
0.0000
0.0000 0.0000 0.0000 0.0000 0.0000 0.0000 0.0000 0.0000 0.0000
0.0000
0.0409 0.2184 0.0259 0.0001 0.0000 0.0000 0.0000 0.0000 0.0000
0.0000
0.0000 0.0000 0.0001 0.0000 0.0000 0.0000 0.0000 0.0000 0.0000
0.0001
0.0000 0.0000 0.0000 0.0001 0.0000 0.0002 0.0000 0.0000 0.0000
0.0000
0.0100 0.0031 0.0113 0.0012 0.1931 0.0003 0.0127 0.0003 0.0004
0.0000
0.0000 0.0000 0.0000 0.0001 0.0000 0.0002 0.0000 0.0000 0.0000
0.0000
0.0100 0.0031 0.0113 0.0012 0.1931 0.0003 0.0127 0.0003 0.0004
0.0000
0.0000 0.0002 0.0000 0.0000 0.0000 0.0000 0.0000 0.0000 0.0000
0.0000
0.0000 0.0000 0.0000 0.0000 0.0000 0.0000 0.0000 0.0000 0.0000
0.0000
0.0032 0.0006 0.0000 0.0002 0.0000 0.0000 0.0772 0.0004 0.0005
0.0002
0.2020 0.1046 0.0000 0.0000 0.0004

====Lowdin Weight====
0.0954 0.0066 0.0010 0.0005 0.0000 0.0000 0.0000 0.0000 0.0000
0.0000
0.0000 0.0000 0.0000 0.0000 0.0000 0.0000 0.0000 0.0000 0.0020
0.0004
0.0001 0.0000 0.0002 0.0000 0.0000 0.0000 0.0000 0.0000 0.0000
0.0000

```

0.0087	0.1883	0.0072	0.0001	0.0002	0.0002	0.0000	0.0000	0.0000
0.0000								
0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
0.0000								
0.0000	0.0000	0.0033	0.0000	0.0000	0.0001	0.0000	0.0000	0.0000
0.0000								
0.0325	0.0023	0.0037	0.0033	0.1575	0.0014	0.0392	0.0006	0.0015
0.0001								
0.0044	0.0001	0.0026	0.0001	0.0007	0.0000	0.0000	0.0000	0.0000
0.0000								
0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0009	0.0000
0.0001								
0.0002	0.0000	0.0000	0.0001	0.0000	0.0000	0.0814	0.0000	0.0001
0.0001								
0.2307	0.1140	0.0001	0.0000	0.0079				
====Hiberty Weight====								
0.4097	0.0011	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
0.0000								
0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
0.0000								
0.0001	0.0001	0.0001	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
0.0000								
0.0389	0.2065	0.0250	0.0001	0.0000	0.0000	0.0000	0.0000	0.0000
0.0000								
0.0000	0.0000	0.0001	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
0.0001								
0.0000	0.0000	0.0000	0.0001	0.0000	0.0002	0.0000	0.0000	0.0000
0.0000								
0.0101	0.0030	0.0113	0.0011	0.1865	0.0003	0.0119	0.0002	0.0004
0.0000								
0.0000	0.0003	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
0.0000								
0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
0.0000								
0.0008	0.0002	0.0000	0.0001	0.0000	0.0000	0.0196	0.0001	0.0002
0.0001								
0.0470	0.0245	0.0000	0.0000	0.0001				
ENERGY AND DIFF OF MACROITER 11 = -76.0649967790 -0.0000001996								
ORBITALS OF EACH ELECTRON GROUP IN AO BASIS: ORBITAL(GROUP)								
=====								
Orbital#		1	2	3	4	5	6	
Group#		1	2	2	2	2	2	

AO# LABELS								
1	1	0	s	-0.58750	-0.09887	-0.00000	-0.05044	-0.00505
2	1	0	s	-0.47083	-0.15643	-0.00000	-0.07135	-0.00822
3	1	0	s	0.02955	0.51856	0.00000	0.26848	-0.07832
4	1	0	s	0.02752	0.52462	0.00000	0.04566	0.09473
5	1	0	x	-0.00000	0.00011	-0.00000	0.58979	-0.07099
6	1	0	y	0.00000	0.00000	-0.72888	-0.00000	0.00000
7	1	0	z	-0.01634	-0.28873	-0.00000	0.66861	-0.15853
8	1	0	x	-0.00000	0.00001	-0.00000	0.05389	0.12420
9	1	0	y	0.00000	0.00000	-0.40885	0.00000	0.00000
10	1	0	z	-0.00863	-0.16700	-0.00000	0.11574	0.24931
11	2	H	s	0.00158	0.02895	0.00000	0.00327	0.63987
12	2	H	s	-0.00176	-0.03446	-0.00000	-0.00388	0.29352

13	3	H	s	0.00158	0.02895	0.00000	0.00520	0.00322	0.00644
14	3	H	s	-0.00176	-0.03447	-0.00000	-0.00611	-0.05784	-0.00771

```
=====
Orbital#           7
Group#             2
-----
```

AO# LABELS

1	1	O	s	-0.00504
2	1	O	s	-0.00821
3	1	O	s	-0.07833
4	1	O	s	0.09464
5	1	O	x	0.07084
6	1	O	y	0.00000
7	1	O	z	-0.15862
8	1	O	x	-0.12420
9	1	O	y	0.00000
10	1	O	z	0.24933
11	2	H	s	0.00321
12	2	H	s	-0.05785
13	3	H	s	0.63987
14	3	H	s	0.29353

STATISTICS OF CPU TIMES (SECONDS)

CPU TIME FOR INITIALIZATION	0.004
CPU TIME FOR MACROITERATION	10.531
TOTAL CPU TIME	10.535

8.4 VB(6) calculation of C₆H₆

Input file (example03_c6h6v6.inp):

```

#! VB(6)/D95 UNITS=BOHR

TEST BENZENE

0 1
6 .000000      2.637000      .000000
1 .000000      4.688000      .000000
6 2.28370898978 1.318500      .000000
1 4.05992709294 2.344000      .000000
6 2.28370898978 -1.318500      .000000
1 4.05992709294 -2.344000      .000000
6 .000000      -2.637000      .000000
1 .000000      -4.688000      .000000
6 -2.28370898978 -1.318500      .000000
1 -4.05992709294 -2.344000      .000000
6 -2.28370898978 1.318500      .000000
1 -4.05992709294 2.344000      .000000

$02PIVB0
6
1,3,5,7,9,11

$02VBSTR
5
1 2 3 4 5 6
2 3 4 5 6 1
1 4 2 3 5 6
2 5 3 4 1 6
3 6 1 2 4 5

$PRINTHS

```

Output:

```

SYMBOLIC VB STRUCTURE(S) OF GROUP 2
STR. NO.      RUMER PATTERN
  1           1 2 3 4 5 6
  2           2 3 4 5 6 1
  3           1 4 2 3 5 6
  4           2 5 3 4 1 6
  5           3 6 1 2 4 5

ENERGY AND DIFF OF MACROITER   1 =  -230.7016473284  -230.7016473284
ENERGY AND DIFF OF MACROITER   2 =  -230.7101485860  -0.0085012576
ENERGY AND DIFF OF MACROITER   3 =  -230.7101729558  -0.0000243699
ENERGY AND DIFF OF MACROITER   4 =  -230.7101732278  -0.0000002720

OVERLAP MATRIX OF VB ORBITALS FOR GROUP 2

      1      2      3      4      5      6
=====
1 1.0000
2 0.5241 1.0000
3 0.0423 0.5241 1.0000

```

```

4-0.1420 0.0423 0.5241 1.0000
5 0.0423-0.1420 0.0423 0.5241 1.0000
6 0.5241 0.0423-0.1420 0.0423 0.5241 1.0000

OVERLAP MATRIX OF VB STRUCTURES FOR GROUP 2

=====
      1      2      3      4      5
=====
1  1.000000
2  0.665076  1.000000
3 -0.825339 -0.825339  1.000000
4 -0.825339 -0.825339  0.748606  1.000000
5 -0.825339 -0.825339  0.748606  0.748606  1.000000

====Normalized structure coefficients====
-0.4432 -0.4432  0.0703  0.0703  0.0703

====Mulliken Weight (Chirgwin-Coulson)====
0.4043  0.4043  0.0638  0.0638  0.0638

====Inverse Overlap Weight (Gallup-Norbeck)====
0.4816  0.4816  0.0123  0.0123  0.0123

====Lowdin Weight====
0.2928  0.2928  0.1381  0.1381  0.1381

====Hiberty Weight====
0.4818  0.4818  0.0121  0.0121  0.0121

ENERGY AND DIFF OF MACROITER 5 = -230.7101732400 -0.0000000121

```

5 VB structures (two Kekule and three Dewar) are used in the computation. Apparently, the two Kekule structures have much larger weights than the three Dewar structures.

8.5 VB(6) calculation of CC triple bond of C₂H₂

Input file (example05_c2h2sp.inp):

```

#! VB(6)/D95

TEST C2H2 TRIPLE BOND (SIGMA+PI)

0 1
6 .000000 .000000 .610009
6 .000000 .000000 -.610009
1 .000000 .000000 1.690006
1 .000000 .000000 -1.690006

$VBGA
1#2 => 2

```

Output:

```

SYMBOLIC VB STRUCTURE(S) OF GROUP 2
STR. NO.      RUMER PATTERN
  1           1 2 3 4 5 6

ENERGY AND DIFF OF MACROITER  1 =  -76.8492475052  -76.8492475052
ENERGY AND DIFF OF MACROITER  2 =  -76.8568274178  -0.0075799126
ENERGY AND DIFF OF MACROITER  3 =  -76.8569746329  -0.0001472151
ENERGY AND DIFF OF MACROITER  4 =  -76.8569839885  -0.0000093557
ENERGY AND DIFF OF MACROITER  5 =  -76.8569849411  -0.0000009526

OVERLAP MATRIX OF VB ORBITALS FOR GROUP 2

  1      2      3      4      5      6
=====
1 1.0000
2 0.9056 1.0000
3 0.0020 0.0025 1.0000
4 0.0025 0.0020 0.6854 1.0000
5 0.0020 0.0025 0.0000 0.0000 1.0000
6 0.0025 0.0020 0.0000 0.0000 0.6854 1.0000

ENERGY AND DIFF OF MACROITER  6 =  -76.8569850698  -0.0000001287

ORBITALS OF EACH ELECTRON GROUP IN AO BASIS: ORBITAL(GROUP)
=====
Orbital#      1      2      3      4      5      6
Group#        1      1      1      1      2      2
-----
AO# LABELS
1 1 C  s  -0.00079  0.60348 -0.07158  0.00045  0.05454  0.10851
2 1 C  s  -0.00106  0.44132 -0.10972  0.00119  0.07099  0.14325
3 1 C  s   0.00211 -0.00657  0.34306  0.00753 -0.05044 -0.49392
4 1 C  s   0.00244 -0.01020  0.27493 -0.00713 -0.08104 -0.16904
5 1 C  x  -0.00000 -0.00000 -0.00001 -0.00001 -0.00059  0.00069
6 1 C  y  -0.00000 -0.00000 -0.00001 -0.00001 -0.00051  0.00060
7 1 C  z  -0.00064 -0.01425  0.35471 -0.00708  0.17449  0.60557
8 1 C  x  -0.00000  0.00000 -0.00001 -0.00000  0.00023 -0.00022
9 1 C  y  -0.00000  0.00000 -0.00000 -0.00000  0.00020 -0.00019
10 1 C  z  -0.00076 -0.00146  0.06390  0.02281 -0.02801  0.00560
11 2 C  s   0.60348 -0.00079  0.00045 -0.07158  0.10851  0.05454

```

12	2	C	s	0.44131	-0.00106	0.00119	-0.10972	0.14325	0.07099
13	2	C	s	-0.00656	0.00211	0.00753	0.34306	-0.49392	-0.05044
14	2	C	s	-0.01020	0.00244	-0.00713	0.27493	-0.16904	-0.08104
15	2	C	x	-0.00000	-0.00000	-0.00001	-0.00001	0.00069	-0.00059
16	2	C	y	-0.00000	-0.00000	-0.00001	-0.00001	0.00060	-0.00051
17	2	C	z	0.01425	0.00064	0.00708	-0.35471	-0.60557	-0.17449
18	2	C	x	0.00000	-0.00000	-0.00000	-0.00001	-0.00022	0.00023
19	2	C	y	0.00000	-0.00000	-0.00000	-0.00000	-0.00019	0.00020
20	2	C	z	0.00146	0.00076	-0.02281	-0.06390	-0.00560	0.02801
21	3	H	s	0.00063	-0.01026	0.30355	0.00372	0.05061	0.05800
22	3	H	s	0.00059	-0.00606	0.15467	-0.00862	0.09330	0.06206
23	4	H	s	-0.01026	0.00063	0.00372	0.30355	0.05800	0.05061
24	4	H	s	-0.00606	0.00059	-0.00862	0.15467	0.06206	0.09330
=====									
Orbital#				7	8	9	10		
Group#				2	2	2	2		

A0#	LABELS								
1	1	C	s	0.00020	0.00021	0.00020	0.00021		
2	1	C	s	0.00026	0.00027	0.00026	0.00027		
3	1	C	s	-0.00064	-0.00070	-0.00064	-0.00070		
4	1	C	s	-0.00028	-0.00036	-0.00028	-0.00036		
5	1	C	x	-0.01150	-0.05386	-0.16041	-0.75098		
6	1	C	y	-0.16041	-0.75098	0.01150	0.05386		
7	1	C	z	0.00094	0.00097	0.00093	0.00097		
8	1	C	x	-0.00450	-0.01717	-0.06278	-0.23949		
9	1	C	y	-0.06278	-0.23949	0.00450	0.01717		
10	1	C	z	-0.00003	-0.00002	-0.00003	-0.00002		
11	2	C	s	0.00021	0.00020	0.00021	0.00020		
12	2	C	s	0.00027	0.00026	0.00027	0.00026		
13	2	C	s	-0.00070	-0.00064	-0.00070	-0.00064		
14	2	C	s	-0.00036	-0.00028	-0.00036	-0.00028		
15	2	C	x	-0.05386	-0.01150	-0.75098	-0.16041		
16	2	C	y	-0.75098	-0.16041	0.05386	0.01150		
17	2	C	z	-0.00097	-0.00094	-0.00097	-0.00093		
18	2	C	x	-0.01717	-0.00450	-0.23949	-0.06278		
19	2	C	y	-0.23949	-0.06278	0.01717	0.00450		
20	2	C	z	0.00002	0.00003	0.00002	0.00003		
21	3	H	s	0.00012	0.00014	0.00012	0.00014		
22	3	H	s	0.00018	0.00021	0.00018	0.00021		
23	4	H	s	0.00014	0.00012	0.00014	0.00012		
24	4	H	s	0.00021	0.00018	0.00021	0.00018		
STATISTICS OF CPU TIMES (SECONDS)									
CPU TIME FOR INITIALIZATION						0.009			
CPU TIME FOR MACROITERATION						0.265			
TOTAL CPU TIME						0.274			

8.6 VB calculation of the 3 banana bonds in C₂H₂

Input (example06_c2h2bn.inp):

```
#! VB(6)/D95 PRINTALL
TEST C2H2 TRIPLE BOND (3 BANANA BONDS, 6 ELECTRONS, VBGA INPUT)

0 1
6 .528275 0.305 0.0
6 -.528275 -0.305 0.0
1 1.463583 0.845 0.0
1 -1.463583 -0.845 0.0

$VBGA
(1#2) => 2

$02VBORB
(1->2) + 1^
(2->1) + 2^
(1->2) - 0.5 (1^ ) + 0.866025 (1^(2))
(2->1) - 0.5 (2^ ) + 0.866025 (2^(2))
(1->2) - 0.5 (1^ ) - 0.866025 (1^(2))
(2->1) - 0.5 (2^ ) - 0.866025 (2^(2))
```

Final energy:

```
ENERGY AND DIFF OF MACROITER    9 =    -76.8647190750    -0.0000001627
```

8.7 VB(10) calculation of C₂H₂

Input (example07_c2h210.inp):

```
#! VB(10)/D95  
TEST C2H2 TRIPLE BOND (SIGMA+PI)  
  
0 1  
6 .000000 .000000 .610009  
6 .000000 .000000 -.610009  
1 .000000 .000000 1.690006  
1 .000000 .000000 -1.690006
```

Final results:

ENERGY AND DIFF OF MACROITER	1 =	-76.8811350398	-76.8811350398
ENERGY AND DIFF OF MACROITER	2 =	-76.8874067478	-0.0062717080
ENERGY AND DIFF OF MACROITER	3 =	-76.8875469654	-0.0001402176
ENERGY AND DIFF OF MACROITER	4 =	-76.8875489401	-0.0000019747
ENERGY AND DIFF OF MACROITER	5 =	-76.8875490039	-0.0000000638
ENERGY AND DIFF OF MACROITER	6 =	-76.8875490152	-0.0000000113

8.8 VB(12) calculation of C₂H₄

Input (example08_c2h412.inp):

```

#! VB(12)/D95

TEST C2H4 DOUBLE BOND. ALL VALENCE ELECTRONS AND BONDS

0 1
6 .000000 .000000 .000000
6 .000000 .000000 1.319979
1 .935305 .000000 -.539999
1 -.935305 .000000 -.539999
1 -.935305 .000000 1.859978
1 .935305 .000000 1.859978

$02VBSCF
0.001,5

```

Output:

```

ENERGY AND DIFF OF MACROITER   1 =   -78.0946472691   -78.0946472691
ENERGY AND DIFF OF MACROITER   2 =   -78.1113030008   -0.0166557317
ENERGY AND DIFF OF MACROITER   3 =   -78.1115812275   -0.0002782267
ENERGY AND DIFF OF MACROITER   4 =   -78.1115846984   -0.0000034709
ENERGY AND DIFF OF MACROITER   5 =   -78.1115849266   -0.0000002282

OVERLAP MATRIX OF VB ORBITALS FOR GROUP   2
=====
      1      2      3      4      5      6      7      8      9     10     11     12
=====
 1 1.0000
 2 0.8917 1.0000
 3 -0.2248 -0.1723 1.0000
 4 -0.1661 -0.1286 0.8352 1.0000
 5 0.1723 0.2248 0.0860 0.0317 1.0000
 6 0.1286 0.1661 0.0317 -0.0074 0.8352 1.0000
 7 0.1723 0.2248 -0.1208 -0.0575 0.2174 0.1542 1.0000
 8 0.1286 0.1661 -0.0575 -0.0102 0.1542 0.1141 0.8352 1.0000
 9 0.2248 0.1723 -0.2174 -0.1542 0.1208 0.0575 -0.0860 -0.0317 1.0000
10 0.1661 0.1286 -0.1542 -0.1141 0.0575 0.0102 -0.0317 0.0074 0.8352 1.0000
11 0.0008 0.0011 -0.0002 -0.0002 0.0002 0.0002 0.0002 0.0002 0.0002 0.0002 1.0000
12 0.0011 0.0008 -0.0002 -0.0002 0.0002 0.0002 0.0002 0.0002 0.0002 0.0002 0.6462 1.0000

ENERGY AND DIFF OF MACROITER   6 =   -78.1115850009   -0.0000000744

```

8.9 VB(14) calculation of C₂H₆

Input (example09_c2h614):

```

#! VB(14)/D95

TEST C2H6 (7 SIGMA BONDS)

0 1
6 .000000 .000000 .000000
6 .000000 .000000 1.540011
1 1.027669 .000000 -.363311
1 -.513834 -.889987 -.363311
1 -.513834 .889987 -.363311
1 -1.027669 .000000 1.903322
1 .513834 .889987 1.903322
1 .513834 -.889987 1.903322

$02VBSCF
0.001,5

```

Output:

```

SYMBOLIC VB STRUCTURE(S) OF GROUP 2
STR. NO.      RUMER PATTERN
  1          1 2 3 4 5 6 7 8 9 10 11 12 13 14

ENERGY AND DIFF OF MACROITER    1 =    -79.2784050719    -79.2784050719

ENERGY AND DIFF OF MACROITER    2 =    -79.3154434169    -0.0370383451

ENERGY AND DIFF OF MACROITER    3 =    -79.3160887499    -0.0006453330

ENERGY AND DIFF OF MACROITER    4 =    -79.3160987894    -0.0000100395

ENERGY AND DIFF OF MACROITER    5 =    -79.3160991701    -0.0000003808

OVERLAP MATRIX OF VB ORBITALS FOR GROUP 2

=====
      1      2      3      4      5      6      7      8      9      10      11      12
=====
  1 1.0000
  2 0.8397 1.0000
  3 0.1530 0.2025 1.0000
  4 0.1148 0.1565 0.8348 1.0000
  5 -0.2025 -0.1530 -0.0684 -0.0391 1.0000
  6 -0.1565 -0.1148 -0.0391 -0.0184 0.8348 1.0000
  7 0.1530 0.2025 0.2026 0.1614 0.0738 0.0344 1.0000
  8 0.1148 0.1565 0.1614 0.1252 0.0344 0.0092 0.8348 1.0000
  9 0.1530 0.2025 0.2026 0.1614 -0.0684 -0.0391 0.2026 0.1614 1.0000
 10 0.1148 0.1565 0.1614 0.1252 -0.0391 -0.0184 0.1614 0.1252 0.8348 1.0000
 11 -0.2025 -0.1530 -0.0684 -0.0391 0.2026 0.1614 -0.0684 -0.0391 0.0738 0.0344 1.0000
 12 -0.1565 -0.1148 -0.0391 -0.0184 0.1614 0.1252 -0.0391 -0.0184 0.0344 0.0092 0.8348 1.0000
 13 0.2025 0.1530 0.0738 0.0344 -0.2026 -0.1614 0.0684 0.0391 0.0684 0.0391 -0.2026 -0.1614
 14 0.1565 0.1148 0.0344 -0.0092 -0.1614 -0.1252 0.0391 0.0184 0.0391 0.0184 -0.1614 -0.1252

      13      14
=====
 13 1.0000
 14 0.8348 1.0000

ENERGY AND DIFF OF MACROITER    6 =    -79.3160992157    -0.0000000455

```

8.10 VB(4) calculation of the triplet state and the spin density of TMM

Input (example10_tmm_t.inp):

```
#! VB(4)/D95 UNITS=BOHR SPDEN PRINTALL

TMM TRIPLET

0 3
6 0.0000000000 0.0000000000 0.0000000000
6 0.0000000000 2.6985287117 0.0000000000
6 2.3369944172 -1.3492643558 0.0000000000
6 -2.3369944172 -1.3492643558 0.0000000000
1 -1.7528496640 3.7475875942 0.0000000000
1 1.7528496640 3.7475875942 0.0000000000
1 4.1219308914 -0.3557814591 0.0000000000
1 2.3690812275 -3.3918061351 0.0000000000
1 -4.1219308914 -0.3557814591 0.0000000000
1 -2.3690812275 -3.3918061351 0.0000000000

$GRPDIM
13,8

$02VBSTR
3
1 2 3 4
1 3 4 2
1 4 2 3

$AOGROUP
8
7,17,27,37,10,20,30,40
2,2, 2, 2, 2, 2, 2, 2

$NOTROT
2
1,2
2,3
```

This is a case of σ - π separation. We can divide AO space into two subspaces, one for σ electrons and the other for π electrons. With D95 basis set, there are 8 π AOs, all assigned to Group 2, i.e. the π electron group described by VB method. The remaining AOs belong to Group 1 (Hartree-Fock group) and Group 3 (virtual orbitals). Because of the symmetry of the molecule, the σ and π orbitals should not be mixed, i.e. Group 2 does not mix with Group 1 and Group 3. This is controlled by **\$NOTROT** flag. Even though only 4 VB orbitals are used for the π electrons, the VB orbitals are optimized within the 8 π AO subspace. The core-electrons are optimized within the remaining AO subspace (Group 1 and Group 3). The key word **SPDEN** tells the program to compute spin density. The triplet state of the TMM has the full symmetry. To describe the state with proper symmetry, three symmetry related VB structures (also linearly independent) are involved. In each VB structure, the first two orbitals form a single bond between the central carbon atom and one of its neighbor carbon, and the last two orbitals have unpaired (parallel) spins. As shown by the calculation, all three structures have exactly the same weights. The 4 VB orbitals show the same symmetry. The results of the final iteration are shown

below:

```

OVERLAP MATRIX OF VB ORBITALS FOR GROUP    2
      1      2      3      4      5      6      7      8
=====
  1  1.0000
  2  0.4519  1.0000
  3  0.4519-0.0615  1.0000
  4  0.4519-0.0615-0.0615  1.0000
  5  0.8083  0.4089  0.4089  0.4089  1.0000
  6  0.5051  0.7603  0.0834  0.0834  0.6588  1.0000
  7  0.5051  0.0834  0.7603  0.0834  0.6588  0.2860  1.0000
  8  0.5051  0.0834  0.0834  0.7603  0.6588  0.2860  0.2860  1.0000

OVERLAP MATRIX OF VB STRUCTURES FOR GROUP    2
      1      2      3
=====
  1  1.000000
  2  0.711632  1.000000
  3  0.711632  0.711632  1.000000

====Normalized structure coefficients====
-0.3709 -0.3709 -0.3709

====Mulliken Weight (Chirgwin-Coulson)====
  0.3333  0.3333  0.3333

====Inverse Overlap Weight (Gallup-Norbeck)====
  0.3333  0.3333  0.3333

====Lowdin Weight====
  0.3333  0.3333  0.3333

====Hiberty Weight====
  0.3333  0.3333  0.3333

SPIN DENSITY ON ATOMS
-----
ATOM      DENSITY
  1      -0.33634
  2       0.77878
  3       0.77878
  4       0.77878
  5       0.00000
  6       0.00000
  7       0.00000
  8       0.00000
  9       0.00000
 10       0.00000
-----

ENERGY AND DIFF OF MACROITER    5 =   -154.8692940179   -0.0000000046

```

8.11 VB(4) calculation of the singlet state of TMM

Input (example11_tmm_s.inp):

```

#! VB(4)/D95 UNITS=BOHR

TEST TMM SINGLET

0 1
6      0.0000000000      0.0899765479      0.0000000000
6      0.0000000000      2.6789011522      0.0000000000
6      2.4206928037     -1.370278709      0.0000000000
6     -2.4206928037     -1.370278709      0.0000000000
1     -1.7489492060      3.7381024410      0.0000000000
1      1.7489492060      3.7381024410      0.0000000000
1      4.2227041376     -0.4121322875      0.0000000000
1      2.3886356760     -3.4109309960      0.0000000000
1     -4.2227041376     -0.4121322875      0.0000000000
1     -2.3886356760     -3.4109309960      0.0000000000

$GRPDIM
13,8

$AOGROUP
8
7,17,27,37,10,20,30,40
2,2, 2, 2, 2, 2, 2, 2

$02VBSTR
2
1 2 3 4
1 3 2 4

$NOTROT
2
1,2
2,3

```

The input of this calculation is very similar to that for Example 10. The only difference is the multiplicity and the choice of VB structures. The singlet state has a lower symmetry. As the results confirm, only one VB structure is needed to describe the singlet state properly. Since the singlet state has no unpaired spin, no spin-density calculation is needed.

Output:

```

SYMBOLIC VB STRUCTURE(S) OF GROUP 2
STR. NO.      RUMER PATTERN
  1           1 2 3 4
  2           1 3 2 4

ENERGY AND DIFF OF MACROITER    1 =   -154.8274343149   -154.8274343149
ENERGY AND DIFF OF MACROITER    2 =   -154.8382554220    -0.0108211070

```

ENERGY AND DIFF OF MACROITER	3 =	-154.8388516348	-0.0005962128
ENERGY AND DIFF OF MACROITER	4 =	-154.8388928990	-0.0000412642
ENERGY AND DIFF OF MACROITER	5 =	-154.8388958285	-0.0000029296
ENERGY AND DIFF OF MACROITER	6 =	-154.8388960930	-0.0000002644

OVERLAP OF VB ORBITALS FOR GROUP 2

	1	2	3	4	5	6	7	8
1	1.0000							
2	0.6287	1.0000						
3	0.4154	0.1255	1.0000					
4	0.4154	0.1255	0.0972	1.0000				
5	0.7818	0.5763	0.4141	0.4141	1.0000			
6	0.5372	0.7695	0.0929	0.0929	0.6811	1.0000		
7	0.4373	0.2446	0.7763	0.1358	0.6326	0.2794	1.0000	
8	0.4373	0.2446	0.1358	0.7763	0.6326	0.2794	0.2610	1.0000

OVERLAP MATRIX OF VB STRUCTURES FOR GROUP 2

	1	2
1	1.000000	
2	-0.590604	1.000000

====Normalized structure coefficients====
-1.0000 0.0000

====Mulliken Weight (Chirgwin-Coulson)====
1.0000 0.0000

====Inverse Overlap Weight (Gallup-Norbeck)====
1.0000 0.0000

====Lowdin Weight====
0.9329 0.0671

====Hiberty Weight====
1.0000 0.0000

ENERGY AND DIFF OF MACROITER	7 =	-154.8388961328	-0.0000000398
------------------------------	-----	-----------------	---------------

8.12 A two-VB group description of CH₃CH₂OH

Input (example12_etOH.inp):

```
#! VB(8).VB(8)/D95 PRINTALL
TEST C2H5OH (TWO VB GROUPS)

0 1
6 -0.4413 1.9095 -0.1486
1 0.1529 2.7834 -0.4194
1 -1.2624 1.8366 -0.8622
1 -0.8754 2.0884 0.8352
1 -0.2076 -0.2379 0.0887
6 0.4029 0.6280 -0.1666
1 0.8338 0.4543 -1.1531
1 2.0387 1.4420 0.5056
8 1.4477 0.7230 0.7814

$VBGA
1-2 => 2
1-3 => 2
1-4 => 2
1-6 => 2
6-7 => 3
5-6 => 3
6-9 => 3
8-9 => 3
```

This example shows how to divide a molecule into electron groups by grouping chemical bonds. The \$VBGA control flag provides a simple way of doing this. The results of two-VB group calculation are shown as follows:

```
SYMBOLIC VB STRUCTURE(S) OF GROUP 2
STR. NO.      RUMER PATTERN
      1      1 2 3 4 5 6 7 8

VB-ORBITAL OPTIMIZATION OF GROUP 2 ...
GROUP = 2 ITER = 1 CMAX = 0.13924293439705
GROUP = 2 ITER = 2 CMAX = 0.08342591696204
GROUP = 2 ITER = 3 CMAX = 0.04181617971171
GROUP = 2 ITER = 4 CMAX = 0.01564416087022
GROUP = 2 ITER = 5 CMAX = 0.00312063574502
GROUP = 2 ITER = 6 CMAX = 0.00100537619157
GROUP = 2 ITER = 7 CMAX = 0.00033746279460

SYMBOLIC VB STRUCTURE(S) OF GROUP 3
STR. NO.      RUMER PATTERN
      1      1 2 3 4 5 6 7 8

VB-ORBITAL OPTIMIZATION OF GROUP 3 ...
GROUP = 3 ITER = 1 CMAX = 0.15000000000000
GROUP = 3 ITER = 2 CMAX = 0.14971704297699
GROUP = 3 ITER = 3 CMAX = 0.10304118090716
GROUP = 3 ITER = 4 CMAX = 0.05332745446400
GROUP = 3 ITER = 5 CMAX = 0.03228901817009
GROUP = 3 ITER = 6 CMAX = 0.01389017735583
GROUP = 3 ITER = 7 CMAX = 0.00113239572375
GROUP = 3 ITER = 8 CMAX = 0.00016400465668
```

```

ENERGY AND DIFF OF MACROITER      1 =   -154.0701115160   -154.0701115160

VB-ORBITAL OPTIMIZATION OF GROUP  2 ...
GROUP =  2  ITER =  1  CMAX =  0.02120275962196
GROUP =  2  ITER =  2  CMAX =  0.01962587012748
GROUP =  2  ITER =  3  CMAX =  0.00429610870464
GROUP =  2  ITER =  4  CMAX =  0.00585827700563
GROUP =  2  ITER =  5  CMAX =  0.00155171691335
GROUP =  2  ITER =  6  CMAX =  0.00001977279807

VB-ORBITAL OPTIMIZATION OF GROUP  3 ...
GROUP =  3  ITER =  1  CMAX =  0.02047895280394
GROUP =  3  ITER =  2  CMAX =  0.04335388752638
GROUP =  3  ITER =  3  CMAX =  0.00502992659617
GROUP =  3  ITER =  4  CMAX =  0.00880878670298
GROUP =  3  ITER =  5  CMAX =  0.00099669761204

ENERGY AND DIFF OF MACROITER      2 =   -154.1708378715   -0.1007263555

VB-ORBITAL OPTIMIZATION OF GROUP  2 ...
GROUP =  2  ITER =  1  CMAX =  0.00301127200409
GROUP =  2  ITER =  2  CMAX =  0.00368234286654
GROUP =  2  ITER =  3  CMAX =  0.00023060589108

VB-ORBITAL OPTIMIZATION OF GROUP  3 ...
GROUP =  3  ITER =  1  CMAX =  0.00373166288427
GROUP =  3  ITER =  2  CMAX =  0.00837749797015
GROUP =  3  ITER =  3  CMAX =  0.00105869434887
GROUP =  3  ITER =  4  CMAX =  0.00000322992691

ENERGY AND DIFF OF MACROITER      3 =   -154.1784486042   -0.0076107327

VB-ORBITAL OPTIMIZATION OF GROUP  2 ...
GROUP =  2  ITER =  1  CMAX =  0.00090948125615

VB-ORBITAL OPTIMIZATION OF GROUP  3 ...
GROUP =  3  ITER =  1  CMAX =  0.00222321149693
GROUP =  3  ITER =  2  CMAX =  0.00238967550651
GROUP =  3  ITER =  3  CMAX =  0.00023912292670

ENERGY AND DIFF OF MACROITER      4 =   -154.1794423518   -0.0009937476

VB-ORBITAL OPTIMIZATION OF GROUP  2 ...
GROUP =  2  ITER =  1  CMAX =  0.00042789532930

VB-ORBITAL OPTIMIZATION OF GROUP  3 ...
GROUP =  3  ITER =  1  CMAX =  0.00098051895593

ENERGY AND DIFF OF MACROITER      5 =   -154.1796312700   -0.0001889182

VB-ORBITAL OPTIMIZATION OF GROUP  2 ...
GROUP =  2  ITER =  1  CMAX =  0.00029858811036

VB-ORBITAL OPTIMIZATION OF GROUP  3 ...
GROUP =  3  ITER =  1  CMAX =  0.00112891230973
GROUP =  3  ITER =  2  CMAX =  0.00148159296096
GROUP =  3  ITER =  3  CMAX =  0.00006116589196

ENERGY AND DIFF OF MACROITER      6 =   -154.1796733250   -0.0000420551

```

```

VB-ORBITAL OPTIMIZATION OF GROUP 2 ...
GROUP = 2 ITER = 1 CMAX = 0.00016917796923

VB-ORBITAL OPTIMIZATION OF GROUP 3 ...
GROUP = 3 ITER = 1 CMAX = 0.00043956851106

ENERGY AND DIFF OF MACROITER 7 = -154.1796854008 -0.0000120758

VB-ORBITAL OPTIMIZATION OF GROUP 2 ...
GROUP = 2 ITER = 1 CMAX = 0.00010199548897

VB-ORBITAL OPTIMIZATION OF GROUP 3 ...
GROUP = 3 ITER = 1 CMAX = 0.00046409390505

ENERGY AND DIFF OF MACROITER 8 = -154.1796891573 -0.0000037565

VB-ORBITAL OPTIMIZATION OF GROUP 2 ...
GROUP = 2 ITER = 1 CMAX = 0.00007470211545

VB-ORBITAL OPTIMIZATION OF GROUP 3 ...
GROUP = 3 ITER = 1 CMAX = 0.00030921628942

ENERGY AND DIFF OF MACROITER 9 = -154.1796905018 -0.0000013446

VB-ORBITAL OPTIMIZATION OF GROUP 2 ...
GROUP = 2 ITER = 1 CMAX = 0.00005055080195

VB-ORBITAL OPTIMIZATION OF GROUP 3 ...
GROUP = 3 ITER = 1 CMAX = 0.00028715707145

ENERGY AND DIFF OF MACROITER 10 = -154.1796910483 -0.0000005464

VB-ORBITAL OPTIMIZATION OF GROUP 2 ...
GROUP = 2 ITER = 1 CMAX = 0.00003869551720

OVERLAP MATRIX OF VB ORBITALS FOR GROUP 2
      1      2      3      4      5      6      7      8
=====
1 1.0000
2 0.8378 1.0000
3 0.2106 0.1611 1.0000
4 0.1648 0.1212 0.8368 1.0000
5 0.2118 0.1567 0.2057 0.1645 1.0000
6 0.1573 0.1159 0.1645 0.1263 0.8366 1.0000
7 0.2042 0.1509 0.2063 0.1669 0.2115 0.1682 1.0000
8 0.1521 0.1120 0.1670 0.1289 0.1686 0.1291 0.8367 1.0000

EIGENVALUES OF OVERLAP MATRIX OF VB ORBITALS
=====
      0.1521D+00      0.1599D+00      0.1613D+00      0.1683D+00      0.1495D+01
0.1506D+01
      0.1533D+01      0.2825D+01

VB-ORBITAL OPTIMIZATION OF GROUP 3 ...
GROUP = 3 ITER = 1 CMAX = 0.00019176684512

OVERLAP MATRIX OF VB ORBITALS FOR GROUP 3
      1      2      3      4      5      6      7      8

```

```

=====
1 1.0000
2 0.8033 1.0000
3-0.2232-0.1181 1.0000
4-0.1177-0.0157 0.8064 1.0000
5 0.0650 0.0467-0.1287-0.2012 1.0000
6 0.0401 0.0269-0.0904-0.1369 0.8334 1.0000
7 0.1089 0.0895 0.0776 0.1404-0.2022-0.1622 1.0000
8 0.0580 0.0406 0.0578 0.0963-0.1644-0.1221 0.8316 1.0000
ENERGY AND DIFF OF MACROITER 11 = -154.1796912768 -0.0000002285

```

8.13 All-valence electron VB calculation of C₆H₆

Input (example13_bzGF14.inp):

```

#! GPF(14)/STO-3G UNITS=BOHR NOBIAS FROZEN

TEST BENZENE 14 GROUPS (1 FROZEN CORE GROUP + 12 SIGMA
BONDS + 1 PI-GROUP)

0 1
6 .000000 2.637000 .000000
1 .000000 4.688000 .000000
6 2.28370898978 1.318500 .000000
1 4.05992709294 2.344000 .000000
6 2.28370898978 -1.318500 .000000
1 4.05992709294 -2.344000 .000000
6 .000000 -2.637000 .000000
1 .000000 -4.688000 .000000
6 -2.28370898978 -1.318500 .000000
1 -4.05992709294 -2.344000 .000000
6 -2.28370898978 1.318500 .000000
1 -4.05992709294 2.344000 .000000

$GENCTL
12,2,2,2,2,2,2,2,2,2,2,2,6
1,2,2,2,2,2,2,2,2,2,2,2,2

$VBSCF
0.001,10

$14PIVB0
6
1,3,5,7,9,11

$14VBSTR
5
1 2 3 4 5 6
2 3 4 5 6 1
1 4 2 3 5 6
2 5 3 4 6 1
3 6 4 5 1 2

$NOTROT
13
1,14
2,14
3,14
4,14
5,14
6,14
7,14
8,14
9,14
10,14
11,14
12,14
13,14

$PRINTHS

```

This example shows how to perform a VB calculation with all valence electrons included in the VB groups. The electrons are divided into 14 groups, including an HF group of 12 carbon core electrons, 12 GVB pairs, and the 6- π electron group. The σ - π mixing is suppressed by using the \$NOTROT control. The results are shown as follows:

```

PARTITIONING OF LMOs INTO GROUPS ($LMOGRPMODIFY)
=====
LMO#      1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20
Group#    1  1  1  1  1  1  2  3  4  5  6  7  8  9 10 11 12 13 14 14
Split     0  0  0  0  0  0  1  1  1  1  1  1  1  1  1  1  1  1  1  1
LMO#      21
Group#    14
Split     1

      GENERAL CONTROLS ($GENCTL)
=====
Number of electron groups      = 14
Maximum macro-iterations      = 12
Energy threshold               = 10**(- 6)
Restart calculation(0/1/2)     = 0
Group#                         1  2  3  4  5  6  7  8  9 10 11 12 13 14
Num. of electrons              12  2  2  2  2  2  2  2  2  2  2  2  2  6
Num. of spins                  0  0  0  0  0  0  0  0  0  0  0  0  0  0
Num. of orbitals               6  2  2  2  2  2  2  2  2  2  2  2  2  6
Method#                        1  2  2  2  2  2  2  2  2  2  2  2  2  2

SYMBOLIC VB STRUCTURE(S) OF GROUP 2
STR. NO.      RUMER PATTERN
  1           1  2

SYMBOLIC VB STRUCTURE(S) OF GROUP 3
STR. NO.      RUMER PATTERN
  1           1  2

SYMBOLIC VB STRUCTURE(S) OF GROUP 4
STR. NO.      RUMER PATTERN
  1           1  2

SYMBOLIC VB STRUCTURE(S) OF GROUP 5
STR. NO.      RUMER PATTERN
  1           1  2

SYMBOLIC VB STRUCTURE(S) OF GROUP 6
STR. NO.      RUMER PATTERN
  1           1  2

SYMBOLIC VB STRUCTURE(S) OF GROUP 7
STR. NO.      RUMER PATTERN
  1           1  2

SYMBOLIC VB STRUCTURE(S) OF GROUP 8
STR. NO.      RUMER PATTERN
  1           1  2

SYMBOLIC VB STRUCTURE(S) OF GROUP 9
STR. NO.      RUMER PATTERN
  1           1  2

SYMBOLIC VB STRUCTURE(S) OF GROUP 10

```

STR. NO.	RUMER PATTERN					
1	1	2				
SYMBOLIC VB STRUCTURE(S) OF GROUP 11						
STR. NO.	RUMER PATTERN					
1	1	2				
SYMBOLIC VB STRUCTURE(S) OF GROUP 12						
STR. NO.	RUMER PATTERN					
1	1	2				
SYMBOLIC VB STRUCTURE(S) OF GROUP 13						
STR. NO.	RUMER PATTERN					
1	1	2				
SYMBOLIC VB STRUCTURE(S) OF GROUP 14						
STR. NO.	RUMER PATTERN					
1	1	2	3	4	5	6
2	2	3	4	5	6	1
3	1	4	2	3	5	6
4	2	5	3	4	6	1
5	3	6	4	5	1	2
ENERGY AND DIFF OF MACROITER						
1 =	-227.8976820484					-227.8976820484
ENERGY AND DIFF OF MACROITER						
2 =	-228.1203811094					-0.2226990610
ENERGY AND DIFF OF MACROITER						
3 =	-228.1272358854					-0.0068547760
ENERGY AND DIFF OF MACROITER						
4 =	-228.1274777898					-0.0002419044
ENERGY AND DIFF OF MACROITER						
5 =	-228.1274885396					-0.0000107498
ENERGY AND DIFF OF MACROITER						
6 =	-228.1274891633					-0.0000006237
OVERLAP MATRIX OF VB ORBITALS FOR GROUP 2						
	1	2				
=====						
1	1.0000					
2	0.8876	1.0000				
OVERLAP MATRIX OF VB ORBITALS FOR GROUP 3						
	1	2				
=====						
1	1.0000					
2	0.8876	1.0000				
OVERLAP MATRIX OF VB ORBITALS FOR GROUP 4						
	1	2				
=====						
1	1.0000					
2	0.8876	1.0000				
OVERLAP MATRIX OF VB ORBITALS FOR GROUP 5						
	1	2				
=====						

```

1 1.0000
2 0.8876 1.0000

OVERLAP MATRIX OF VB ORBITALS FOR GROUP 6

      1      2
=====
1 1.0000
2 0.8876 1.0000

OVERLAP MATRIX OF VB ORBITALS FOR GROUP 7

      1      2
=====
1 1.0000
2 0.8876 1.0000

OVERLAP MATRIX OF VB ORBITALS FOR GROUP 8

      1      2
=====
1 1.0000
2 0.8360 1.0000

OVERLAP MATRIX OF VB ORBITALS FOR GROUP 9

      1      2
=====
1 1.0000
2 0.8360 1.0000

OVERLAP MATRIX OF VB ORBITALS FOR GROUP 10

      1      2
=====
1 1.0000
2 0.8360 1.0000

OVERLAP MATRIX OF VB ORBITALS FOR GROUP 11

      1      2
=====
1 1.0000
2 0.8360 1.0000

OVERLAP MATRIX OF VB ORBITALS FOR GROUP 12

      1      2
=====
1 1.0000
2 0.8360 1.0000

OVERLAP MATRIX OF VB ORBITALS FOR GROUP 13

      1      2
=====
1 1.0000
2 0.8360 1.0000

OVERLAP MATRIX OF VB ORBITALS FOR GROUP 14

```

```

      1      2      3      4      5      6
=====
1 1.0000
2 0.5059 1.0000
3 0.0626 0.5060 1.0000
4-0.1062 0.0626 0.5059 1.0000
5 0.0627-0.1062 0.0626 0.5060 1.0000
6 0.5061 0.0627-0.1062 0.0626 0.5059 1.0000

OVERLAP MATRIX OF VB STRUCTURES FOR GROUP 14

      1      2      3      4      5
=====
1 1.000000
2 0.619062 1.000000
3 -0.795875 -0.795907 1.000000
4 -0.795876 -0.795906 0.701985 1.000000
5 -0.795874 -0.795907 0.701985 0.701985 1.000000

====Normalized structure coefficients====
0.4379 0.4380 -0.0797 -0.0797 -0.0797

====Mulliken Weight (Chirgwin-Coulson)====
0.3938 0.3939 0.0708 0.0708 0.0708

====Inverse Overlap Weight (Gallup-Norbeck)====
0.4757 0.4764 0.0160 0.0160 0.0160

====Lowdin Weight====
0.2927 0.2931 0.1381 0.1381 0.1381

====Hiberty Weight====
0.4762 0.4765 0.0158 0.0158 0.0158

ENERGY AND DIFF OF MACROITER 7 = -228.1274892103 -0.0000000470

```

8.14 A three-VB group description of a model system of $\text{Si}\equiv\text{Si}$ triple bond in Si_4H_6

Input (example14_si4h6.inp):

```
#! VB(8).CASVB(6,6).VB(8)/6-31G* PRINTALL
```

```
TEST SI4H6 TRIPLE BOND
```

```
0 1
```

```
14 -0.8920 -0.5156 .000000
14 0.8920 0.5156 .000000
14 -3.1696 0.1200 .000000
14 3.1696 -0.1200 .000000
1 -3.2926 1.5924 .000000
1 -3.8226 -0.4408 -1.203300
1 -3.8226 -0.4408 1.203300
1 3.2926 -1.5924 .000000
1 3.8226 0.4408 -1.203300
1 3.8226 0.4408 1.203300
```

```
$MEMORY
```

```
9000000
```

```
$VBGA
```

```
(3-5) => 2
```

```
(3-6) => 2
```

```
(3-7) => 2
```

```
(1-3) => 2
```

```
(1#2) => 3
```

```
(4-8) => 4
```

```
(4-9) => 4
```

```
(4-10) => 4
```

```
(2-4) => 4
```

```
$03VBORB
```

```
(1-2)
```

```
1^
```

```
2^
```

```
1:
```

```
2:
```

```
$NOCANONPI
```

```
$CMAXCUT
```

```
0.001
```

One prominent difference between the $\text{Si}\equiv\text{Si}$ triple bond and the $\text{C}\equiv\text{C}$ triple bond is that the two terminal groups of the $\text{Si}\equiv\text{Si}$ triple bond are not co-linear as shown in the following figure:

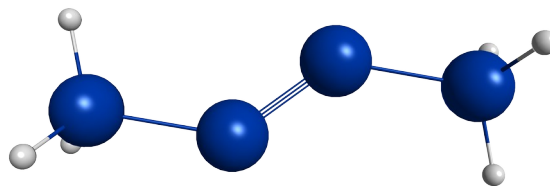


Figure 8.1. Chemical structure and molecular geometry of Si₄H₆.

The molecule is divided into three parts, the two -SiH₃ groups and the Si≡Si triple bond. The Si≡Si triple bond is described by CASVB(6,6), and the other two groups are described by VB(8). The CASVB provides the maximum flexibility for the description of the unusual triple bond, while the simple VB method is used for the 4 regular bonds in each -SiH₃ group.

The specification of the initial 6 VB orbitals of the Si≡Si triple bond is a little tricky. Owing to the bend structure, the bonds in the Si-Si link cannot be regarded as forming a normal triple bond. Thus, the method for generating initial VBOs by splitting the corresponding LMOs does not work well here. Therefore, we use VBO library and the `$02VBORB` control flag to generate the 6 orbitals. In the `$02VBORB` control, the first two VBOs are forming the Si-Si σ bond, as specified by (1-2). The next two lines specify the two π orbitals on the two central Si atoms. The last two specify the lone pairs on each central Si atom. Even though there are no lone pairs on any Si atom in this molecule, we use a virtual lone pair orbital to generate an initial VBO with proper orientation. VB2000 uses the lone pair orbital of sulfur as an approximation for the silicons. The VBOs make an angle with the central Si-Si link of roughly 120 degree.

This calculation has 88 basis function involved. The memory allocated by default does not allow a full in-core integral transformation. By allocating large memory with the flag `$MEMORY`, the CPU time can be reduced nearly by half, and the elapsed time can be reduced significantly.

The HF energy and the three-VB group calculation energy result as follows:

```
Total number of two-electron integrals =      3284766
Time for integral evaluation              3.626

SKIPPED INTEGRALS =                        3909291
Number of iter =      26, DMAX =           0.000006705581
Final Hartree-Fock Energy =              -1159.035780128072
```

```
ENERGY AND DIFF OF MACROITER  17 =  -1159.2069615533      -0.0000004670
```

8.15 VB calculation of transition state of $[\text{Cl}\cdots\text{CH}_3\cdots\text{Cl}]^-$

Input (example15_clmocl.inp):

```

#! VB(10)/6-31G* PRINTALL DIIS

TEST SN2 REACTION MECHANISM: CL- + CH3CL -> CLCH3 + CL-
THIS RUN GENERATE INITIAL GUESS FOR THE TRANSITION STATES AND REACTION

-1 1
6      0.000000    0.000000    0.000000
17     0.000000    0.000000   -2.000000
1      0.000000    1.061099    0.000000
1     -0.918939   -0.530549    0.000000
1      0.918939   -0.530549    0.000000
17     0.000000    0.000000    2.000000

$GRPDIM
17,9

$02VBORB
2->1
6->1
1->2(PI)
1-3
1-4
1-5

$02VBSTR
2
1 1 2 3 4 5 6 7 8 9
2 2 1 3 4 5 6 7 8 9

$FULLHESS

```

The transition state is symmetric, and so can be easily optimized at any theoretical level. The DIIS keyword was added to achieve convergence keeping the symmetry. The optimized $\text{Cl}\cdots\text{CH}_3$ distance at HF/6-31G* level is 2.38 Å. Since the HF method gives a very poor description for long weak bonds, we first use a much shorter $\text{Cl}\cdots\text{CH}_3$ distance (2.0 Å) for the transition state: this gives LMOs from which the electron groups can be properly assigned. The VB orbitals generated in this calculation are used as the initial guess for the final calculation of Example 16. To provide a uniform description of the electronic structure of the reaction system from reactants to products, 10 electrons and 9 orbitals are used. In the transition state, each heavy atom provides an orbital for $\text{Cl}\cdots\text{C}\cdots\text{Cl}$ bonding, and 6 orbitals for three C-H bonds. The central carbon provides a π orbital pointing to both Cl atoms. Two VB structures, which serve to describe the state of reactant and product, provide a smooth coverage of the whole reaction path.

Output:

```

OVERLAP MATRIX OF VB ORBITALS FOR GROUP    2

      1      2      3      4      5      6      7      8      9
=====
  1  1.0000
  2  0.0526  1.0000

```

```

3-0.5890 0.5891 1.0000
4-0.1877-0.1877 0.0000 1.0000
5-0.0995-0.0995 0.0000 0.8324 1.0000
6-0.1877-0.1877 0.0000 0.2356 0.1272 1.0000
7-0.0996-0.0996 0.0000 0.1272 0.0520 0.8324 1.0000
8-0.1877-0.1877 0.0000 0.2356 0.1272 0.2356 0.1272 1.0000
9-0.0996-0.0996 0.0000 0.1272 0.0520 0.1272 0.0521 0.8324 1.0000

EIGENVALUES OF OVERLAP MATRIX OF VB ORBITALS
=====
      0.1403D+00      0.1450D+00      0.1450D+00      0.1696D+00      0.8966D+00
0.1567D+01
      0.1567D+01      0.1807D+01      0.2562D+01

====Normalized structure coefficients====
0.5372 -0.5373

====Mulliken Weight (Chirgwin-Coulson)====
0.5000 0.5000

====Inverse Overlap Weight (Gallup-Norbeck)====
0.5000 0.5000

====Lowdin Weight====
0.5000 0.5000

====Hiberty Weight====
0.4999 0.5001

ENERGY AND DIFF OF MACROITER      8 =      -958.6162951007      -0.0000000900

```

8.16 Energy profile of the S_N2 reaction of Cl⁻ + CH₃Cl → ClCH₃ + Cl⁻

Input (example16_react.inp):

```

#! VB(10)/6-31G* GUESS=READ PRINTALL

TEST SN2 REACTION MECHANISM: CL- + CH3CL -> CLCH3 + CL-
TWO STRUCTURES ARE USED FOR CL-C-CL BONDS: CL-C :CL + CL: C-CL

-1 1
6          0.000000    0.000000    0.000000
17         0.000000    0.000000   -2.383018
1          0.000000    1.061099    0.000000
1         -0.918939   -0.530549    0.000000
1          0.918939   -0.530549    0.000000
17         0.000000    0.000000    2.383018

$REACTION
10,2,6
GEOM1
6          0.000000    0.000000    0.000000
17         0.000000    0.000000   -2.383018
1          0.000000    1.061099    0.000000
1         -0.918939   -0.530549    0.000000
1          0.918939   -0.530549    0.000000
17         0.000000    0.000000    2.383018
GEOM2
6          0.000000    0.000000   -0.591622
17         0.000000    0.000000   -2.419329
1          0.000000    1.020360   -0.259950
1         -0.883658   -0.510180   -0.259950
1          0.883658   -0.510180   -0.259950
17         0.000000    0.000000    2.674010

$GRPDIM
17,9

$RESTARTFILE
'example15_clmecl.v84'

$02VBSTR
2
1 1 2 3 4 5 6 7 8 9
2 2 1 3 4 5 6 7 8 9

$XYZREACTION

```

This calculation uses the previous one (Example 15) as a starting point. Since the reaction is symmetric, only half of the reaction path needs to be computed. The computed energy profile is given below:

Energy profile					
1	2	3	4	5	6
-958.66836	-958.67141	-958.67366	-958.67655	-958.67988	-958.68347
7	8	9	10	11	12
-958.68715	-958.69072	-958.69398	-958.69667	-958.69850	-958.69911

1	*****
2	*****
3	*****
4	*****
5	*****
6	*****
7	*****
8	*****
9	*****
10	***
11	
12	

The *-full.xyz file generated can be used to make an energy plot in MacMolPlt (from version 7.7.1):

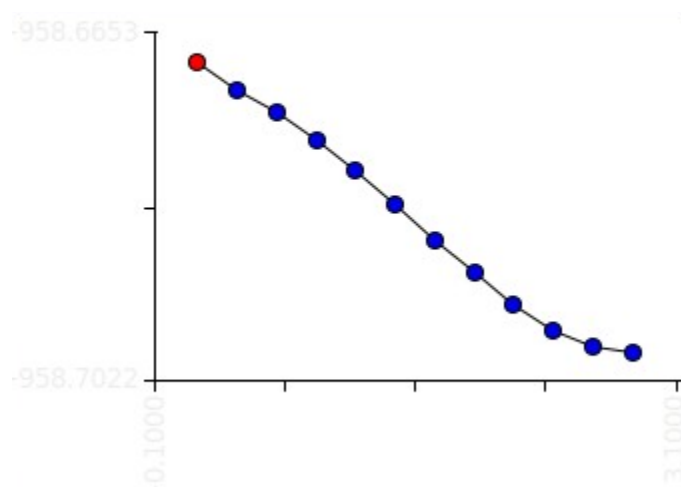


Figure 8.2. Energy plot of the reaction path calculated in Example 16.

8.17 Visualization of VB orbitals of H₂O

Input (example17_h2ocub.inp):

```
#! VB(4)/STO-3G

Water

0 1
8 0.0000000000 0.0000000000 0.0000000000
1 0.0000000000 -0.7572153434 0.5865355237
1 0.0000000000 0.7572153434 0.5865355237

$CUBE
4
4 5 6 7
Water
50

$XYZFILE
```

In this example, the electrons are described by 3 Hartree-Fock orbitals and 4 VB orbitals. The orbitals 4-7 are VB orbitals. Four cube files for the VB orbitals are created. An xyz file is also created.

Output:

```
CREATED XYZ FILE OF MOLECULAR COORDINATES - example17_h2ocub.xyz

CUBE FILE HEADERS

Water
SCF Molecular Orbitals
-3 -6.000000 -7.430930 -6.000000
50 0.244898 0.000000 0.000000
50 0.000000 0.303303 0.000000
50 0.000000 0.000000 0.267518
8 8.000000 0.000000 0.000000 0.000000
1 1.000000 0.000000 -1.430930 1.108391
1 1.000000 0.000000 1.430930 1.108391
1 4

NORMALIZATION OF ORBITAL 4 IS: 0.999436
CUBE FILE FOR ORBITAL 4 CREATED AS example17_h2ocub-orb-004.cube

NORMALIZATION OF ORBITAL 5 IS: 0.999978
CUBE FILE FOR ORBITAL 5 CREATED AS example17_h2ocub-orb-005.cube

NORMALIZATION OF ORBITAL 6 IS: 0.999436
CUBE FILE FOR ORBITAL 6 CREATED AS example17_h2ocub-orb-006.cube

NORMALIZATION OF ORBITAL 7 IS: 0.999978
CUBE FILE FOR ORBITAL 7 CREATED AS example17_h2ocub-orb-007.cube
```

See Figure 6.2 for orbitals 4 and 5.

8.18 Calculation with dynamic VB orbitals

Input (example18_h3bovb.inp):

```
#! VB(3)/D95 SPDEN DIIS PRINTALL

TEST

0 2
1      0.0  0.0  1.0
1      0.0  0.0  0.0
1      0.0  0.0 -1.0

$GRPDIM
4

$01VBORB
(1->2)
(2->1)+0.50(1->2)
(2->3)+0.50(3->2)
(3->2)

$01VBSTR
2
1 2 4
3 4 1
```

In this example, two VB structures are included in the calculation and each structure has a different set of VB orbitals. Such a treatment is quite similar to the BOVB type calculation. However, the original BOVB[17] approach has additional constraints that each orbital has to be “strictly” localized on its parent atom. In this calculation, such constraints are removed. From version 1.8.2, significant algorithm enhancement has been made for the orbital optimization. The optimization can be performed even in the case where the VB orbitals have linear dependency.

Output:

```
====Normalized structure coefficients====
-0.5743  0.5755

====Mulliken Weight (Chirgwin-Coulson)====
0.4993  0.5007

====Inverse Overlap Weight (Gallup-Norbeck)====
0.4983  0.5017

====Lowdin Weight====
0.4988  0.5012

====Hiberty Weight====
0.4990  0.5010

SPIN DENSITY ON ATOMS
-----
ATOM      DENSITY
1         0.62754
```

```

2      -0.25431
3      0.62677

```

LINEAR DEPENDENT VECTOR IDENTIFIED

ENERGY AND DIFF OF MACROITER 6 = -1.6140827445 -0.0000000060

ORBITALS OF EACH ELECTRON GROUP IN AO BASIS: ORBITAL(GROUP)

```

=====
Orbital#           1           2           3           4
Group#            1           1           1           1
=====

```

AO# LABELS

1	1	H	s	0.43524	0.14279	-0.02513	0.00914
2	1	H	s	0.60042	0.11779	-0.16658	-0.12176
3	2	H	s	0.11261	0.42132	0.42155	0.11275
4	2	H	s	0.03830	0.60985	0.61045	0.03839
5	3	H	s	0.00815	-0.02482	0.14249	0.43530
6	3	H	s	-0.12317	-0.16587	0.11759	0.59996

8.19 VB calculation with strictly localized orbitals

Input (example19_ohloc.inp):

```
#!/ VB(9)/D95 DIIS PRINTALL

TEST H 0

0 2
8      .000000 .000000 .000000
1      1.00000 .000000 .000000

$GRPDIM
12

$AOGROUP
12
1,2,3,4,5,6,7,8,9,10,11,12
1,1,1,1,1,1,1,1,1,1,1,1

$01VBSTR
3
1 1 3 3 5 11 6 6 7
1 1 3 3 5 5 6 6 7
1 1 3 3 11 11 6 6 7

$BRILLMASK
20
1,11
1,12
3,11
3,12
5,11
5,12
6,11
6,12
7,11
7,12
11,1
11,2
11,3
11,4
11,5
11,6
11,7
11,8
11,9
11,10

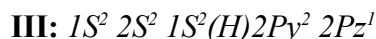
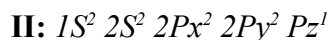
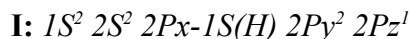
$01VBSCF
0.00001, 30
```

In this example, the two atoms of OH are put on x-axis. Such an orientation makes it more convenient to divide the atomic orbitals into different groups. The 12 atomic orbitals of OH in D95 basis set are shown as follows:

O: $1S$, $1S'$, $2S$, $2S'$, $2P_x$, $2P_y$, $2P_z$, $2P_x'$, $2P_y'$, $2P_z'$ (basis functions 1-10)

H: $1S$, $1S'$ (basis functions 11,12)

Three structures, which corresponds to the following VB structures, are included in the calculation:



The subscript for each AO specifies the number electrons on the AO. 1S(H) is the 1S orbital of the hydrogen atom. All other orbitals are from O. 2P_x-1S(H) specifies the O-H bond formed by 2P_x(O) and 1S(H). According to this convention, the first structure corresponds to a covalent OH bond with a negative charge on O. The second structure corresponds to a ionic structure with two negative charges on O, and the third one corresponds to a ionic structure with a negative charge on H. The initial VB orbitals are from atomic orbitals as shown in the above VB structures (the AO on H are explicitly specified in parenthesis), therefore they are all strictly localized on their corresponding atoms.

In a general VB calculation, each VB orbital is optimized within the whole basis function space, therefore the final resulting VB orbitals are somewhat delocalized to neighbor atoms. In this case, we want to preserve the strict localization while still allow each VB orbital to be optimized within the atomic orbitals of their corresponding atoms. This is controlled by \$BRILLMASK (Brillouin Mask). In this case, there are 20 lines of controls, each line specify which orbital will NOT be mixed with which AO. For instance, the first line is

1,11

which means the VB orbital *I* will NOT be mixed with AO *II*, which is 1S(H), during orbital optimization.

The final optimized VB orbitals are strictly localized on their corresponding atoms as shown below:

Output:

```
====Normalized structure coefficients====
 0.6600  0.4028  0.0522

====Mulliken Weight (Chirgwin-Coulson)====
 0.6297  0.3397  0.0306

====Inverse Overlap Weight (Gallup-Norbeck)====
 0.4130  0.5759  0.0111

====Lowdin Weight====
 0.3600  0.5174  0.1226

====Hiberty Weight====
 0.7253  0.2702  0.0045

ENERGY AND DIFF OF MACROITER      6 =      -75.4052392790      -0.0000000000
```

ORBITALS OF EACH ELECTRON GROUP IN AO BASIS: ORBITAL(GROUP)

Orbital#				1	2	3	4	5	6
Group#				1	1	1	1	1	1

AO#	LABELS								
1	1	0	s	0.58446	-0.00000	0.29138	-0.00000	0.00093	0.00000
2	1	0	s	0.46613	1.00000	0.16945	-0.00000	-0.00117	0.00000
3	1	0	s	-0.01491	-0.00000	0.42252	-0.00000	0.01511	0.00000
4	1	0	s	-0.01146	-0.00000	0.37762	1.00000	-0.14265	-0.00000
5	1	0	x	0.00521	0.00000	-0.11690	0.00000	0.80770	0.00000
6	1	0	y	0.00000	0.00000	0.00000	0.00000	-0.00000	0.74990
7	1	0	z	-0.00000	0.00000	0.00000	0.00000	-0.00000	-0.00000
8	1	0	x	0.00210	-0.00000	-0.07807	-0.00000	0.29658	-0.00000
9	1	0	y	0.00000	-0.00000	-0.00000	-0.00000	0.00000	0.38320
10	1	0	z	-0.00000	0.00000	0.00000	0.00000	-0.00000	-0.00000
11	2	H	s	-0.00000	-0.00000	0.00000	-0.00000	-0.00000	0.00000
12	2	H	s	-0.00000	-0.00000	0.00000	0.00000	-0.00000	0.00000
=====									
Orbital#				7	8	9	10	11	12
Group#				1	1	1	1	1	1

AO#	LABELS								
1	1	0	s	0.00000	0.00000	-0.00000	0.00000	-0.00000	-0.00000
2	1	0	s	0.00000	0.00000	-0.00000	0.00000	0.00000	0.00000
3	1	0	s	0.00000	0.00000	0.00000	-0.00000	-0.00000	-0.00000
4	1	0	s	-0.00000	0.00000	0.00000	0.00000	0.00000	0.00000
5	1	0	x	0.00000	-0.00000	-0.00000	0.00000	0.00000	0.00000
6	1	0	y	0.00000	-0.00000	-0.00000	-0.00000	0.00000	0.00000
7	1	0	z	0.78284	-0.00000	-0.00000	0.00000	0.00000	0.00000
8	1	0	x	-0.00000	1.00000	-0.00000	-0.00000	-0.00000	0.00000
9	1	0	y	-0.00000	-0.00000	1.00000	0.00000	0.00000	0.00000
10	1	0	z	0.34149	0.00000	-0.00000	1.00000	0.00000	0.00000
11	2	H	s	-0.00000	-0.00000	0.00000	0.00000	0.74993	0.00000
12	2	H	s	-0.00000	-0.00000	0.00000	-0.00000	0.32441	1.00000

8.20 VB calculation of structure weights with localization enhanced VB orbitals

Input (example20_h2oloc.inp):

```
#!/ VB(4)/cc-pVDZ printall

TEST H2O

0 1
8 .000000 .000000 .000000
1 .801842 .000000 .555582
1 -.801842 .000000 .555582

$02VBORB
1-2
1-3

$DPWEIGHT
0.1

$02LENHANCE
4
1,1,1
2,1,2
3,1,1
4,1,3

$02VBSTR
5
1 2 3 4
1 1 3 4
3 3 1 2
2 2 3 4
4 4 1 2

$MACROITER
30

$CUBE
4
4 5 6 7
water local
64

$PLTORB
2 2 1 3 1
4 4 5 6 7
3ATOMS
1 3 2
0.0
-2.0 2.0 -2.0 2.0
Water VBO 4
Water VBO 5
Water VBO 6
Water VBO 7
```

In this case, 5 structures are included for 4-electron VB calculation of the two OH bonds in H₂O. We want to calculate the structure weights for the covalent structure and the four ionic structures. The delocalization of VB orbitals lead to some ambiguity for the distinction between covalent structures and ionic structures. More over, there is also a mathematical problem for computing the weights of ionic structures: since the VB orbitals are optimized, the formal contribution of singly excited ionic structures to the fully optimized VBSCF wave function can be zero as shown by Brillouin theorem. Therefore, we need a more convenient way to obtain optimized VB orbitals with localization enhancement. This will not only solve the mathematical problem for the computation of ionic structure weights but also reduce the ambiguity between covalent and ionic structures in modern VB descriptions of a molecular system. The strict localization method in example 19 is cumbersome to use and is difficult to be applied for more general cases. The localization enhancement control provides a flexible and convenient method for the balance of total wave function energy and localization.

The localization enhancement control is specified by the key word \$02LENHANCE. By default, the 4-electron VB group is group 2, therefore the LENHANCE flag has a number prefix \$02. The 4 VB orbitals as specified by \$02VBORB are

- (1) orbital on O pointing to the first H
- (2) orbital on the first H
- (3) orbital on O pointing to the second H
- (4) the orbital on the second H

In this example, each VB orbital is localized on one parent atom, i.e. the four VB orbitals 1-4 are localized on atom 1, 2, 1 and 3 respectively.

The delocalization quantities and structure weights from the optimized VB orbitals are shown as follows.

Output:

```
DELOCALIZATION FOR VB ORBITALS OF GROUP 2
=====
VBO INDEX = 1 DELOCALIZATION = 0.00614
VBO INDEX = 2 DELOCALIZATION = 0.00963
VBO INDEX = 3 DELOCALIZATION = 0.00614
VBO INDEX = 4 DELOCALIZATION = 0.00963

OVERLAP MATRIX OF VB ORBITALS FOR GROUP 2
      1      2      3      4
=====
1 1.0000
2 0.6942 1.0000
3 0.2581 0.1753 1.0000
4 0.1753 0.1495 0.6942 1.0000

====Normalized structure coefficients====
-0.4476 -0.3171 -0.3171 -0.0068 -0.0068

====Mulliken Weight (Chirgwin-Coulson)====
```

0.4315 0.2794 0.2794 0.0049 0.0049

====Inverse Overlap Weight (Gallup-Norbeck)====

0.1227 0.4384 0.4384 0.0002 0.0002

====Lowdin Weight====

0.1933 0.3060 0.3060 0.0974 0.0974

====Hiberty Weight====

0.4990 0.2504 0.2504 0.0001 0.0001

ENERGY AND DIFF OF MACROITER 13 = -76.0663293310 0.0000105779

ORBITALS OF EACH ELECTRON GROUP IN AO BASIS: ORBITAL(GROUP)

=====									
Orbital#				1	2	3	4	5	6
Group#				1	1	1	2	2	2

AO# LABELS									
1	1	0	s	0.89611	0.44588	-0.00001	0.04918	-0.04583	0.04918
2	1	0	s	-0.16194	0.30912	-0.00000	0.37115	-0.01448	0.37115
3	1	0	s	-0.22603	0.43593	-0.00000	0.22578	-0.04721	0.22578
4	1	0	x	-0.00000	-0.00001	0.00000	0.45552	-0.00563	-0.45551
5	1	0	y	-0.00000	-0.00001	-0.62990	-0.00000	-0.00000	-0.00000
6	1	0	z	0.19225	-0.39443	0.00000	0.38754	0.02189	0.38755
7	1	0	x	-0.00000	-0.00000	0.00000	0.18798	0.02332	-0.18798
8	1	0	y	-0.00000	-0.00001	-0.49904	-0.00000	0.00000	-0.00000
9	1	0	z	0.14648	-0.30022	0.00000	0.19632	0.03575	0.19633
10	1	0	xx	-0.00130	-0.00030	0.00000	0.01068	-0.00382	0.01068
11	1	0	yy	-0.00344	0.00430	-0.00000	-0.02441	0.02822	-0.02441
12	1	0	zz	0.00698	-0.01724	0.00000	0.00747	0.01468	0.00747
13	1	0	xy	-0.00000	-0.00000	-0.00000	0.00000	-0.00000	-0.00000
14	1	0	xz	-0.00000	-0.00000	0.00000	0.04168	-0.02585	-0.04168
15	1	0	yz	-0.00000	-0.00000	-0.01729	0.00000	-0.00000	0.00000
16	2	H	s	0.02942	-0.07214	0.00000	0.07438	0.87492	-0.00901
17	2	H	s	0.01487	-0.03063	0.00000	0.00117	0.17048	-0.00461
18	2	H	x	-0.00128	0.00555	-0.00000	-0.01054	-0.09306	-0.01358
19	2	H	y	-0.00000	-0.00000	-0.02973	0.00000	0.00000	-0.00000
20	2	H	z	0.00823	-0.01523	0.00000	-0.00396	-0.04613	0.00251
21	3	H	s	0.02942	-0.07214	0.00000	-0.00901	0.01617	0.07438
22	3	H	s	0.01487	-0.03063	0.00000	-0.00461	-0.04282	0.00117
23	3	H	x	0.00128	-0.00555	0.00000	0.01358	0.02331	0.01054
24	3	H	y	-0.00000	-0.00000	-0.02973	-0.00000	0.00000	0.00000
25	3	H	z	0.00823	-0.01523	0.00000	0.00251	0.02102	-0.00396

=====				
Orbital#		7		
Group#		2		

A0#	LABELS			
1	1	0	s	-0.04584
2	1	0	s	-0.01448
3	1	0	s	-0.04721
4	1	0	x	0.00563
5	1	0	y	-0.00000
6	1	0	z	0.02189
7	1	0	x	-0.02332
8	1	0	y	0.00000
9	1	0	z	0.03575

```

10 1 0 xx -0.00382
11 1 0 yy 0.02822
12 1 0 zz 0.01468
13 1 0 xy 0.00000
14 1 0 xz 0.02586
15 1 0 yz -0.00000
16 2 H s 0.01618
17 2 H s -0.04282
18 2 H x -0.02331
19 2 H y 0.00000
20 2 H z 0.02102
21 3 H s 0.87492
22 3 H s 0.17048
23 3 H x 0.09306
24 3 H y 0.00000
25 3 H z -0.04613

```

The structure weights of the covalent structure and the four ionic structures are shown as follows:

Table 8.1. Structure weights from localization enhanced VB calculation.

Structure (type)	VB structure	Chirgwin-Coulson Weight
I (covalent)	H–O–H	0.432
II (ionic)	H–O [−] H ⁺	0.279
III (ionic)	H ⁺ O [−] –H	0.279
IV (ionic)	H–O ⁺ H [−]	0.005
V (ionic)	H [−] O ⁺ –H	0.005

The delocalization of each VB orbital is below 0.01 as shown in the following table:

Table 8.2. Delocalization of VB orbitals in localization enhanced calculation.

VB orbital	Location	Delocalization
1	O(1)	0.0061
2	H(2)	0.0096
3	O(1)	0.0061
4	H(3)	0.0096

In a VB structure calculation with only one covalent structure, the resulting VB orbital on hydrogens are considerably delocalized with a delocalization coefficient 0.16. The delocalization is reduced by a factor of 16 with the localization enhanced optimization. The VB orbitals of localization enhanced optimization and the regular optimization are shown as follows:

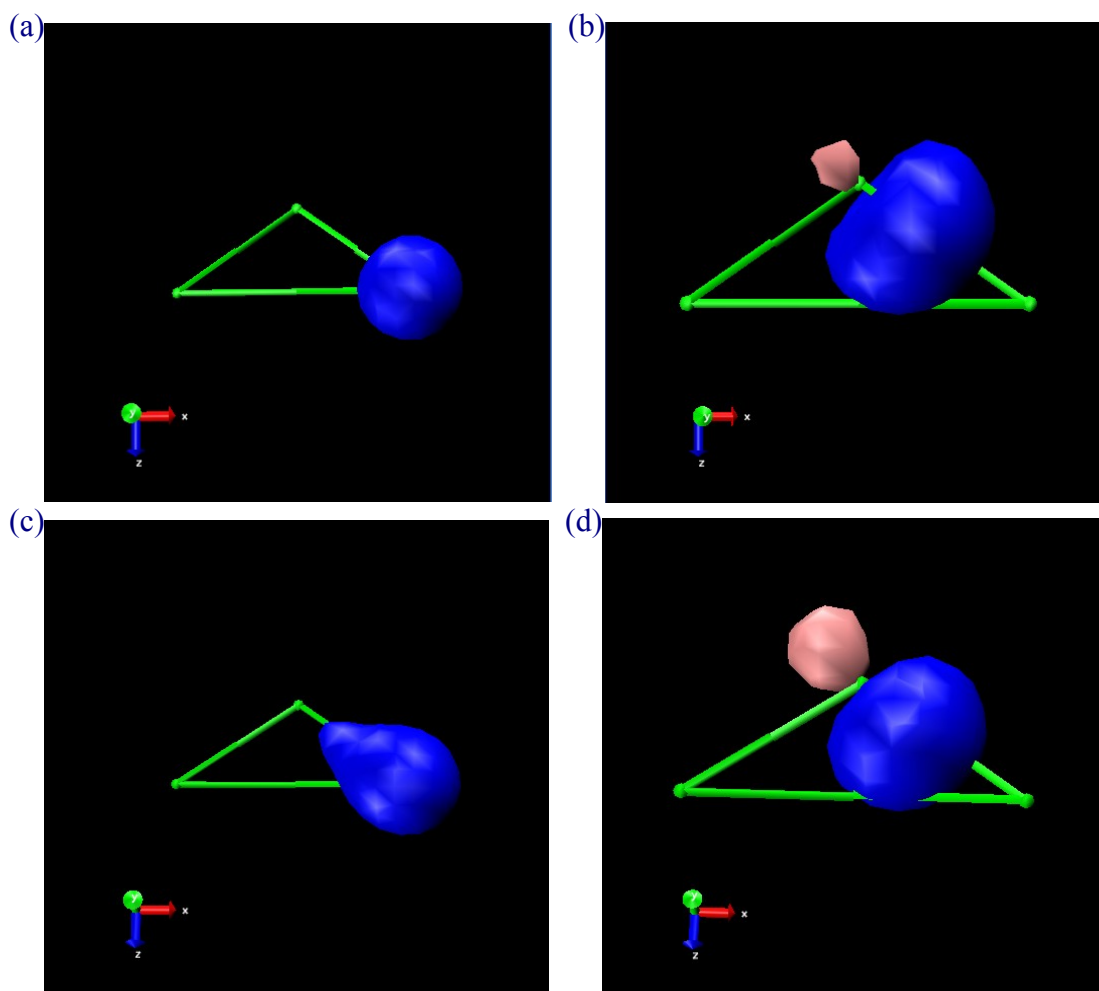


Figure 8.1. Optimized VB orbitals from localization enhanced calculation (a, b) and VB orbitals from a normal single covalent VB calculation (c, d) for H_2O .

8.21 Benzene with ionicity (0,2) / seniority (6,2)

Input (example21_c6h6sn.inp):

```
#! SEN(6,6)/STO-6G PRINTALL UNITS=BOHR

Benzene STO-6G test - MP2 geom

0 1
6   1.3211947495      2.2883764328 0.0000000000
6  -1.3211947495      2.2883764328 0.0000000000
6  -2.6423894990      0.0000000000 0.0000000000
6  -1.3211947495     -2.2883764328 0.0000000000
6   1.3211947495     -2.2883764328 0.0000000000
6   2.6423894990      0.0000000000 0.0000000000
1   2.3437065580      4.0594188365 0.0000000000
1  -2.3437065580      4.0594188365 0.0000000000
1  -4.6874131160      0.0000000000 0.0000000000
1  -2.3437065580     -4.0594188365 0.0000000000
1   2.3437065580     -4.0594188365 0.0000000000
1   4.6874131160      0.0000000000 0.0000000000

$MEMORY
20000000

$CMAXCUT
0.0

$NOTROT
1
1 2

$02IONICITY
2
0 2

02SENIORITY
2
6 2

$MACROITER
20

$ECONV
7

$VBSCF
0.0001, 55

$AOGROUP
6
5, 10, 15, 20, 25, 30
2 2 2 2 2 2
```

This is an example of a calculation for a closed shell case with the structures selected by ionicity or seniority number. It is for benzene with STO-6G and structures selected from ionicity 0 and 2, which is equivalent to seniority numbers 6 and 2. This leads to 90 structures of ionicity 2 (seniority 2) added to the 5 spin coupled structures of ionicity 0

(seniority 6). Adding these 90 structures gets a result closer to the full CASVB than adding any other single ionicity (seniority) set of structures to the 5 SC structures.

Output:

```
Reading $IONICITY data
No. of Structure Groups      2
Ionicity Groups              0  2
Seniority Nos                6  2
```

WRITING STRUCTURES FILE example21_c6h6sn.loc

SYMBOLIC VB STRUCTURE(S) OF GROUP 2

STR. NO.	RUMER PATTERN
1	1 2 3 4 5 6
2	1 3 2 4 5 6
3	1 4 2 5 3 6
4	1 2 3 5 4 6
5	1 3 2 5 4 6
6	1 1 2 2 3 4
7	1 1 2 2 4 5
8	1 1 2 2 5 6
9	1 1 2 2 3 5
10	1 1 2 2 3 6
11	1 1 2 2 4 6
12	2 2 3 3 4 5
13	2 2 3 3 5 6
14	2 2 3 3 6 1
15	2 2 3 3 4 6
16	2 2 3 3 4 1
17	2 2 3 3 5 1
18	3 3 4 4 5 6
19	3 3 4 4 6 1
20	3 3 4 4 1 2
21	3 3 4 4 5 1
22	3 3 4 4 5 2
23	3 3 4 4 6 2
24	4 4 5 5 6 1
25	4 4 5 5 1 2
26	4 4 5 5 2 3
27	4 4 5 5 6 2
28	4 4 5 5 6 3
29	4 4 5 5 1 3
30	5 5 6 6 1 2
31	5 5 6 6 2 3
32	5 5 6 6 3 4
33	5 5 6 6 1 3
34	5 5 6 6 1 4
35	5 5 6 6 2 4
36	1 1 3 3 4 5
37	1 1 3 3 5 6
38	1 1 3 3 6 2
39	1 1 3 3 4 6
40	1 1 3 3 4 2
41	1 1 3 3 5 2
42	1 1 4 4 5 6
43	1 1 4 4 6 2
44	1 1 4 4 2 3
45	1 1 4 4 5 2

46	1	1	4	4	5	3
47	1	1	4	4	6	3
48	1	1	5	5	6	2
49	1	1	5	5	2	3
50	1	1	5	5	3	4
51	1	1	5	5	6	3
52	1	1	5	5	6	4
53	1	1	5	5	2	4
54	1	1	6	6	2	3
55	1	1	6	6	3	4
56	1	1	6	6	4	5
57	1	1	6	6	2	4
58	1	1	6	6	2	5
59	1	1	6	6	3	5
60	2	2	4	4	5	6
61	2	2	4	4	6	1
62	2	2	4	4	1	3
63	2	2	4	4	5	1
64	2	2	4	4	5	3
65	2	2	4	4	6	3
66	2	2	5	5	6	1
67	2	2	5	5	1	3
68	2	2	5	5	3	4
69	2	2	5	5	6	3
70	2	2	5	5	6	4
71	2	2	5	5	1	4
72	2	2	6	6	1	3
73	2	2	6	6	3	4
74	2	2	6	6	4	5
75	2	2	6	6	1	4
76	2	2	6	6	1	5
77	2	2	6	6	3	5
78	3	3	5	5	6	1
79	3	3	5	5	1	2
80	3	3	5	5	2	4
81	3	3	5	5	6	2
82	3	3	5	5	6	4
83	3	3	5	5	1	4
84	3	3	6	6	1	2
85	3	3	6	6	2	4
86	3	3	6	6	4	5
87	3	3	6	6	1	4
88	3	3	6	6	1	5
89	3	3	6	6	2	5
90	4	4	6	6	1	2
91	4	4	6	6	2	3
92	4	4	6	6	3	5
93	4	4	6	6	1	3
94	4	4	6	6	1	5
95	4	4	6	6	2	5
.						
ENERGY AND DIFF OF MACROITER				4 =	-230.2376633398	-0.0000000003

This energy is 99.23% of the RHF → CASVB difference, while SC is 93.54%. Adding ionicity 1 (seniority 4) to SC leads to 99.70%, but symmetry is broken. Adding ionicity 3 (seniority 0) to SC leads to 96.27%.

8.22 SCGVB(6,5) calculation of the cyclopentadienyl anion

Input (example22_SCnm.inp):

```

#! SC(6,5)/6-31+G* DIIS

Cyclopentadienyl anion pi system SC(6,5)

-1 1
6 0.3720 -1.1448 0.0000
6 1.2037 0.0000 0.0000
6 0.3720 1.1448 0.0000
6 -0.9738 0.7075 0.0000
6 -0.9738 -0.7075 0.0000
1 0.7068 -2.1752 0.0000
1 2.2871 0.0000 0.0000
1 0.7068 2.1752 0.0000
1 -1.8503 1.3443 0.0000
1 -1.8503 -1.3443 0.0000

$MEMORY
18000000

$02PIVB0
5
1,2,3,4,5

$NOTROT
1
1,2

```

This calculation is based on Reference 7, which introduces the SCGVB method for N electrons in M orbitals. The SCGVB(6,5) wave function is much more compact than the corresponding CASVB(6,5) and recovers most of the CAS energy. The DIIS keyword has shown to be necessary to obtain symmetric final orbitals, although using \$02LENHANCE with a low \$DPWEIGHT (0.001) produces almost the same result without DIIS. The initial geometry was obtained by a previous CASSCF(6,5)/aug-cc-pVDZ geometry optimization.

Output:

```

Final Hartree-Fock Energy = -192.208209181433

... ..

SYMBOLIC VB STRUCTURE(S) OF GROUP 2
STR. NO. RUMER PATTERN
 1 1 1 2 3 4 5
 2 1 1 2 4 3 5
 3 2 2 3 4 5 1
 4 2 2 3 5 4 1
 5 3 3 4 5 1 2
 6 3 3 4 1 5 2
 7 4 4 5 1 2 3
 8 4 4 5 2 1 3
 9 5 5 1 2 3 4
10 5 5 1 3 2 4

```

```

... ..
ENERGY AND DIFF OF MACROITER      1 =   -192.1425340749   -192.1425340749
ENERGY AND DIFF OF MACROITER      2 =   -192.2444020148    -0.1018679399
ENERGY AND DIFF OF MACROITER      3 =   -192.2497496311    -0.0053476163
ENERGY AND DIFF OF MACROITER      4 =   -192.2499816543    -0.0002320232
ENERGY AND DIFF OF MACROITER      5 =   -192.2499923652    -0.0000107108
ENERGY AND DIFF OF MACROITER      6 =   -192.2499928358    -0.0000004706

OVERLAP MATRIX OF VB ORBITALS FOR GROUP    2

      1      2      3      4      5
=====
  1  1.0000
  2  0.6764  1.0000
  3  0.4835  0.6768  1.0000
  4  0.4826  0.4844  0.6775  1.0000
  5  0.6757  0.4826  0.4841  0.6765  1.0000

====Normalized structure coefficients====
  0.2978  0.0399  0.2976  0.0397  0.2973  0.0394  0.2970  0.0395  0.2983
0.0399

====Mulliken Weight (Chirgwin-Coulson)====
  0.2056 -0.0053  0.2052 -0.0052  0.2047 -0.0052  0.2046 -0.0052  0.2059 -
0.0053

====Inverse Overlap Weight (Gallup-Norbeck)====
  0.2001  0.0008  0.1993  0.0008  0.1982  0.0007  0.1978  0.0007  0.2009
0.0008

====Lowdin Weight====
  0.2003  0.0008  0.1992  0.0008  0.1980  0.0008  0.1980  0.0007  0.2007
0.0008

====Hiberty Weight====
  0.1968  0.0035  0.1965  0.0035  0.1961  0.0034  0.1957  0.0035  0.1974
0.0035

ENERGY AND DIFF OF MACROITER      7 =   -192.2499928796    -0.0000000438

```

Notice that the overlaps between neighbouring orbitals are very consistent (~ 0.677) and so also are the structure weights (about 0.20 for the corresponding classical VB structures and almost zero for the other structures). The energy difference between HF and SCGVB(6,5) is 26.2 kcal/mol. If the calculation is repeated replacing SC(6,5) by CASVB(6,5), the new energy is:

```

ENERGY AND DIFF OF MACROITER      8 =   -192.2510578609    -0.0000005293

```

which gives a difference of 26.9 kcal/mol with the HF energy. Thus, SCGVB(6,5) recovers 97.4% of the CAS correlation energy.

8.23 Multiconfigurational SCGVB calculation of allyl cation

Input (example23_mcsc.inp):

```
#! VB(2).VB(2).VB(2,4)/cc-pVDZ FROZEN DIIS
ALLYL CATION (2 CC BONDS AND 2 PI ELECTRONS)

1 1
6 0.0003660801 0.4928856632 -0.0000000000
1 0.0025391625 1.5748362291 -0.0000000000
6 1.1872363195 -0.2122069099 -0.0000000000
6 -1.1893268513 -0.2074337121 -0.0000000000
1 -2.1454624741 0.3034330089 0.0000000000
1 -1.2038562519 -1.2915236890 -0.0000000000
1 1.1974110057 -1.2963465051 0.0000000000
1 2.1454163692 0.2948149050 0.0000000000

$GRPDIM
8,2,2,4

$VBGA
1-3(1) => 2
1-4(1) => 3

$04VBSTR
2
1 2
3 4

$04VBORB
0.32 A0 6 +0.22 A0 9 +0.49 A0 26 +0.31 A0 29
0.32 A0 6 +0.22 A0 9 +0.49 A0 41 +0.31 A0 44
0.71 A0 6 +0.30 A0 9
0.16 A0 9 +0.37 A0 26 +0.30 A0 29 +0.37 A0 41 +0.30 A0 44

READGUESS
example23_mcsc.orb

$PRINTHS

$NOTROT
3
1,2
1,3
1,4

$MACROITER
20
```

This example demonstrates a multiconfiguration spin-coupled VB approach for the pi system of allyl cation in conjunction with perfect-pairing GVB for the two C-C sigma bonds. This is based on the reference 30, but was slightly modified for simplicity. The original calculation was performed on GAMESS / VB2000 using the cc-pVTZ basis set. We have ported the calculation to stand alone VB2000 by using the cc-pVDZ basis set. The calculation reads an initial guess file example23_mcsc.orb, which was generated by following the very complicated procedure described in the paper, which includes a

previous CASSCF calculation, construction of the initial orbitals from the CASSCF result and pre-optimization of SC(2) structures separately. The final result is equivalent to the reported on ref. 30.

Output:

```

SYMBOLIC VB STRUCTURE(S) OF GROUP 4
STR. NO.      RUMER PATTERN
  1           1 2
  2           3 4

... ..

ENERGY AND DIFF OF MACROITER   1 =   -116.2543636696   -116.2543636696
ENERGY AND DIFF OF MACROITER   2 =   -116.2544154511     -0.0000517815
ENERGY AND DIFF OF MACROITER   3 =   -116.2544163723     -0.0000009213

... ..

      1      2      3      4
=====
  1 1.0000
  2 0.4667 1.0000
  3 0.7620 0.7630 1.0000
  4 0.7891 0.7896 0.6444 1.0000

OVERLAP MATRIX OF VB STRUCTURES FOR GROUP 4

      1      2
=====
  1 1.000000
  2 0.916977 1.000000

... ..

====Normalized structure coefficients====
 0.5690 0.4521

====Mulliken Weight (Chirgwin-Coulson)====
 0.5597 0.4403

====Inverse Overlap Weight (Gallup-Norbeck)====
 0.5398 0.4602

====Lowdin Weight====
 0.4592 0.5408

====Hiberty Weight====
 0.6130 0.3870

ENERGY AND DIFF OF MACROITER   4 =   -116.2544164861   -0.0000001137

```

8.24 Other included input files

The directory `TESTINP` contains the 23 examples discussed above, plus 15 test jobs. Tests 01-13 are for testing the built-in basis sets. Test 14 is for testing the use of an external basis set file. Tests 15 is for testing a system with 13 electrons. More tests will be added as VB2000 is extended to run larger systems by default.

The directory `TESTINP/extra` contains more 43 input files, some of which are used as examples throughout this manual. Most of the extra files were old test jobs of the program.

The directory `molten` contains 7 input files which test the `$READMOLDEN` directive for reading input (geometry, basis set, Mos) from `*.molten` files generated from different quantum chemistry programs. There are also 7 input files which are the same but the `$READMOLDEN`, for comparison.

9 Acknowledgments

VB2000 is based on the extensive re-engineering of two independent programs AMOY-VB and VB98 (both were not publicly distributed). The former one was originated from the early investigation on group theoretical approach to VB calculations during one of the authors (JL)'s stay at Xiamen University (1990 -1992). He would like to express his thanks to Professor Qianer Zhang for teaching him the spin-free approach to VB theory in early 1990s. He is also very thankful to his former colleagues, Dr. Wei Wu and Dr. Yirong Mo at Xiamen University, for the early development of permanent method and for VB calculations (1991-1992). Since the early days of the program, many other people have been contributed to the development of VB2000 in one way or another. JL is very grateful to Professor Ruben Pauncz for the very inspiring collaboration on developing the algebrant algorithm (1995-1997), which is one of the mathematical foundations of VB2000. Sincere thanks are due to Dr. David L. Cooper for introducing him to the use of the Pipek-Mezey (PM) localization method for electron group partitioning (1995), which is the technical foundation of VB2000. He also thanks for Professor Keiji Morokuma for his hospitality during the author's 6-month stay at Institute of Molecule Science, Okazaki, Japan, 1992-1993. During this period, Professor Morokuma offered complete freedom for doing research on VB algorithm development. JL is grateful to the Alexander von Humboldt Foundation for the award of a fellowship to make his 2-year stay in Germany possible. He is also very grateful to Professor Janos Ladik for providing a position as an AvH Research Fellow at the Institute of Physical and Theoretical Chemistry, University of Erlangen-Nürnberg, Erlangen, Germany, 1993-1995. Thanks are also due to Professor Donald G. Truhlar for reading the manual of the VB program during his stay in the Truhlar-Cramer group (1996-1999).

Since the first release of VB2000 in the new Millennium, a lot of feedbacks have been received from many enthusiastic users. Their excellent comments are a strong motivation for us to develop an even more efficient, more flexible and easy to use VB program. In particular, we would like to thank Professor Thomas M. Klapötke of University of Munich for his creativity in using VB2000: he has provided us with many examples which challenge our program. JL thanks Drs. Heribert Reis, Aggelos Avramopoulos and Professor Manthos G. Papadopoulos for many years collaboration on applying VB2000 to novel materials. We thank Saryu Jindal, Felipe Fleming and Nuno A. G. Bandeira for their feedbacks on the use of the program.

We thank the GAMESS(US) team and in particular Dr Mike Schmidt for many helpful discussions by email and in person about introducing VB2000 into GAMESS(US). His work in ensuring that it worked with all the machines that he tests GAMESS on and not just the machines we developed VB2000 on has been particularly useful.

We thank Dr Rika Kobayashi at the National Computational Infrastructure (NCI) hosted at the Australian National University for compiling the old Gaussian03/VB2000 and Gaussian09/VB2000 versions, and, with particular thanks, for her help and advice with the code changes between Gaussian03 and Gaussian09. The new version of VB2000 using input from Gaussian would not have been possible without the strong and helpful

collaboration with Mike Frisch from Gaussian Inc, and assistance from Philip Chan, Campus Cluster Administrator at Monash University, along with access to the Monash Cluster.

We thank Michelle Styles and Harry Quiney, both of the University of Melbourne, for the initial Windows .NET GUI and the first compilation of VB2000 for Windows respectively (with version 1.6). One of the authors (BD) thanks Professor Jack Linnett for introducing him to Valence Bond theory over 50 years ago and to Dick Harcourt of the University of Melbourne for many recent stimulating discussions.

One of the authors (RMcW) wishes to record his indebtedness to Wei Wu for introducing him to the algorithms under development at Xiamen University, during a fruitful collaboration at the University of Pisa (1993-94).

We thank Brett Bode, developer of MacMolPlt, which collaborated implementing the importing of \$VEC blocks, which made possible visualizing VB orbitals in MacMolPlt by using the directives \$VECINIT, \$VECMOS, \$VECLMO and \$VECONLY. Also he collaborated implementing the reading of energy values in the label line of *.xyz files, which made possible building energy plots of reaction paths in MacMolPlt.

We are grateful to Professor André Gustavo Barbosa from Universidade Federal Fluminense (Brazil) for sharing his work (example23) with the VB2000 community.

From version 3.0, the new members of VB2000 team (David Sousa and Rodrigo Bitzer) thank the authors Brian Duke and Jiabo li for the opportunity to contribute to their great work.

Finally, the VB2000 team express our enormous gratitude to Roy McWeeny, who always have been the spiritual symbol for VB2000. Unfortunately, our dear friend Roy passed away on April 29th, 2021.



10. Appendix I - Limits on parameters

10.1 Limitations on molecule size

Limitations on the size of molecules that can be studied are implemented by maximum values for several parameters.

Stand-alone version.

Four limits are imposed in the integral routines in vb2000ints.src:

MAXCEN = 200. The maximum number of atoms allowed.

MAXSHL = 500. The maximum total number of shells (S, P, D) allowed in the molecule.

MAXBFN = 1350. The maximum total number of basis functions in the molecule.

MAXEXP = 4400. The maximum total number of primitive Gaussian functions used to construct the basis functions.

The code in vb2000ints.src is very old and difficult to alter. These values should not be changed. If they are insufficient move to using the GAMESS / VB2000 version.

The value of MAXEXP is rather large. The ratio of MAXEXP to MAXBFN is just over 3, so it would be possible to use 726 basis functions which are all STO-3G. Modern larger basis sets have some functions with many primitive orbitals, but also many that only use one primitive orbital. The average value of that ratio is usually around 2 (e.g. 2.33 and 1.4 for cc-pVDZ on C and H atoms respectively and 1.36 and 1.08 for aug-cc-pVTZ on C and H.). Only MAXCEN can be checked in the vb2000.src routines before entering the integral package. You will get a proper error message if MAXCEN is exceeded and the program will finish. In the other cases the program will end with rather odd error messages, but there will be a proper error message in the *.output file that documents the integral package workings.

GAMESS version.

MAXCEN, MAXSHL and MAXBFN are set to the same values as in the stand-alone version, but they can be easily changed (see below).

MAXSHL is only used in the visualisation routines. The number of shells allowed for the molecular integrals is of course set in GAMESS.

Several other maximum values are used:

MAXPRM = 15. The maximum number of primitive Gaussian functions in a basis function.

MAXNAB is the maximum value of the product of the number of atoms and the number of basis functions. Since large basis sets are likely to be used for small molecules and smaller basis sets for larger molecules, this is set to a smaller value than the product of the maximum number of atoms and basis functions and is $\text{MAXCEN} * \text{MAXBFN} / 2$.

Modifying these values:

If one of these values is exceeded, the program will give you an extensive report on what to do, indicating that you need make a global edit of a text such as /MAXCEN=200/. This is done in the vi editor by typing, for example:

```
:%s/MAXCEN=200/MAXCEN=300/
```

MAXNAB can be changed by a global edit of /MAXNAB=MAXCEN*MAXBFN/2/.

Make sure you make the changes in all program module files, such as vb2000.src, vb2000-so.src, vb2gms.src, where it is present. After editing the program, you will need to recompile.

10.2 Limitations of number of electrons

Three parameters in the program are the most important with relation to the maximum number of electrons which can be putted in the same group: MAXNODE, NW1 and MM.

MAXNODE

MAXNODE=500000 is appropriate for normal use. MAXNODE=37000000 was tried unsuccessfully SC for 14 electrons. In this case, gfortran compilation has to be with "-mmodel=medium" (for GAMESS / VB2000 it has to be done for all GAMESS routines, not just vb2000. This, however, is not recommended).

SC (SCGVB) for 12 electrons in 12 orbitals is OK, but SC for 12 electrons in more or less than 12 orbitals may need a larger MAXNODE. MAXNODE=20000000 may be the largest that we can use.

MAXNODE is used at the routines VBMATR and VBHESS to set the size of the array KTNODE. It has a maximum theoretical value which depends on (N,NSTR,IDERANK), but often the maximum index of KTNODE is much less than this value. From VB2000 3.0, the array KTNODE is dynamically allocated. An upper bound of 500000 (GAMESS version) and 1000000 (stand alone version) was set. The dynamic allocation was implemented by request of the GAMESS development team (for version 2021 R2) in order to avoid linking issues with the gfortran compiler.

NW1

NW1=300000 was used earlier, although NW1=5000000 is needed for SCGVB(12) NW1=5800000 is the default for VB2000 2.8. However this may not be sufficient for CASVB(10,10).

SCGVB for anything over 12 electrons would require much larger values of NW1 and very long run times. However, SC with CIONLY starting from a good set of VB orbitals can get close to the full SC. For example, the pi system of naphthalene with 12 electrons and 42 SC structures has a energy of -383.49424637, while the CIONLY starting from the VB orbitals of 3 Kekule-like structures (energy -383.49289121) gives -383.49414658.

The 4 Kekule-like structures for anthracene gives an energy of -536.19157404 while the 420 SC structures with CIONLY gives -536.19423937. All these calculations are with the cc-pVDZ basis set and the GAMESS / VB2000 version.

Increasing either MAXNODE or NW1 in GAMESS/VB2000, from the 2016 GAMESS version on, leads to excessive memory use and fails to link.

NW1 is used at the routine GSYMBO to set the size of the arrays NW and NPATN. It has a well-defined maximum theoretical value which depends on (N, NSTR, IDERANK). Up to VB2000 3.0, the arrays NW and NPATN are dynamically allocated, so NW1 is not used anymore. The dynamic allocation was implemented by request of the GAMESS development team (for version 2021 R2) in order to avoid linking issues with the gfortran compiler.

If somehow you still have problems with linking the GAMESS executable with VB2000 using large parameter values, you can try this simple hack: set EFP_MXFRG to 10 (it is 1050) in mx_limits.src and MXLMO to 5 (it is 50) and NFRQ to 2 (it is 12) in efpmodule.src. This routine uses a massive amount of memory. The hack essentially disables EFP, but allows GAMESS to link properly. This should be done if you experiment with values of MM > 12500000 in the GAMESS version.

MM

MM controls the maximum size of the arrays related to reading the MAP files. Note that MM is set in 5 subroutines and it must be set in all of them with the same value. MM is determined entirely by MAP files, not by the number of structures.

MM=500000 is sufficient for MAP files in the GAMESS release, up to 12 electrons in a singlet group. MM=2500000 is the default for the stand alone version. If you want to use larger MAP files from the VB2000 stand alone in the GAMESS / VB2000 version, you have to alter MM in vb2000.src as specified below.

If the multiplicity is greater than 1, you may also need to increase MM. A warning will be given if MM is not large enough.

For up to 14 electrons in singlet use – use MM=2500000
For up to 16 electrons in singlet use – use MM=12500000
For up to 18 electrons in singlet use – use MM=62500000
For up to 20 electrons in singlet use – use MM=600000000
For up to 22 electrons in singlet use – use MM=6000000000

For convenience, the value of MM used in the compile is printed in the VB2000 banner.

Using a moderate computer (16GB core memory, 100GB disk space), VB2000 stand alone version can handle up to 20 electrons in a VB group. N=22 is also possible if you have a robust computer with a lot of core memory (80GB+). A test calculation using 22 electrons was performed by JL using a machine running CentOS (Linux) with 16 cores and 250GB RAM. This and other test input files (for 16, 18 and 20 electrons) are available under personal request.

BD has made an investigation into how many electrons can be used in the SCGVB code.

For a singlet state, it appears that SCGVB(12) is the highest and a maximum of 12 is set in the code (You can alter it if you want to try larger cases). SCGVB(n,m) has more structures than SCGVB(n), so the limit there is likely to be lower. For odd-electron systems the number of structures for doublet SCGVB(n-1) is equal to that for singlet SCGVB(n), so doublet SCGVB(11) may be possible. SCGVB(14) is not possible, but close to the cusp. If the code were made parallel and improved, and then run on a faster machine, it might run. Currently (2016) we estimate it may take about 100 days on an Intel i7 computer.

The problem is largely the very rapid increase in the number of structures, which his shown below for doublet odd and singlet even systems:

No. of electrons.	No. of structures.
5, 6	5
7, 8	14
9, 10	42
11, 12	132
13, 14	429
15, 16	1430
17, 18	4862
19, 20	16796
21, 22	58786
23, 24	208012
25, 26	742900
27, 28	2674440
29, 30	9694845

It is suggested that SC calculations be started from the orbitals from a simpler calculation. For example if the PP-GVB structure is expected to be the dominant one, one could start from a VB(2n) calculation with one structure (removing the strong orthogonality condition from the n pairs). Intermediate between this single structure and SCGVB(2n) is to optimise the coefficients of the structures and not the orbitals. This is achieved using SCGVB(2n) in the command line along with CIONLY. This is significantly faster than the full SCGVB which optimises both the coefficients and the orbitals. However, the energy is higher than the full SC. SCGVB(14) with CIONLY is possible while full SC as we explain above is not. We are grateful to David Sousa for pointing out the advantage of this use of CIONLY. Using CIONLY as an initial guess may also speed up full SCGVB.

This should be put into the context of other work. Using his CASVB code in MOLPRO, David Cooper (personal communication) has done SCGVB(14) but it took a lot of effort and computer time. Jeppe Olsen has recently published a new method ([J. Chem. Phys. 143, 114102, 2015](<https://doi.org/10.1063/1.4929724>)) reporting a

SCGVB(18) calculation for the chromium trimer. He reported in 2016 (personal communication) that the present limit is SCGVB(20), but hoped to get up to SCGVB(24) in 2016 and SCGVB(30) in 2017. We are not aware that he has achieved this. This will be an amazing achievement if he does.

11 References

VB2000 Algorithms

1. Jiabo Li and Roy McWeeny, VB2000: Pushing Valence Bond Theory to New Limits, *Int. J. Quant. Chem.*, 89 (2002) 208.
2. Jiabo Li, Brian J. Duke, Thomas M. Klapötke and Roy McWeeny, Spin-Density of Spin-Free Valence Bond Wave Functions and Its Implementation in VB2000, *J. Theor. Comp. Chem.* 7 (2008) 853.
3. Jiabo Li, and Ruben Pauncz, Efficient Evaluation of the Algebrants of VB Wave Functions Using the Successive Expansion Method. I: Spin $S = 0, 1/2$. *Int. J. Quant. Chem.*, 62 (1997) 245.
4. Jiabo Li, Graphical Representation of a New Algorithm for Nonorthogonal Ab Initio Valence Bond Calculations, *Theor. Chim. Acta*, 93 (1996) 35.
5. Jiabo Li, New Algorithm for Nonorthogonal Ab Initio Valence Bond Calculations II: Subgraph-Driven Method, *J. Math. Chem.*, 17 (1995) 295.
6. Jiabo Li, and Wei Wu, New Algorithm for Nonorthogonal Ab Initio Valence Bond Calculations. I: New Strategy and Basis Expressions, *Theor. Chim. Acta*, 89 (1994)
7. Peter B. Karadakov, David L. Cooper, Brian J. Duke, and Jiabo Li, "Spin-Coupled Theory for 'N Electrons in M Orbitals' Active Spaces", *J. Phys. Chem. A*, 116, 7238 - 7244, 2012.

Group Function Theory

8. Roy McWeeny, The Density Matrix in Many-Electron Quantum Mechanics I. Generalized Product Functions. Factorization and Physical Interpretation of the Density Matrices, *Proc. R. Soc. London, Ser A* 253(1959) 242.
9. János G. Ángyán, Chemical Building Blocks in Quantum Chemistry Calculations. Perspective on "The Density Matrix in Many-Electron Quantum Mechanics I. Generalized Product Functions. Factorization and Physical Interpretation of the Density Matrices", *Theor. Chem. Acc.* 103 (2000) 238.
10. Roy McWeeny, Separability of Quantum Systems: A Density Matrix Approach, *Adv. Quantum Chem.*, 31 (1999) 15.

Applications of VB2000

11. Anton Hammerl, Thomas M. Klapötke, Heinrich Nöth, and Markus Warchhold,

- Synthesis, Structure, Molecular Orbital and Valence Bond Calculations for Tetrazole Azide, CHN₇, *Propellants, Explosives, Pyrotechnics* 28 (2003) 165.
12. T. M. Klapötke, Nichtmetallchemie, in: *Moderne Anorganische Chemie*, E. Riedel (ed.), Walter de Gruyter, Berlin, New York, 2nd edition, **2003**. 3rd edition, **2007**.
 13. Aggelos Avramopoulos, Herbert Reis, Jiabo Li, and Manthos G. Papadopoulos, The Dipole Moment, Polarizabilities, and the First Hyperpolarizabilities of HArF. A Computational and Comparative Study, *J. Am. Chem. Soc.*, 126 (2004) 6179.
 14. Thomas M. Klapötke, Jiabo Li, and Richard D. Harcourt, Ab initio Double- ζ (D95) Valence Bond Calculations for the Ground States of S₂N₂ and S₄²⁺, *J. Phys. Chem.*, 108 (2004) 6527.
 15. Thomas M. Klapötke, Richard D. Harcourt, and Jiabo Li, Ab initio Double- ζ (D95) Valence Bond Calculations for the Ground States of NO₂, O₃ and ClO₂, *Inorg. Chim. Acta*, 358 (2005) 4131.
 16. R. D. Harcourt, T. M. Klapötke, J. Li, *Inorg. Chim. Acta*, **2005**, 358, 4049 – 4050.
 17. D. Benker, T. M. Klapötke, G. Kuhn, J. Li, Ch. Miller, An Ab Initio Valence Bond (VB) Calculation of the π Delocalization Energy in Borazine, B₃N₃H₆ *Heteroatom Chemistry*, **2005**, 16(5), 311 - 315.
 18. A. Avramopoulos, L. Serrano-Andrés, J. Li, H. Reis, M. G. Papadopoulos, *J. Chem. Phys.*, 127 (2007) 214102.
 19. L. Serrano-Andrés, A. Avramopoulos, J. Li, P. Labeguerie, D. Begue, V. Kello and M. G. Papadopoulos, Linear and nonlinear optical properties of a series of Ni-dithiolene derivatives, *J. Chem. Phys.*, 231(2009) 134312.
 20. Brian J. Duke and Remco W.A. Havenith. Implications of the complete basis set limit in valence bond theory: A case study of molecular hydrogen, *Theor. Chem. Acc.*, 135, Issue 3, March 2016.

Structure Weight Analysis

21. B. H. Chirgwin, and C. A. Coulson, *Proc. R. Soc. Lond., Ser. A* 201 (1950) 196.
22. P. O. Löwdin, *J. Mol. Struct. (THEOCHEM)* 229 (1991) 1.
23. G. A. Gallup and J. M. Norbeck, *Chem. Phys. Lett.* 21 (1973) 495
24. P. C. Hiberty, G. Ohanessian, *Int. J. Quantum Chem.*, 27 (1985) 245.

BOVB Theory

25. Philippe C. Hiberty and Sason Shaik, BOVB - A Modern Valence Bond Method that Includes Dynamic Correlation, *Theor. Chem. Acc.* 108 (2002) 255.

ORMAS (Occupation Restricted Multiple Active Space)

26. W. Humphrey, A. Dalke and K. Schulten, VMD - Visual Molecular Dynamics, *J. Mol. Graphics*, 14(1996)33.
27. Joseph Ivanic, CI and MCSCF Method for Multiple Active Spaces with Variable Occupations. I. Theory. *J. Chem. Phys.* 119 (2003) 9364.
28. Joseph Ivanic, Direct CI and MCSCF Method for Multiple Active Spaces with Variable Occupations. II. Application to oxoMn(salen) and N₂O₄. *J. Chem. Phys.* 119 (2003) 9377.

VMD (Visual Molecular Dynamics)

29. W. Humphrey, A. Dalke and K. Schulten, VMD - Visual Molecular Dynamics, *J. Mol. Graphics*, 14 (1996) 33.

Newest Applications of VB2000

30. André Gustavo H. Barbosa, André M. Henriques, João G. S. Monteiro *et al.* The multiconfiguration Spin-Coupled approach for the description of the three-center two-electron chemical bond of some carbenium and nonclassical ions. *Theor. Chem. Acc.* 137 (2018) 21.